
Safe Parallel Programming Language

Development process of ParaLang

- Project P4 -

Project Report

Group CS-26-SW-4-15

Anders Terpager Hansen

Daniel Alberg Ibrahim

Emil Bæk Ravnholt

Jonas Bork Gammelgaard

Mikkel Aagaard Bielefeldt

Ulrik Kryhlmand Christensen

Aalborg University



Software
Aalborg University
<http://www.aau.dk>

AALBORG UNIVERSITY

STUDENT REPORT

Title:

Safe Parallel Programming Language.

Theme:

Developing a Safe and Simple Parallel Programming Language.

Project Period:

Spring Semester 2026

Project Group:

CS-25-SW-4-P4-15

Participants:

Anders Terpger Hansen
Daniel Alberg Ibrahim
Emil Bæk Ravnholt
Jonas Bork Gammelgaard
Mikkel Aagaard Bielefeldt
Ulrik Kryhmand Christensen

Supervisor:

Oliver Steen Bak

Page Numbers: 113**Date of Completion:**

2026-05-25

Abstract:

This report presents the development of ParaLang, a safe and simple parallel programming language aimed at tackling key challenges in concurrent computing. With Moore's Law slowing due to physical limits, utilizing multi-core processors effectively has become essential. However, parallel programming often leads to errors such as race conditions, deadlocks, livelocks, starvation, and granularity issues. ParaLang addresses these through built-in safety features, including threads, mutex-protected critical sections, await constructs, safe arrays, and synchronization primitives. ParaLang was implemented with ANTLR4 for parsing and a visitor-based translator targeting Java code, following MoSCoW requirements. The report covers problem analysis, language design, implementation, testing, and evaluation of the project. The resulting prototype enables safer parallel programming with reduced concurrency risks while maintaining simplicity for practical applications.

The content of this report is freely available, but publication (with reference) may only be pursued due to agreement with the author.

Contents

Preface	1
1 Introduction	2
2 Problem Analysis	3
2.1 Introduction to Parallel and Distributed Computing	4
2.2 Concurrency Correctness Errors	9
2.3 Parallel Performance Issues	17
2.4 Parallel Programming	18
2.5 Existing Solutions	23
2.6 Summary	28
3 Requirements for ParaLang	29
3.1 Must Have	30
3.2 Should Have	32
3.3 Could Have	33
3.4 Won't Have	34
4 Design	35

Contents

4.1	Language Design Decisions	35
4.2	Syntax & Semantics	37
5	Development	56
5.1	Grammar and ANTLR4	56
5.2	Compilation Context	65
5.3	Visitor Pipeline	66
5.4	Critical Sections and Mutexes	68
5.5	Thread declaration and Usage	68
5.6	Await Constructions	69
5.7	Read Construction	70
5.8	Array Declaration and Usage	71
5.9	Casting Construction	77
5.10	Scopes and Indentation	79
5.11	Implementation and demo of the Solution	82
6	Testing	87
6.1	Unit Testing	87
6.2	Integration Testing	90
6.3	Acceptance Testing	94
7	Discussion	100
7.1	Discussion: Design	100
7.2	Discussion: Implementation	100
7.3	Something about antlr	107

Contents

8 Conclusion	109
Bibliography	110
9 Appendices	114
9.1 Context Free Grammar	114
9.2 Syntax and Semantics	119
9.3 Acceptance Testing Test Cases	141
9.4 ParaLang Factorial code example	154
9.5 Java Factor Output Code	156

Preface

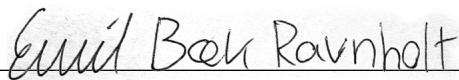
Aalborg University 2026-05-25



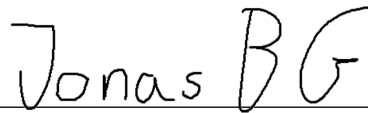
Anders Terpager Hansen
<atha24@student.aau.dk>



Daniel Alberg Ibrahim
<dibrah24@student.aau.dk>



Emil Bæk Ravnholt
<eravnh24@student.aau.dk>



Jonas Bork Gammelgaard
<jgamme24@student.aau.dk>



Mikkel Aagaard Bielefeldt
<mbiele24@student.aau.dk>



Ulrik Kryhmand Christensen
<uchris23@student.aau.dk>

Chapter 1

Introduction

Intro here

Chapter 2

Problem Analysis

A well-known and frequently cited law in computer engineering is Moore's Law, which was proposed in 1965 by Gordon Moore, an engineer and chemist who was, at the time, the Director of Research and Development at Fairchild Semiconductors and who later co-founded Intel.

The law describes an observed trend in hardware development. It states that the number of transistors in microchips tends to double approximately every two years, depicted in [Figure 2.1]. This growth has historically resulted in increased computational performance and more powerful processors while also reducing the cost per transistor. However, this trend has begun to approach physical limitations, making it increasingly difficult to further minimize transistor sizes [1].

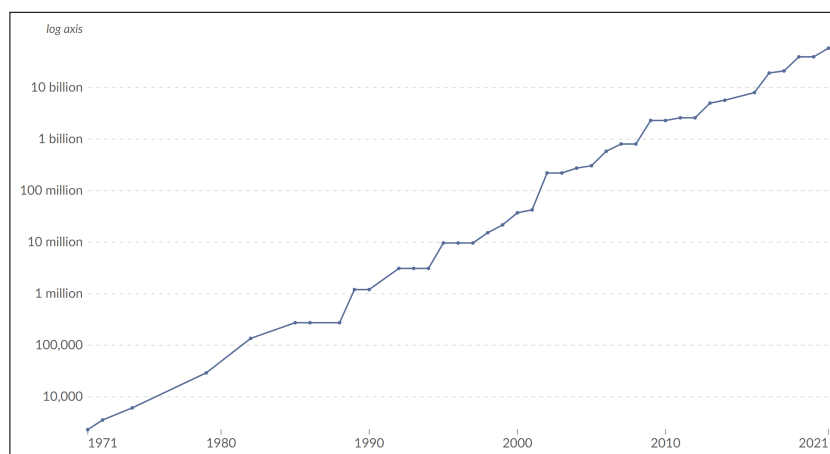


Figure 2.1: Number of transistors per current generation microprocessor from 1971-2021 [2].

The reason for the slowdown is the physical constraints involved in making smaller transistors. The technology and methodology currently used are limited by the rising cost of production, the current level of scientific advancement, fundamental physical limits and laws of physics at the scale at which transistors operate, and the resulting movement of electrons [3, pp. 42–43].

With Moore’s law trend ceasing to be relevant due to the fundamental limitations, alternative avenues for performance improvements must be explored [3, pp. 42–43]. Computing performance gains now increasingly rely on better software optimization, more efficient algorithms, hardware architectures, and the adoption of multi-core CPUs [4]. Therefore, approaches that make use of parallel- and distributed computing have become essential for fully utilizing the already existing hardware and take full advantage of the multiple cores that exist in modern CPUs.

2.1 Introduction to Parallel and Distributed Computing

Parallel computing assigns sub-tasks across multiple threads running on different CPU cores of a single machine. In contrast, distributed computing functions as a collaborative system where multiple independent machines communicate with each other over a network and coordinate task assignments using algorithms.

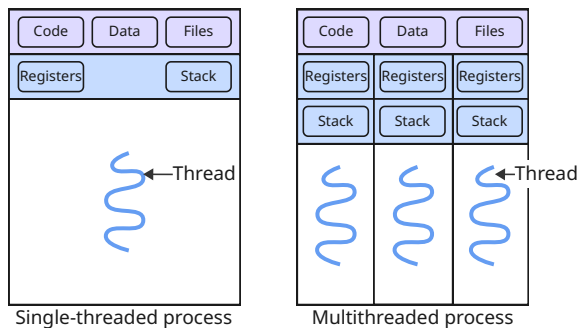


Figure 2.2: Illustration of single- and a multi-threaded process

A process is an instance of a program running with its own address space and resource allocation, while a thread is a unit that executes within a process and shares the allocated memory, but runs its own independent execution flow. Threads function as the computer’s fundamental mechanism for multitasking, also called multi-threading, within a program [Figure 2.2]. A thread follows a well-defined life-cycle, typically passing through several states such as: new, ready,

running, blocked, and terminated [Figure 2.3]. The transition between these different states is handled by the thread scheduler, which is an operating system (OS) level component. It ensures that the threads utilize the CPU efficiently, minimizing downtime between tasks while accounting for the individual thread priorities. Various operating systems have different thread schedulers, each functioning in its own way. This report focuses specifically on the Windows thread scheduler [5, ch. 2]. When a thread is created, it enters the Ready state, where it waits until the thread scheduler selects it to run. Once

running, the thread may become blocked if it must wait for shared resources, among other things. When the wait is over, it returns to the Ready state. After completing its work, it terminates. This model also makes it easier for the scheduler to manage thread allocation and execution at any time [6, Chapter 4, pp. 160–162]. Parallel computing can function on a single machine or as a combination of the two approaches; this is called distributed parallel computing, also known as cluster computing [7, pp. 218–220].

Some challenges that arise when working with these archetypes in combination with multi-core parallelism are nondeterminism. When using only a single thread, a task is deterministic, its start and end are determined from the beginning, and the path is already predicted, meaning there is always only one order in which the task can be done. A nondeterministic task makes use of multiple threads that, in combination, try to finish a task, and whose execution order is unpredictable due to how the sub-tasks are scheduled. Because threads can be interrupted and assigned a new task before finishing its sub-task, this results in the context that is used in the execution of the task getting reassigned and deprioritized in the cache [Figure 2.4]. This means that some parts of the task finish before others, and it is only when everything has finished that the task is done. The consequence of this is that the final result can differ between runs, even if the tasks are assigned correctly, which makes it complicated to guarantee correct behavior and the assignment of tasks [6, pp. 131]. An example could be two threads who want to update the same variable at the same time. If one of the threads reads the value while the other thread is updating the variable, the result could vary depending on the execution order.

A simple way to illustrate nondeterminism is to imagine two people, Abe and Bob, sending print jobs to the same shared printer at nearly the same time. In a deterministic system, the printer processes jobs in a fixed, predictable order, e.g. Abe's job first, then Bob's. In a nondeterministic concurrent system, the order depends on unpredictable timing and scheduling, Bob's job might start first, even if Abe sent his slightly earlier, or Abe's might finish before Bob's begins, leading to different completion orders on different runs, even though both jobs are identical and correctly submitted [8].

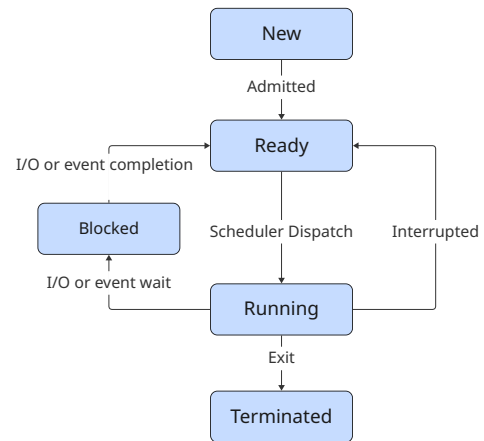


Figure 2.3: Thread lifecycle model showing the five states: new, ready, running, blocked, and terminated

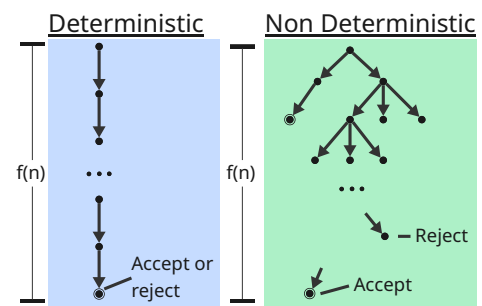


Figure 2.4: Illustration of deterministic and non-deterministic operations

2.1.1 Distributed Computing

In distributed computing, multiple autonomous machines, ranging from physical hardware and its components to microservices and virtual instances of an operating system that run instructions, are loosely coupled to present a unified, powerful, and reliable system. Although the overall system appears as reliable as a single-threaded process, the communication between the individual processes can be unreliable due to the risk of packet loss and network latency, among other things. Such failures are expected to occur and are isolated through the loosely coupled connections between the different components. As a result, when one component stops working, it can happen silently without stopping other processes that are already underway or preventing the task from being reassigned to a different system. In this way, the system maintains a consistent and dependable front despite internal faults [9, p. 1–2].

By using this approach, more than one service needs to stop working before the system fails and the services stop running. This ensures the reliability of the system and provides fault tolerance. Here, a fault means any unexpected stop of a component, this could be a crash, network issue, or hardware faults. This allows processes to stop silently without affecting the overall operation, by distributing the load and operations through multiple pipelines. While this behavior might be a disadvantage in most small scale cases, it is crucial in large scale infrastructures where continuous up-time is required. Such systems often aim for 99.99999% up-time, often also called “seven nines”. Beyond reliability, distributed architectures are also used for performance and scalability. For example, Google or Amazon would never be able to handle billions of simultaneous requests just using a single machine. These goals are achieved through distributed computing architectures [10].

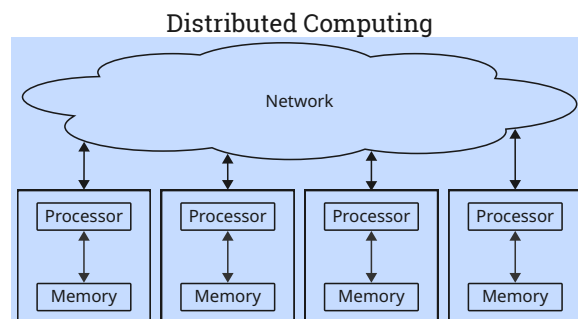


Figure 2.5: Illustration of distributed computing

Because the system is distributed across multiple machines, factors such as network latency and communication overhead must be considered when evaluating processing time and slowdown of tasks. These slowdowns often occur due to the physical distance between machines, network traffic through the cables, and the need to coordinate scheduling and resource access across multiple systems. This can result in slower theoretical operations, but for large-scale data sets, the distribution can still result in faster computations [11].

One of the main advantages of distributed computing is its scalability and horizontal expansion capabilities during installment and expansion after deployment. Because the

different machines in the system run their own processes and tasks, additional machines can be introduced into the system to share the workload. When new machines are deployed, they can automatically be assigned tasks and begin contributing to the overall computation. This differs from vertical scaling, where a single machine is upgraded.

2.1.2 Parallel Computing

In parallel systems, multiple processors operate under a single operating system and share the same physical memory, represented in [Figure 2.6]. This tightly coupled architecture enables extremely fast communication between tasks, as data is exchanged directly through memory rather than over a network. As a result, parallel computing typically offers lower latency and higher throughput for communication-heavy tasks than distributed systems. This makes it well-suited for tightly coupled and frequent data-sharing workloads.

In parallel computing, tasks can be split into smaller subtasks that can be executed simultaneously by multiple threads. A thread is the smallest unit of execution within a process; it represents a sequence of instructions that can be scheduled independently. Each thread contains its own program counter, register set, and stack [6, ch. 4]. In this context, the term thread refers to software threads managed by the operating system, not hardware threads.

When multiple threads need to operate on shared data, it is sometimes necessary to coordinate their access to ensure that operations on the data are handled in a predictable and well-defined manner. The challenges that are involved, along with various relevant synchronization strategies, are discussed in [Section 2.2.1].

In order to manage multiple threads, the operating system uses the thread scheduler described in [Section 2.1]. It determines which threads are run on which CPU cores and when each thread is executed. This execution order can vary from run to run, making the order of execution unpredictable [6, ch. 5].

The shared memory model introduces challenges for scalability. As multiple processors try to access data at the same time, they compete for resources, such as memory bandwidth, synchronization primitives, and cache contention. This can result in delays, race conditions, and unpredictable execution times, which can become more pronounced

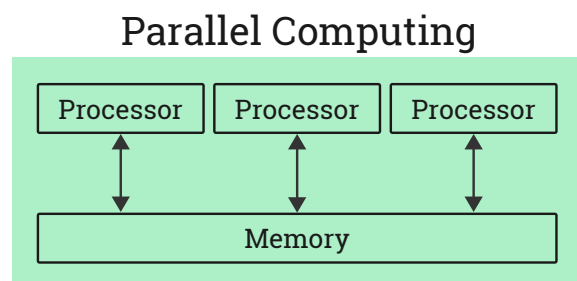


Figure 2.6: Illustration of Parallel Computing.

as the number of processes increases [12, pp. 13-16].

Parallel architecture is constrained by the physical limits of a single machine. CPU cores, cache sizes, and memory capacity cannot be expanded indefinitely, meaning performance improvements eventually require architectural redesign rather than horizontal scaling. Modern multicore systems further introduce non uniform memory access (NUMA), where memory access times vary depending on the physical processor placement, potentially degrading performance if threads and data are not carefully managed [12, pp. 9, 15].

Despite these limitations, parallel computing remains essential for high-performance workloads, such as scientific simulations, numerical algorithms, and real-time processing. Its ability to exploit shared memory for rapid communication makes it highly efficient for tasks that cannot be easily decomposed across multiple machines, distinguishing it from distributed systems that prioritize fault tolerance and horizontal scalability.

Simultaneous Multithreading

Modern processors benefit greatly from multithreading thanks to simultaneous multithreading (SMT). SMT usually lets two independent threads run at the same time on one physical core. It is used to make better use of the processor resources that would otherwise do nothing, waiting for instructions. By allowing multiple independent threads to execute simultaneously on the same physical core, SMT helps to fill in these idle gaps, keeping execution units busy, increasing instruction throughput, and thereby higher performance within a system. SMT works by sharing almost all the execution units while duplicating only the bare minimum state needed so each thread thinks it's running on its own [13].

The key part is duplicating the architectural registers that the software actually sees. Each logical processor gets its own full set of general-purpose registers, control registers, program counter, flags, and so on. That way, the OS and applications treat them just like separate cores [14].

Because the hardware already keeps those separate copies of the architectural state (registers, program counter, flags, and so on) in the L1 and L2 cache for each logical thread, switching between threads on the same core is really fast. No big save/restore operations are needed for all that architectural state, unlike what happens during a full context switch between different physical cores. The processor can simply select the next ready thread and continue execution in the following cycle with almost zero overhead.

Meanwhile, the much larger physical register file used for register renaming remains

shared among all threads on the core. The renaming logic maps each thread's architectural registers onto distinct entries in this common pool to avoid name conflicts between threads [15].

The execution units are shared too. These are hardware blocks specialized and optimized for fast specific operations, like integer adders, floating-point units, load/store pipelines, and branch units. The core can hand them out, cycle by cycle, to whichever thread has instructions ready. A cycle is one tick of the processor clock, the basic time unit during which the core performs a small step of work. That helps hide latencies that would otherwise leave a single-threaded core sitting idle[15].

Modern processors organize their memory system as a cache hierarchy. This hierarchy consists of multiple levels, each balancing speed against capacity. The L1 cache sits closest to the execution units and is the smallest but fastest. It is typically split into separate instruction and data caches. In most SMT designs, the L1 caches are shared between the logical threads on the same physical core. The next level, the L2 cache, is also usually shared within the core, while L3 cache is shared across different physical cores. This shared hierarchy lets one thread see data written by its sibling almost instantly through the common cache lines, which improves throughput and helps hide memory latency. At the same time, it creates contention for the limited space in each level. One thread's working set can evict lines needed by the other thread, raising miss rates and introducing extra stalls. Some implementations therefore use dynamic allocation inside the shared structures to reduce these fights [15].

SMT is a smart trade-off. You don't pay a huge hardware penalty, but you still get much better throughput on multithreaded workloads. When one thread stalls on a long-latency operation or fails, the other can keep the functional units busy. Single-thread performance usually takes only a small hit, if any, from the extra scheduling complexity and occasional resource fights [15].

What makes SMT so clean is that it duplicates just enough to keep threads isolated at the architectural level while sharing the expensive parts of the data path as much as possible. In the end, one physical core acts like two (or more) logical ones, pulling both instruction-level and thread-level parallelism out of whatever workload is running [15].

2.2 Concurrency Correctness Errors

When working with multiple concurrent processes operating on the same problem, whether through distributed computing or parallel computing, assuming that they complete their tasks in a certain order would be incorrect. Thread scheduling may cause

one process to finish first for one run of the solution and another process to finish first for the next execution. When these concurrent processes access and change the same location in memory, issues can occur and some possible sequences for the parallel processes that interact with the data stored may cause the data to be ruined [12].

Before going further into the different issues that could occur if the right precautions have not been implemented, locks need to be explained. A lock is often like a boolean variable used to protect critical sections of an application and regulate which threads can access these critical sections. It does this by making the threads check if the boolean is false, and if it is, the thread "acquires" the lock by setting it to true. This makes it so no other threads are able to occupy the lock until the current thread is finished and "releases" it by setting it to false again. The lock can be categorized into two primary types, the first being an exclusive lock that allows only one thread to enter and use a critical section at once. The second type is a reader-writer lock that allows multiple threads to read a critical section together or allows only one thread to edit a critical section at once, also called a writer thread [12, p. 588]. The word 'mutex' comes from 'mutual exclusion', and is the name of a software abstraction that is used for exclusive locks, such as the Linux-kernel spinlock [12, p. 39]. Many languages have existing standard libraries that implement mutex as a class, such as C++ `#include <mutex>` [16], or Python threading library `threading.Lock()` [17]. In terms of reader-writer locks, the name typically used is `rwl`, `rwlock`, or another form of abbreviation of the reader-writer lock [12, p. 34].

2.2.1 Race Conditions

Race Conditions refer to when multiple concurrent processes interact with the same memory, though the term is often used when undesirable interaction takes place. This is usually in the form of a Data Race, a condition where more than one thread of execution attempts to access the same memory location, and at least one of those accesses is a write operation. There are two possible outcomes of this: Either the read operation will occur first, reading the original data from memory before the new data is written, or the write operation occurs first, meaning the following read operation will instead read the new data. This means that depending on which thread of execution accesses the memory location first, there will be a different result, thus the term "race" [12, 18].

Algorithm 1 Race Condition with Parallel Threads

```

1: counter  $\leftarrow$  0

2: function INCREMENT
3:   temp  $\leftarrow$  counter
4:   counter  $\leftarrow$  temp + 1
5: end function

6: function MAIN
7:   newThread(1) : INCREMENT
8:   newThread(2) : INCREMENT
9: end function

```

In Algorithm 1, *counter* is initialized to 0, and two threads are spawned, each calling *increment*. A race condition can occur in the following interleaving: Thread 1 reads *counter* into *temp*, obtaining 0. Before it can write back, Thread 2 also reads *counter*, also obtaining 0. Thread 1 then writes *temp* + 1 = 1 to *counter*, followed by Thread 2 writing its own *temp* + 1 = 1. Despite two increments being performed, *counter* ends up as 1 rather than the expected 2, because Thread 2's read happened before Thread 1's write, causing the first increment to be lost.

A few approaches exist that attempts to solve the problem of race conditions. Having the CPUs or threads not share any memory would a possible solution, although a rather extreme and undesirable one. A better solution with the same methodology of non-interaction would be having the only shared memory be read-only/immutable, both options preventing data races entirely. These both suffer from limiting the functionality of splitting up the program however. If the concurrent processes need to write to some shared variable, to ensure that they do not write at the same time, resulting in scrambled data, using atomic variables is usually the solution. These data types ensure that each interaction is performed indivisibly, meaning no other interactions can overlap. While this removes the possibility of two values being mixed together to make scrambled data with no meaning, it does not solve the issue of data races and atomicity violations [Section 2.2.4].

Another very popular solution lies in the use of locks. Locking variables to prevent multiple threads from interacting with them at once, or locking entire segments of memory while a CPU or thread executes a critical section is a common solution to race conditions. This however introduces other issues when attempting to manage these locks [Section 2.2.2].

2.2.2 Deadlocks

Deadlocks are issues that occur when at least two threads get into a situation where they wait for each other indefinitely. This results in a form of stalemate that will never be lifted because each thread is waiting to get the mutex that the other thread has yet to unlock. This could also occur on a single thread if said thread locked an address and then at a later stage before unlocking the address if it tries to occupy and wait for it to be unlocked, it would result in an unending wait. A deadlock scenario will always contain at least one deadlock cycle that goes from a thread in a cycle towards itself again, with one or more locks and threads in between [12, p. 101-103]. This is also illustrated with [Figure 2.7], that creates a deadlock cycle.

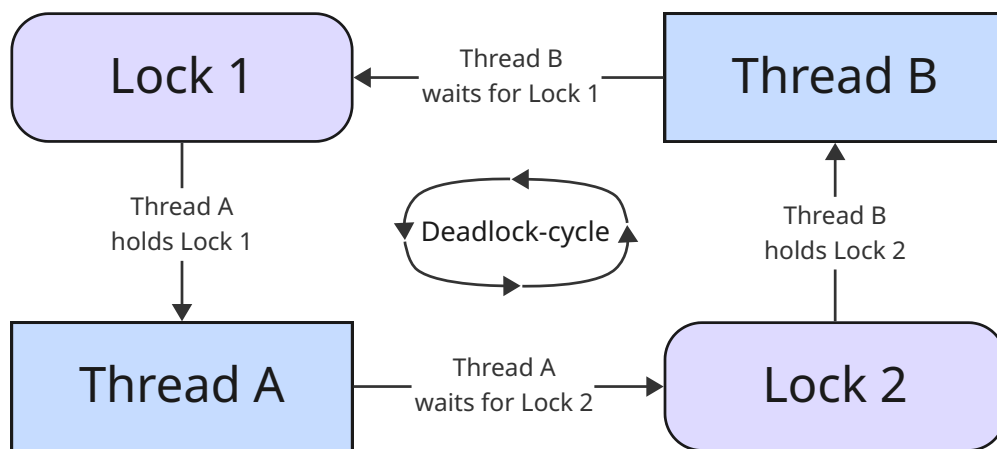


Figure 2.7: Illustration of a deadlock cycle.

An approach to recover from a deadlock would either be to kill the threads or to have the lock forcibly stolen from one of the threads. The right approach would be to avoid deadlocks in its entirety because dealing with the resulting partially updated structures can be extremely complex, hazardous, and error-prone.

Deadlock avoidance strategies

- **Locking hierarchies** order locks in a numerical order forbidding a thread from acquiring a given lock if it already holds a lock with the same or a higher number. It is a powerful strategy in terms of preventing a deadlock, however, maintaining a strict global hierarchy in larger programs can be difficult without automated tools like the Linux-kernel *lockdep*[19] to enforce the order [12, p. 103].

- **Local locking hierarchies** prevent deadlocks by releasing all of the thread's locks before invoking unknown code, such as user-provided callback or function. The strategy is designed for modularity, especially when accessing libraries, and hereby avoids potential deadlock cycles [12, p. 103-104].
- **Layered locking hierarchies** are mainly used when a lock cannot be released by a thread before invoking unknown code. Instead of releasing the lock, it establishes a layered hierarchy in which functions are assigned to specific layers, and the thread is only allowed to acquire the locks from the higher layers after all of the locks from the layers below have been acquired. This results in a use case where an important lock can be maintained while invoking an unknown code [12, p. 104-106].
- **Temporal locking hierarchies** is a strategy that is used when a thread is trying to acquire a lock that has already been acquired by another thread, preventing a risky environment. The strategy will arrange for the lock to be acquired in an environment where the thread does not maintain any other locks, either using a timer handler or a scheduler-clock interrupt handler preventing deadlocks [12, p. 106-107].
- **Locking Hierarchies and Pointers to Locks** is used if a thread is using an API that has a pointer to an internal lock, which is in general the antithesis of information hiding. Such an API is generally wrongly designed and should be avoided. There are, although rare, exceptions, such as *pthread_cond_wait()* where passing the mutex [Section 2.2] is necessary to avoid lost wakeups [12, p. 107].
- **Conditional Locking** can be used if there is a need to have a hierarchical order with two locks, meaning if the thread wants to acquire a lock that is hierarchically beneath its current lock, it will have to release the current lock and reacquire both in the correct hierarchical order. However, it can lead to livelock [Sub 2.2.3] or starvation [Sub 2.3.1] if the attempt repeatedly fails to acquire the locks in the correct order. To avoid livelock it should be used in conjunction with exponential backoff, waiting longer for each failed attempt before trying to acquire the lock again [12, p. 107].
- **Acquire Needed Locks First** is a similar case to Conditional Locking, as such it also has the possibility to livelock [Sub 2.2.3]. Within this strategy, the difference between it and Conditional Locking is that instead of releasing a single lock to acquire one lock, it will release all of its locks to acquire a single lock. It will then reacquire all the released locks along with the single lock. The small change in this strategy is that it will acquire all of the necessary locks before starting to process anything. Therefore, the strategy is used in scenarios where all necessary locks can be acquired before starting any processing, such as transactional systems [12, p. 107-108].

- **Single-Lock-at-a-Time Designs** is as the name suggests a strategy where a thread will only be able to hold a single lock at a time, eliminating all possibilities of a deadlock. However, this strategy requires mechanisms, such as the existence guarantees [12, p. 117-118], which ensure that the data structures are maintained during the time the lock is not held [12, p. 108].
- **Signal/Interrupt Handlers** is a strategy that is used to avoid deadlocks when using handlers, such as signal or interrupt handlers. The strategy prevents self-deadlock where a handler interrupts mainline code and attempts to acquire a lock that the mainline code already holds. When the mainline code shares a lock with a handler, it blocks signals or disables interruptions when acquiring any lock that might be acquired within a signal or an interrupt handler. When holding this form of lock and the handler tries to acquire a lock that is ever acquired outside of a signal handler without blocking signals, it would be determined illegal and it should not be implemented this way. However, this strategy would be expensive for some operating systems, such as Linux and would result in that all communication between the application code and the signal handlers is done using lockless methods [12, p. 108].

Although there is no deadlock-avoidance strategy that works perfectly for all situations, a good selection of strategies is available, each determined by the situation and needs.

2.2.3 Livelock

A livelock is when multiple threads repeatedly interfere with one another, causing them to make no progress. This is inherently a more dynamic issue than a deadlock, as the threads still seem "alive", they are simply caught in an endless loop of mutual sabotage, making it in some regards a harder issue to identify, as these interference cycles can span many operations before looping. Furthermore, as they are actively working and not simply waiting on a lock, it also eats up processing power while remaining stuck [12, p. 109].

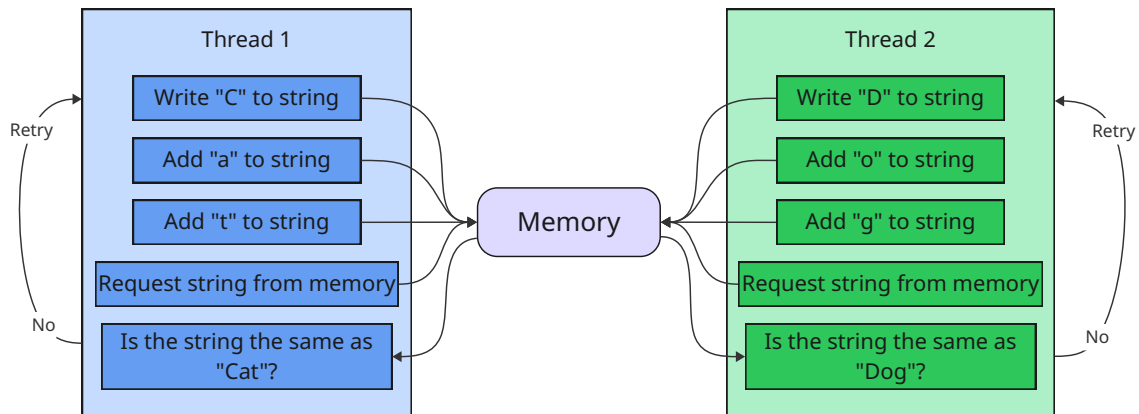


Figure 2.8: Illustration of a livelock.

An example of livelock could be the figure above [Figure 2.8], where the string will start with having either a "C" or a "D" and then a combination of the rest of the characters in cat and dog. Such a result could be "Catog", "Cogat", "Daogt", or another combination. There is a scenario where it could even pass both threads depending on which combination the thread scheduler [Section 2.1] would take. The last scenario is very unlikely, and the possibility of this scenario happening would only get exponentially smaller as the thread requires more continuous actions, and an interruption would result in a livelock.

2.2.4 Order and Atomicity violations

When a sequence of operations occurs without any interruptions or intermediate state, it is considered atomic [12, p. 585]. Breaking this strict order of execution is known as an atomicity violation. If the injected tasks change any variables or states related to the atomic segment, it will likely break as it encounters a state different from what it would expect to see after the first part of the atomic unit [20].

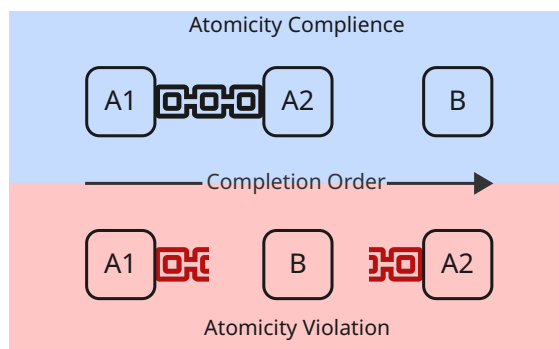


Figure 2.9: Atomicity Violation

Similar to atomicity violations, order violations are a broader class of issues that occur when the two or more operations are expected to occur in a certain order (e.g., $A \rightarrow B$), whether across multiple threads or

sequentially, but are executed in another order ($B \rightarrow A$). These violations typically arise when programs rely on implicit ordering assumptions without enforcing them through synchronization mechanisms such as locks or barriers. As a result, one thread may read a shared state before another thread has completed the necessary initialization or update, leading to incorrect program behavior [21].

2.2.5 Atomic Operations

Using mutexes or locks can help solve the problem of race conditions. However there is an issue that can occur, if two or more threads attempt to lock a mutex at exactly the same time. This can be solved using atomic operations.

Atomic operations are single operations that cannot be interrupted [22]. Typically, a thread checks if a mutex is locked, and if the mutex is unlocked, the thread could then lock the mutex. This normally involves multiple operations, and is therefore vulnerable to race conditions.

By using atomic operations such as compare-and-swap (CAS), the thread can check and acquire the lock in a single operation, which appears instantaneous to other threads [22]. This way another thread cannot check the lock in-between the operations, and the mutex can only be acquired by a single thread at a time.

Another implementation of atomic operations is atomic variables, which in most cases eliminates the need for mutexes or locks for that specific variable. The idea is similar, but instead of using the atomic operations on the locks, the atomic operations are used to modify the atomic variables [23].

An example of this is shown at [Algorithm 2], where Compare-and-Swap (CAS) is used to atomically increment a variable. Here, the function "`CAS_pseudocode`" represents CAS, which all happens in a single operation, and the function "`atomic_increment`" shows how CAS is repeated until success to ensure it accounts for other changes to the variable.

By implementing it this way, the atomic increment will attempt to switch out the saved variable with a value that is one higher, but only if the current value in the operation is the expected value, otherwise it returns false. This way, the update of the variable happens atomically, while the computations on the value can be multiple operations long. This way, atomic operations and variables can be used to prevent race conditions in concurrent programming.

Algorithm 2 Atomic Increment using Compare-And-Swap

```

1: function CAS_PSEUDOCODE(address, expected_value, new_value)
2:   if *address = expected_value then
3:     *address  $\leftarrow$  new_value
4:     return true
5:   end if
6:   return false
7: end function

8: function ATOMIC_INCREMENT(address)
9:   repeat
10:    old_value  $\leftarrow$  *address
11:    new_value  $\leftarrow$  old_value + 1
12:  until CAS_PSEUDOCODE(address, old_value, new_value)
13:  return new_value
14: end function

```

2.3 Parallel Performance Issues

"Parallel slowdown" is when a parallel implementation of a program runs slower than the sequential counterpart. Causes include too fine granularity, load imbalance, false sharing, and oversubscription, all of which represent issues in how the program handles the parallel aspect of solving the given task[12].

2.3.1 Starvation

Resource starvation can occur in a number of ways, but when discussing parallel programming, the most relevant occurrence is when a thread waits on a lock for an excessively long time. This is exasperated if other threads keep acquiring the lock faster when it unlocks, forcing it to keep waiting (this assumes there is no manager or queue system). If unlucky, the starvation can be perpetual, thus leading to a critical error [21]. Starvation has a subcategory called unfairness, where a subset of threads trying to acquire a lock is granted priority an *unfairly* large amount of the time, thus locking out one or more threads from acquiring the lock until other threads stop skipping ahead in the queue [12]. Shared cache can cause this in a server with multiple CPU sockets. If one node, meaning a CPU with its own RAM, requests a lock on a remote node (a different node on the same system) it has higher latency due to longer physical distance potentially causing it to be rejected because the local node can acquire the lock faster due

to closer proximity to the CPU core [24].

2.3.2 Granularity

Parallelism granularity refers to how many threads a given computation is split into, and how large a portion of the processing power is being used to facilitate the parallelism compared to what the program itself requires. This is very important to manage, as the overhead for creating a new thread is relatively large, and would need a correspondingly large amount of work before it pays off. Too coarse granularity, and each thread has a significant workload, making it slower than it otherwise could be. Too fine granularity, and each thread is solving trivially small tasks which could have been grouped up further, meaning the overhead is making it slower than it would be with fewer threads. In summary, a bad granularity may cause a program to run significantly slower.

2.4 Parallel Programming

2.4.1 Theoretical Speedup in Parallel Computing

Speedup is a core performance-based concept within parallel computing that describes how much quicker a program executes when using additional processors. It is often used to determine how effective parallelization is when comparing the execution time of a sequential and parallel implementation of the same program. Different models are used to evaluate the limits of speedup, each using various assumptions.

Amdahl's Law

Amdahl's Law is a formula proposed by Gene Amdahl in 1967. It is used to formalize the expected maximum improvements to a system when only part of the system is improved. Amdahl's model is based on three assumptions, which are that the problem is a fixed size, the ratio between parallel and sequential work stays the same, and that there is no overhead when executing tasks in parallel [25]. Gene Amdahl formulated the following formula for determining speedup:

$$Speedup = \frac{1}{1 - P + \frac{P}{N}}$$

In this formula, P represents the part of the program's total execution time that can be

parallelized, and N is the number of processors used. Not all parts of a program can be parallelized; some must run sequentially, which can consist of program initialization or reading data [26]. The sequential part of a program limits the achievable speedup regardless of the number of processors used. As illustrated in [Figure 2.10], adding additional processors to a program will result in diminishing returns.

An example of a program where 75% of the workload can be parallelized, and the remaining 25% remains sequential, using two cores rather than one would result in the following calculations:

$$S = \frac{1}{1 - 0.75 + \frac{0.75}{2}} = \frac{1}{0.625} \approx 1.6 \quad (2.1)$$

Using the speedup formula, it is shown that a doubling of the number of processors will not result in the same reductions in time, as the theoretical maximum speedup of this program would be around 1.6 times as quick. This example shows how the sequential part of a program, which can not be parallelized, limits scalability, as additional processors will return diminishing returns using Amdahl's formula and its assumptions.

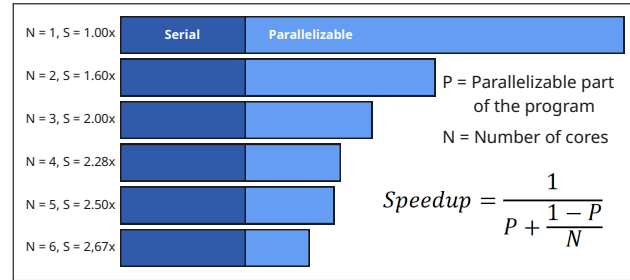


Figure 2.10: Illustration of the diminishing returns of increasing the number of cores, using speedup from Amdahl's Law.

Additional Speedup Models

While Amdahl's Law uses fixed-size problems, other models like Gustafson's Law assume that the problem size grows as additional processors are added. Gustafson's Law was introduced by John Gustafson in 1988. His model is based on three assumptions, which are that the problem size will scale with the number of processors, that the execution time is fixed, meaning that the additional processors are used to perform more computations in the same time frame. Lastly, the parallel fraction increases as the problem size grows, while the sequential portion remains constant [25].

Other models, such as linear and superlinear speedup, describe ideal cases where performance scales in proportion to the number of processors, or even better. These situations are rare in practice and usually occur because of specific properties of the problem rather than general parallel behavior. These models show that speedup can vary depending on the workload and the assumptions made [27].

Granularity speedup

As mentioned in [Section 2.3.2], granularity can be either too coarsely split or too thinly split, which can result in suboptimal granularity. To achieve the fastest granularity level, the task has to be partitioned, so that the time of the largest part of the task and the register write together is the largest inseparable part that all other parts should aim to be at or just below. This is illustrated in [Figure 2.11] below.

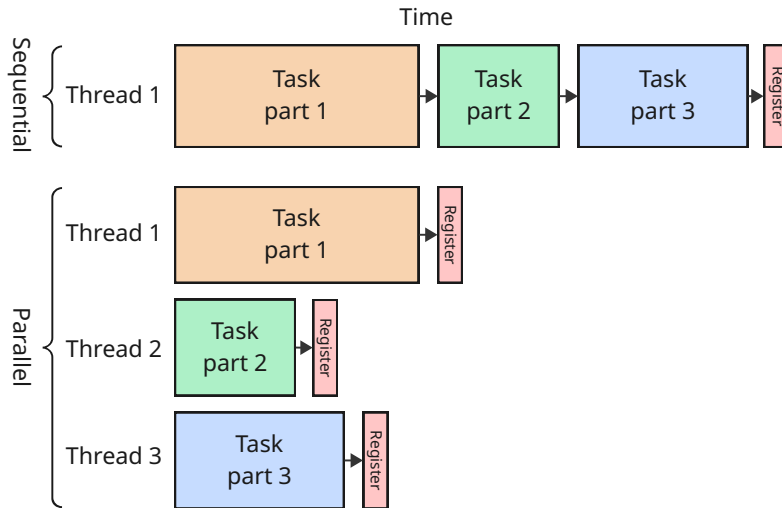


Figure 2.11: Illustration of how granularity can speed up a sequential task.

2.4.2 Structuring of a parallel program

While structuring a parallel program, it is essential to remain mindful of data races and deadlocks, both of which are discussed in detail in [Section 2.2.1] and [Section 2.2.2]. These issues can cause the program to produce incorrect output or halt entirely. To avoid deadlocks one applicable strategy is *Acquire Needed Locks First*, as described in [Section 2.2.2]. This strategy requires a thread to acquire all necessary locks before starting any processing, and to do so in a consistent order. So if two locks (L1 and L2) are required by a thread, it must request and acquire the lock with the smallest number (L1) before the next one (L2). This prevents one thread from holding L1 and waiting for L2, while another thread holds L2 and waits for L1 [12, p. 107–108].

By proactively addressing synchronization hazards through ordered locking and data-race prevention, developers can create parallel programs that are both correct and efficient. A well-structured parallel design not only maximizes performance gains but also ensures long-term maintainability and reliability as the program evolves.

2.4.3 Lock-free and Wait-free Algorithms

In parallel computing, multiple threads can run simultaneously on different CPU cores. In concurrent programming, multiple threads appear to run at the same time, even on a single core, although in reality only one thread executes at any instant.

When threads execute asynchronously, meaning the order of their operations is independent and uncoordinated, e.g., different threads do not directly interact or synchronize with each other, tasks cannot be perfectly timed to be ready exactly when needed. This independence allows multiple threads to attempt to access or modify the same shared memory location at overlapping times. These overlapping accesses can lead to conflicts that must be handled carefully to avoid incorrect results or indefinite stalling.

Lock-free and wait-free algorithms are two categories of non-blocking algorithms. They are designed for scenarios where asynchronous processes may need to access the same place in memory simultaneously. While a traditional mutex can guarantee correctness by enforcing mutual exclusion, lock-free and wait-free algorithms achieve correctness without blocking threads [28]. Instead, they rely on atomic operations and retry mechanisms, allowing threads to make progress even under contention [29].

Asynchronous execution means multiple threads may be active at the same time and perform different tasks, and therefore require access to the same location in memory concurrently with another thread. To resolve such conflicts, lock-free and wait-free algorithms aim to mitigate traditional blocking and instead handle stalled threads without requiring them to wait indefinitely before accessing the location in memory again.

Lock-free algorithm

The lock-free algorithm works on the premise that at least one thread can always continue, even if other threads starve. A thread is starving if it cannot continue because it needs to access a place in memory that is already in use or locked, so other threads cannot access the same place in memory at the same time, or if one of the threads that needs to make a computation for the first thread has failed. With a lock-free algorithm, at least one thread can always continue to make progress [30, 28].

As can be seen in [Figure 2.12], thread 1 is always

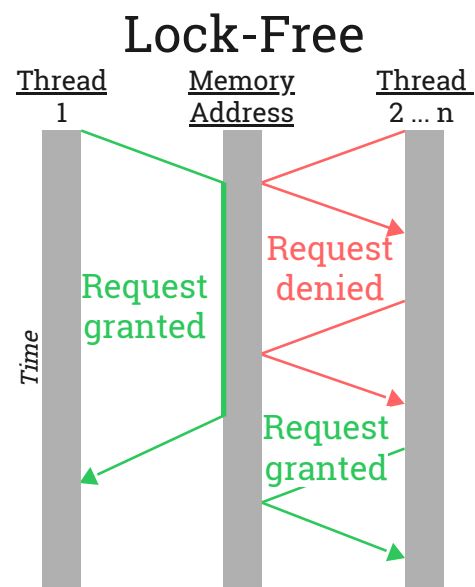


Figure 2.12: Illustration of a lock-free algorithm. Green arrows indicate granted access, while red arrows indicate denied access.

running, while all other threads are starving. This is shown by the green and red arrows. The green arrows for thread 1 indicate that it succeeds in accessing and updating the shared memory location, while the red arrows for threads 2...n show that these threads are repeatedly failing their operations and retrying. This situation occurs even though no locks are used. The green arrow for thread 1 shows it successfully modifying the data structure, while the other threads, marked with red arrows, keep retrying but cannot make progress [29].

Wait-free Algorithms

Wait-free algorithms are based on the premise that when multiple threads attempt to access the same location in memory concurrently, no thread should wait indefinitely. In addition, every operation has bounds. Bounds are the number of steps a process needs to complete before it is finished, ensuring that a new task can be assigned right after the first one is done. In a traditional lock-based approach, one thread may gain access first, this may be via priority or the order of arrival, locking the resource and causing the second thread to get blocked and potentially starve until the first thread finishes.

In a wait-free algorithm, it is guaranteed that every thread completes its operation in a finite number of steps, regardless of contention and scheduling. This guarantees progress for every thread and prevents starvation; no thread can be delayed indefinitely. As can be seen in [Figure 2.13], thread 1 is using the data structure, which is indicated by the green arrows, showing successful access. The other threads 2...n may still contend for access, but the algorithm ensures that every thread will complete its operation without waiting indefinitely for another to finish. This works because wait-free algorithms ensure progress for every thread without requiring any to wait indefinitely for another to finish [30, 28].

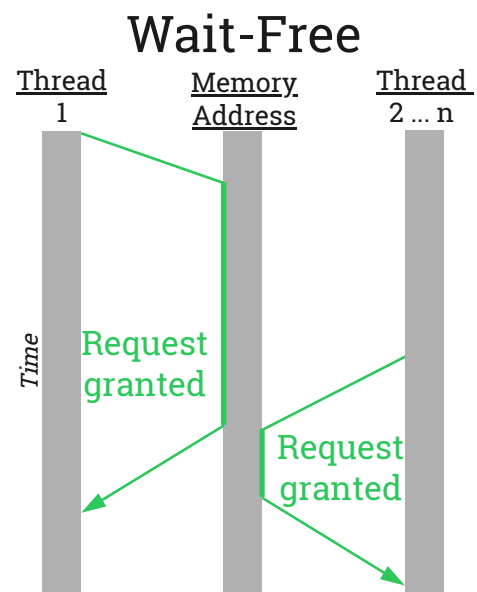


Figure 2.13: Illustration of a wait-free algorithm, Green arrows indicate granted access.

2.5 Existing Solutions

These problems have existed for a long time, and as such, existing programming languages have attempted to account for them. To better understand how a new programming language can be designed for parallel programming, a selection of existing solutions were analyzed. The analysis focuses on what types of languages they are, what features allow for parallel programming, and what their main upsides and downsides are. They were chosen based on how well they address the concurrency problems discussed in [Section 2.2] [31].

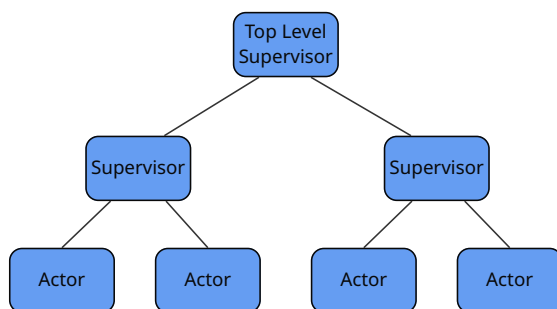
2.5.1 Erlang

The programming language, Erlang, is mainly designed for distributed systems [32, 33]. As this use case shares a lot of the same problems as parallel programming, it is useful to look at how this programming language handles issues such as race conditions and deadlocks.

Erlang is a dynamically typed functional language, which among other things means that it has immutable data and no mutable shared state [32]. This helps the programming language prevent race conditions, as multiple different threads cannot modify the same data at the same time.

The Erlang Actor Model is a key part of Erlang's ability to run concurrent systems. An actor is a lightweight independent unit with its own private state, and the ability to receive and send messages. These actors do their own computations independently of each other, and never share the same memory [32, 34].

Since the different actors never share the same memory, there is no need for mutexes or locks. This effectively means that the most common occurrence of a deadlock does not appear in Erlang.



Another thing the actor model does well is provide stability to the overall system. This is done by having actors create other actors and supervising them in a tree-like structure as shown in [Figure 2.14]. When an actor crashes or gets an error, this is handled by their supervisor, which can restart the actor. This way the system can handle unrecognized errors by letting the

Figure 2.14: Illustration of how actors can supervise other actors in a tree-like structure.

actors die and restarting them [32, 33, 34].

This restart does not affect the other actors, and the actor model therefore provides overall stability, since parts of the system can crash without influencing the overall program [32, 33, 34]. This system also provides good scalability because of the small size of an actor, and the fact that more actors can be added to the system [33].

```

1 -module(erlang_example).
2 -export([start/0, actor/0]).
3
4 %% Start function to spawn an actor and send messages
5 start() ->
6     Pid = spawn(erlang_example, actor, []),
7     Pid ! {hello, "world"},
8     Pid ! stop.
9
10 %% Actor loop
11 actor() ->
12     loop().
13
14 loop() ->
15     receive
16         {hello, Msg} ->
17             io:format("Received message: ~p~n", [Msg]),
18             loop();
19         stop ->
20             io:format("Stopping actor~n")
21     end.

```

Code 1: Simple example of the Erlang actor model creation and usage.

In [Code 1] is an example of how an actor is created in Erlang, along with how messages are sent to and received by an actor. Lines 11-21 shows the actor definition, where it goes into a loop. In this loop it reads a message from the actors mailbox, and then processes it based on the type of message.

Line 7 adds `{hello, "world"}` to the mailbox, and line 8 adds `stop` to the mailbox. This is then read by the actor First In, First Out (FIFO) style. Essentially, the message from line 7 executes lines 16-17 and then the actor loops back to the top, the message from line 8 then executes lines 19-20 which ends the actor, because of the fact that the `stop` pattern does not go back to the loop recursively.

The Erlang Actor Model is especially useful for distributed systems with focus on stability and scalability. This being said, Erlang is not optimized for parallel programming, where

the computational throughput suffers compared to other languages such as C [35].

The Erlang Actor Model is especially useful for distributed systems with focus on stability, but it does not make as much sense for parallel computing. This is because of the fact that having a lot of actors work concurrently in a safe and secure manner requires Erlang to sacrifice a lot of computational throughput.

2.5.2 Pony

A somewhat similar language to Erlang, although way less common, is Pony. This statically typed language is based on the actor model just like Erlang, but unlike Erlang it is designed for faster computations.

The different actors in Pony have no shared mutable memory, although Pony allows shared immutable memory using Pony's reference capabilities. This eliminates mutexes and deadlocks from shared locks, while helping the programming language prevent race conditions [36].

The reference capabilities are designed based on what the different types deny for local and global aliases. Local aliases refer to the actor that holds the current path to an object, while global aliases refer to other actors that have another reference to the same object.

	Deny global read/write aliases	Deny global write aliases	Allow all global aliases
Deny local read/write aliases	<i>Isolated (iso)</i>		
Deny local write aliases	Transition (trn)	<i>Value (val)</i>	
Allow all local aliases	Reference (ref)	Box (box)	<i>Tag (tag)</i>
	(Mutable)	(Immutable)	(Opaque)

Table 2.1: Pony Reference Capability Matrix [36].

Each reference capability is a pair of local and global deny properties, which can be seen in [Table 2.1]. When a reference denies global read/write aliases, it is safe to read and write, and thus it is mutable. If it only denies global write aliases, it is immutable since other actors can read from it. A reference that allows all global aliases is not safe to read nor write, which makes it opaque [36]. A reference is opaque if it is only accessible through methods.

When both the local and global deny properties of a reference are the same, the reference can be safely sent to another actor, as it does not matter which actor holds the reference. These references are "sendable", which can be seen in [Table 2.1] marked with italic.

```

1 class C
2   var x: U32 = 0
3
4   fun ref inc() =>
5     x = x + 1
6
7   fun val get(): U32 =>
8     x
9
10 actor Main
11   new create(env: Env) =>
12     let a: C iso = C
13
14     let b: C ref = consume a
15     b.inc()
16
17     let c: C val = recover val b end
18     env.out.print(c.get().string())

```

Code 2: Simple Pony example that showcases the reference capabilities of the language for some simple data types.

An example of the reference capabilities in Pony is seen in [Code 2]. To better showcase the functionality of the different reference capabilities, the class `C` is introduced. This class has a variable `x` and two methods `inc()` and `get()`, that increments the variable `x` by 1 and returns the variable `x` respectively.

On line 12, the expression `C` creates a new object with `a` being the only reference to said object, hence the `iso` keyword. On line 14 this reference is consumed by a new reference `b`. Since the reference type of `b` is `ref`, it now allows for all local aliases [Table 2.1], and it is now possible to use the `inc()` method.

After having incremented `x` in the object, line 17 creates a new reference `c` which is of the reference type value. This is a more restrictive reference type, since it denies local write aliases, but instead it allows global read aliases as well. To do this, the object `C` which is referenced by `b` is reinterpreted within the scope of that block, such that it can become a `val`. Since `val` allows global read aliases, the `get()` method can be used to get the value of `x` to print in line 18.

One of the reasons Pony runs faster than other languages designed for parallel programming is because of its garbage collection system, that works per actor instead of having one global garbage collector. This means that the individual actors can clean their memory without affecting other actors.

The main downsides of Pony are that it has a very steep learning curve. The reference capabilities make simple variables significantly more complicated, and a developer would need to know how it works exactly before using it. Furthermore, Pony has a small community and is not widely used in the industry.

2.5.3 Rust

Rust is a statically typed programming language that gets compiled ahead of time using the LLVM backend. This process turns the code directly into fast, efficient machine code that runs straight on the hardware, without relying on any high-level runtime or interpreter in between [37].

What makes Rust stand out is how it combines high-level features and pleasant syntax with excellent performance. It feels modern and expressive, yet delivers speed close to languages like C and C++. A key part of this is its memory ownership system. Every piece of data has exactly one owner at any time. When that owner goes out of scope, the data is automatically cleaned up. This approach prevents many common bugs without needing a garbage collector [37].

The language also has a strict borrow checker that enforces clear rules about how data can be accessed. You can have either many immutable references or exactly one mutable reference to a value, but never both at the same time. References are always checked to make sure they stay valid, which eliminates dangling pointers and other memory errors at compile time [37].

When it comes to concurrency, Rust prevents data races entirely. If the code compiles, it is impossible to have data races in safe Rust code. This safety comes from the same ownership and borrowing rules, plus two important marker traits: `Send`, which means a value can be safely moved to another thread, and `Sync`, which means it can be safely shared across threads. The compiler refuses to build code that breaks these guarantees unless in `unsafe` mode [38, 39].

Deadlocks are still possible in Rust, but they are logical issues rather than memory-related ones. The language encourages using message passing through channels instead of shared mutable state and locks whenever practical. When locks are needed, tools like the `parking_lot` crate can help detect potential deadlocks during development [39, 40].

Rust does have some trade-offs for safe parallel processing. The borrow checker can feel restrictive and has a steep learning curve, especially for newcomers. Deadlocks remain a risk if you rely heavily on locks. Logical race conditions, while impossible to create accidentally in safe code, can still happen if `unsafe` mode is used. Overall, though, Rust

gives developers strong guarantees about memory safety and thread safety without sacrificing performance [37, 39, 40].

```

1 fn main() {
2     let s1 = String::from("hello"); // s1 owns the String
3
4     // Move: ownership transfers to s2
5     let s2 = s1;
6
7     // This would fail to compile:
8     // println!("{}", s1); // error: use of moved value
9
10    println!("{}", s2); // works fine
11
12    // Borrowing example
13    let len = calculate_length(&s2); // immutable borrow
14    println!("Length: {}", len);
15
16    // Only one mutable borrow at a time is allowed
17    // let r1 = &mut s2;
18    // let r2 = &mut s2; // would fail: cannot borrow as mutable more than once
19 }
20
21 fn calculate_length(s: &String) -> usize {
22     s.len() // immutable reference
23 }

```

Code 3: Rust ownership and borrow checker code example.

Above is a Rust code example [Code 3] that clearly outlines a use case where the programmer could try to use a value after its ownership has already been transferred to another variable. The code example also clearly shows how an immutable borrow allows a read operation of a variable, while only one mutable borrow is allowed at any time.

2.6 Summary

Chapter 3

Requirements for ParaLang

The requirements chapter reflects the basic functionality and expectations of the projects language, called ParaLang (short for Parallel Language), the transpiler, as well as the fourth semester learning goals [41]. The requirements are presented in the [Table 3.1] in a descending order of priority. The requirements primarily originate from the problem analysis, which highlighted features important for developing a safe parallel programming language. Other requirements stem from the group's understanding of what makes a modern and developer-friendly programming language.

The various requirements are categorized using the MoSCoW method (must have, should have, could have, and won't have) to ensure a clear indication of the requirements' importance and prioritization.

Chapter 3. Requirements for ParaLang

	Requirement	Must Have	Should Have	Could Have	Won't Have
1	ParaLang can create and use multiple threads.	X			
2	The transpiler can translate a file of code in ParaLang into an equivalent new file with Java code.	X			
3	The transpiler is tested using unit-, integration- and acceptance testing.	X			
4	ParaLang is based on formal and correct syntax and semantics.	X			
5	ParaLang supports the data types: Char, Int, Float, Bool, String.	X			
6	ParaLang supports Arrays of the following data types: int, float, bool, char, string.	X			
7	ParaLang supports these expressions: +, -, *, /, %, ==, !=, , <, >, &&, >=, <=.	X			
8	ParaLang supports these Statements: if, if else-if else, if else,for, while, critical, variable & function declaration, variable assignment, variable reassignment, print, read, function call.	X			
9	ParaLang has a method designed for handling critical sections.	X			
10	ParaLang provides functionality to help prevent deadlocks.	X			
11	ParaLang supports input and output via the console.	X			
12	ParaLang has a wait method that has a boolean argument that waits until it is true.	X			
13	ParaLang contains parallel threads as the data type Thread.	X			
14	ParaLang can explicitly cast variables of types: int, float, bool, string.	X			
15	A parallel implementation of some program in ParaLang can be implemented, safely avoiding deadlocks and race conditions.		X		
16	If a variable is marked as shared, ParaLang will automatically wrap it in a mutex.		X		
17	The transpiler throws errors instead of crashing.		X		
18	Threads resolve to a boolean value that cannot be manually reassigned.		X		
19	ParaLang supports modifiers that define how variables are accessed. These will be "Shared", "Const" and "Static".		X		
20	ParaLang includes single- and multiline comments.		X		
21	ParaLang allows the implementation of parallel for-loops.			X	
22	ParaLang supports these expressions: +=, -=, *=, /=, %=			X	
23	ParaLang supports these Statements: do while, switch case.			X	
24	ParaLang has hierarchical queues for acquiring locks in an ascending sequential manner.			X	
25	ParaLang allows developers to override default mutex behavior.			X	
26	ParaLang can implicitly cast variables of type int to type float.			X	
27	ParaLang has lambda expressions.			X	
28	ParaLang will output readable Java code using indentation.			X	
29	ParaLang is a direct extension of Java.				X
30	ParaLang is based on object-oriented programming.				X

Table 3.1: A requirements table, where different features are prioritized using the MoSCoW method.

3.1 Must Have

The strategy for the solution that allows users to utilize parallel programming is through the creation and usage of threads, where different threads can run simultaneously. This can speed up certain computational tasks significantly, but introduce certain risks, that

ParaLang will attempt to minimize, thus placing this requirement in the must have category [Requirement 1] [Section 2.4.1].

In order to compile to a language that is relatively low-level, without being assembly-level code, Java, has been chosen as the destination for the transpiler, which is shown by [Requirement 2]. In specific, the transpiler needs to be compatible with the Java standard libraries: `java.util.concurrent`, `java.util.concurrent.locks` and `java.lang.Thread`, as these libraries introduce necessary functionality for parallel programming.

To ensure correctness of the transpiler, it must be tested thoroughly, which is shown through [Requirement 3]. This means that the smallest modules of the transpiler are tested with unit-testing. The interaction between modules should be tested using integration testing to ensure correct passing of variables and return values. All the requirements should also be evaluated with several acceptance testing strategies to determine in what degree the project succeeded.

[Requirement 4] focuses on describing the grammar of ParaLang in a clear and unambiguous manner. To ensure this, ParaLang will be described using syntax and semantics.

To guarantee that ParaLang functions with standard data types, such as Char, Int, Float, Boolean, and String must be defined within ParaLang. This adds functionality to the transpiler that match data types in Java, thereby fulfilling [Requirement 5].

[Requirement 6], [Requirement 7] and [Requirement 8] are all requirements that are meant to give programmers freedom with their code. Having arrays of all typical data types, arithmetic and boolean expressions, and supporting basic statements are all important functionalities in a programming language.

Critical sections are important whenever working with threads, to make sure the variables being read and written to is correct [Section 2.2.1]. This requirement [Requirement 9] has been placed in the must have for this very reason.

As deadlocks are among the most common and critical issues in parallel programming, a language focused on safe concurrency should provide built-in features to help prevent them. [Requirement 10] encompasses just this, and ensures that ParaLang helps users practice safe parallel programming. In specific, the acquire needed locks first deadlock avoidance strategy as described in [Section 2.2.2].

ParaLang is not meant to provide high-level functionality like Graphical User Interfaces (GUI) or such, so the ability to use the console for input and output is vital for any interaction with a working program, making the [Requirement 11] a must have.

[Requirement 12] describes the ability to pause the a thread until one or more threads have finished their tasks allows for concurrent programming to synchronize seamlessly. This should work like a hidden boolean that blocks when false and continue when true. This allows developers to time specific statements in different threads to perfectly avoid deadlocks and race conditions.

As mentioned in [Requirement 1], ParaLang will implement threads. They will be created in a similar manner to a regular variable assignment, using a Thread type as described in [Requirement 13]. The thread will be initialized with a given piece of code for the thread to execute.

ParaLang should only explicitly cast types to make sure the all data types interact predictably. The program should have rules that define which data types should be able to cast from one data type to another. It was decided that [Requirement 14] would be a must have requirement to give the programmer more choices when coding in ParaLang.

3.2 Should Have

The idea behind using parallel programming instead of sequential programming in this project, is to utilize multiple cores of the CPU. [Requirement 15] ensures that ParaLang allows for safe parallel implementations, which ensures that ParaLang is capable of doing efficient parallel programming.

To help prevent race conditions, [Requirement 16] states that all "shared" variables must be wrapped in a mutex. This means that variables that are mutable from different threads are exclusively modifiable by the thread that has acquired the mutex, which ensures that multiple different threads cannot modify the variable at the same time.

[Requirement 17] is important, although not necessary for a minimal viable product (MVP). The requirement ensures that a developer using ParaLang can get help debugging potential errors in their code, instead of the program just crashing without giving any helpful feedback.

As mentioned in req:1, ParaLang will work with threads. [Requirement 18] describes that each thread should work like a hidden boolean that is false until it has completed its tasks. Then it should be true, such that an await statement, as in req:12[Requirement 12].

To allow for variables to be "shared" as mentioned in [Requirement 16], ParaLang must support these modifiers, that define how the variables are accessed. Other than the "shared" prefix, ParaLang should also support the "Const" and "Static" prefixes. This is defined in [Requirement 19] as a should have requirement.

[Requirement 20] ensures that developers can comment their code. This is an important feature for debugging, along with documenting the functionality of code, such that other developers can easily see what the different parts of a program does. Although not strictly necessary for an MVP, it is a very useful feature.

3.3 Could Have

[Requirement 21] describes the implementation of parallel for-loops. These are useful as they can (depending on the implementation) be simple to use for the developer, but are difficult to implement in ParaLang. If time permits it, this could be a great feature to have in ParaLang.

Although it is nice to have in-line addition, subtraction etc. it is not at all necessary for a language. They are nice for simple and clean code, but the same functionality is easily written with a few more characters in the code (instead of `x += 1` one would write `x = x + 1`), so [Requirement 22] is prioritized as a could have requirement.

[Requirement 23] mentions do while, and switch cases, which are nice for programmers to have, but the same functionality can be achieved with while loops and if-else statements. Implementing these would add little extra complexity to the project without gaining meaningful programming capabilities, so it is prioritized as a could have requirement.

[Requirement 24] is about having each mutex numbered, so if a thread wants two or more locks it has to acquire those locks in ascending order to avoid deadlocks. This could take a long time to implement but it could help a lot in avoiding deadlocks so it was placed as a could have requirement.

Having increased customization in a programming language is nice, and could make for a powerful advanced feature where developers can customize their own locking behavior, but having standard mutex behavior that simply works is the higher priority. Therefore, [Requirement 25] is prioritized as a could have requirement.

Implicit casting is nice to have, but not at all a necessity. Programmers can still explicitly cast variables without it, and therefore [Requirement 26] is prioritized as a could have requirement.

A lambda expression is an anonymous function often used for short, in-line operations where normal functions would be excessive. Although nice to have, this is not strictly necessary, which is why [Requirement 27] is prioritized as a could have requirement.

[Requirement 28] would ensure better readability of the program, once the code has been translated to Java code. This would make debugging way easier during the development phase. Because it is not necessary for a MVP, but would be a user-friendly feature, it has been prioritized as a could have requirement.

3.4 Won't Have

ParaLang could be based directly on Java's syntax and semantics, but Java has very complicated semantics that is way above the groups goals. ParaLang could just take inspiration in how the Java syntax is written and create unique and simplified semantics, but Java is a large language so ParaLang could never cover all areas of Java thus making this requirement a won't have requirement.

ParaLang could have been a language that implemented an object-oriented programming structure, but due to complexity and focus of the project, [Requirement 30] is a won't have requirement.

Chapter 4

Design

In order to create a robust language and incorporate the research done throughout the project, the semantic rules need to be written and designed to follow the design goals of the project to make the use of threads more intuitive, less likely to result in deadlocks, and to handle concurrent threads. The rules are written in a logical abstract manner using big-step semantics. The reason for using big-step semantics is to describe the flow and logic throughout the language, thereby providing a clear and coherent semantic structure.

4.1 Language Design Decisions

Several design decisions shaped ParaLang in an attempt to accomplish the goal of a safe parallel programming language. This section highlights the most significant choices made during the design phase and the reasoning behind them.

4.1.1 Statements as the Highest Construct

ParaLang is designed as a statement-oriented language, meaning a program is simply a sequence of statements with no required top-level structure, such as a class or main function. This keeps the language lightweight and approachable, reflecting its focus on concurrency rather than object-oriented or modular programming, and is consistent with requirement [4], which states ParaLang is based on a formally defined and unambiguous grammar.

4.1.2 Shared Variables and Critical Sections

A central design decision in ParaLang is how race conditions and deadlocks are prevented. Rather than leaving synchronization entirely to the programmer, ParaLang requires shared variables to be explicitly marked with the `shared` prefix, fulfilling requirement [19]. Such variables may only be changed inside a `critical` block, which takes the shared variables to be protected as arguments. Take [Code 4] as an example.

```

1 critical(x, y) {
2     x = x + 1;
3     y = y - 1;
4 }
```

Code 4: Paralang critical section example.

This design makes the intent of the programmer explicit and machine-verifiable: changing the value of a shared variable outside a `critical` block should be a detectable violation rather than a silent bug, directly addressing the data races described in Section [2.2.1].

An important aspect of ParaLang itself having a deadlock avoidance strategy, is how all mutex acquisition and release are handled automatically by ParaLang, fulfilling requirement [16]. To avoid deadlocks arising from inconsistent lock ordering when managing multiple locks as discussed in Section [2.2.2], the compiler always acquires locks in a globally consistent ascending order, fulfilling requirement [10].

4.1.3 Thread Handling and Concurrency

Concurrency in ParaLang is expressed through a deliberately constrained thread model, fulfilling requirement [1]. A thread is created by binding a block of code to an identifier using the `=>` operator, as shown in [Code 5].

```

1 thread t => {
2     doWork();
3 }
```

Code 5: ParaLang thread assignment example.

Once spawned, a thread is intentionally isolated: the programmer cannot pass new data to it, cancel it, or otherwise interact with it during execution. The only observable property of a running thread is its completion status, exposed as a boolean value that is `true` when the thread has finished and `false` while it is still running, fulfilling requirement [13]. This constraint is a deliberate design choice rooted in the analysis of

concurrency bugs in Section [2.2.1]: by preventing mid-execution interference, the model eliminates an entire class of concurrency bugs that arise from threads being modified or canceled in inconsistent states.

Synchronization between threads is handled through `awaitAll` and `awaitAny`. `awaitAll` blocks until every listed thread has completed, while `awaitAny` unblocks as soon as any one of them finishes. Both accept one or more thread identifiers, allowing multiple threads to be synchronized in a single statement [12]. Together with the `critical` block, these constructs form the complete concurrency model of ParaLang: threads are created and run independently, shared state is protected explicitly, and synchronization is expressed at clearly defined points in the program.

4.2 Syntax & Semantics

The goal of the presented syntactic and semantic definitions is to provide a formal and unambiguous definition of the language ParaLang, precisely documenting how each construct evaluates and how the state evolves during execution. This serves two purposes: it acts as a specification and guide for the implementation of the transpiler, and it provides a foundation for reasoning about the correctness of programs written in ParaLang.

The biggest challenge in designing a language that focuses on parallelism is that it introduces more complexity than a sequential language. This results in more variables and aspects that must be designed and considered at both the syntactic and semantic levels. This is evident in how ParaLang handles thread lifecycles, shared mutable state, and mutual exclusion within critical sections. Because of these components, which must be considered throughout the entire language, both the syntax and semantics need to be explicitly represented through the grammar and operational semantics of ParaLang.

The method chosen for formulating ParaLang is big-step semantics accompanied by a type system. These methods describe the different kinds of operations defined in ParaLang. The program state in ParaLang is divided into a local state s and a global state g , making it safer for threads and more intuitive to use. This separation defines how data is secured and how cases with multiple threads running in parallel are handled, especially when they attempt to read from and write to the same section of memory simultaneously.

4.2.1 Setup

This section will establish the fundamental understanding of ParaLang, covering the syntactic categories, formation rules, and the state definitions used in both the syntax and the semantic rules.

Syntactic Categories

The syntactic categories in ParaLang are used to describe the logical associations between the abstract elements of the language, classifying them so the formal rules can refer to them precisely and there meta variables.

$e \in E$ (Exp)	$\sigma \in \Sigma$ (Stm)
$v \in V$ (Val)	$x \in X$ (Var)
$t \in T$ (Types)	$f \in F$ (Func)
$s \in S$ (States)	$g \in G$ (Global States)
$\gamma_s \in \Gamma_s$ (Local Type Environment)	$\gamma_g \in \Gamma_g$ (Global Type Environment)
$l \in L$ (Loc)	$a \in A$ (Array)
$\tau \in \mathcal{T}$ (Thread)	$r \in \mathcal{R}$ (Result)
$k \in K$ (Array Size)	$m \in M$ (Modifier)
$\mu \in \mathcal{M}$ (Mutex)	

$$\mathcal{R} ::= S \mid \text{cnt} \mid \text{brk} \mid \text{ret} \mid \text{ret}(V)$$

For any meta-variable x , $\bar{x}$ will be shorthand for a finite sequence $x_1, \dots, x_n$ for some $n \geq 0$. Sequence concatenation is written $x_1 \oplus \bar{x}'$ and ε denotes the empty sequence.

Expressions, denoted $e \in E$, are used throughout ParaLang and form the building blocks for most computation rules. Statements, denoted $\sigma \in \Sigma$, are used to construct the program itself. Values, denoted $v \in V$, represent the actual data stored and used during execution. Variables, denoted $x \in X$, store references between named identifiers and memory locations.

Different expressions and variables can have a type, denoted $t \in T$, where T is the set of all types. The program state is divided into a local state $s \in S$ and a global state $g \in G$.

This separation is central to how ParaLang handles parallelism, as it determines how variable access is managed across threads running in parallel.

Threads are denoted $\tau \in \mathcal{T}$. The way threads are handled in ParaLang is unique to the language, as each thread maintains an associated boolean value indicating whether it is still running or has finished execution. To manage shared access between threads, mutexes are used to determine which thread may access a resource at a given time, denoted $\mu \in \mathcal{M}$. This ensures that critical sections correctly handle conflicting access requests.

To indicate whether a variable is local or shared across threads, modifiers $m \in \mathcal{M}$ are used. These determine whether a variable should remain local to a thread or be accessible across multiple threads.

Formation Rules

Within the context of the syntactic categories, the uses of the metavariables must be defined to avoid misinterpretation. This section presents a subset of the formation rules of ParaLang, with the full grammar available in [Appendix 9.2].

Types t are used to denote the type of a value or expression:

$$t ::= \text{Int} \mid \text{Float} \mid \text{Bool} \mid \text{Char} \mid \text{String} \mid t[k] \mid \text{Void}$$

A type can be any one of the listed value types. For numbers, ParaLang supports integers (Int) and floating point numbers (Float). For text, Char stores a single symbol and String stores a sequence of symbols. Bool is used for true and false values. The type $t[k]$ denotes an array of type t with size k . Finally, Void is used as a return type to indicate that a function does not return a value.

Array size k is used to denote the size of an array:

$$k ::= \varepsilon \mid e$$

The size k can either be empty ε , meaning the size is not yet known, or an expression e that evaluates to the size of the array.

Modifier m is used as a prefix placed before a variable declaration to determine how the variable interacts with the rest of the program:

$$m ::= \epsilon \mid \text{Shared} \mid \text{Const}$$

If no modifier is given, the variable is local and only accessible within its own thread. If `Shared` is used, the variable is visible and accessible across threads. The `Const` modifier is reserved for future development and has no function in the current version of ParaLang.

Type System

To check whether the inputs for the different rules are followed and the variables are well-typed, a check must be performed before any statements are executed. This is done using a type system. A type system checks the types of expressions and statements used throughout the program, ensuring that only valid types t are accepted, as well as determining how the input should be used and what the final result will be.

Transition functions: The transition functions are used to describe how information is processed and how they relate to the big-step rules.

$$\begin{aligned} S \times G \times E &\rightarrow V \times G \\ S \times G \times \Sigma &\rightarrow \mathcal{R} \times G \end{aligned}$$

The first transition function takes the local state S , the global state G , and an expression E , and evaluates it. The result is a value V along with a potentially updated global state G .

The second transition function takes a local state S , a global state G , and a statement Σ . The result is a transition outcome $\mathcal{R}$, which can be either a new local state or a control signal such as `cnt`, `brk`, or `ret`, together with a potentially updated global state G .

The $\rightarrow$ arrow indicates that the function is partial, meaning it is only defined for well-typed inputs and is not guaranteed to produce a result for all possible inputs.

Typing Contexts: The typing contexts γ_s and γ_g are used to keep track of type information during the type checking of a program. γ_s is the local typing context, which holds information about variables, functions, and whether the current context is inside a function or a loop. γ_g is the global typing context, which holds information about shared variables and threads.

$$\begin{aligned} \gamma_s &::= (\Delta, b_l, (b_f, t)) \\ \gamma_g &::= (X \rightarrow T) \times (\mathcal{T} \rightarrow \text{thread}) \\ \Delta &::= (X \rightarrow T) \times (F \rightarrow \text{func}((\bar{m}, \bar{t}), t, \sigma)) \\ b_l, b_f &\in \{tt, ff\} \end{aligned} \quad \text{where } b_l = tt \text{ in a loop, and } b_f = tt \text{ in a function}$$

The typing context γ_s consists of three components. Δ holds the type information for variables and functions available in the current scope. b_l is used to indicate whether the current context is inside a loop, and b_f is used to see whether the current context is inside a function. The type t represents the return type of the current function and is used to ensure that return statements match the declared return type.

Δ is made of two parts: a mapping from a variable's name to their type, and a mapping from a function's name to their definition, including parameter modifiers $\bar{m}$, parameter types $\bar{t}$, return type t , and function body σ .

The global typing context γ_g consists of two parts. The first maps shared variable names to their types, and the second maps thread identifiers to their thread registrations. This is used to ensure that thread spawning and awaiting are used correctly.

Initial State: The initial state of the type system is the state the program starts in before anything happens. This state ensures that all changes to the contexts can be registered and makes it so the system knows what the starting point of the language looks like.

$$(\text{PROGRAM}_{TS}) \frac{(\gamma_{s_0}, \gamma_{g_0}) \vdash \sigma : \text{ok}}{\vdash \sigma : \text{ok}} \quad \begin{array}{l} \gamma_{s_0} = (\emptyset, ff, (ff, \text{Void})) \\ \gamma_{g_0} = (\emptyset, \emptyset) \end{array}$$

The (Program_{TS}) rule defines the initial typing context from which type checking begins. The local typing context γ_{s_0} starts with an empty Δ , meaning no variables or functions have been declared yet. The flags b_l and b_f are both set to false ff , indicating that the program does not start inside a loop or a function, and the return type is set to Void as no function is being executed and no return is expected. The global typing context γ_{g_0} starts with no predefined shared variables and no registered threads. If the program σ type checks ok under these initial contexts, the entire program is considered well-typed and is ready to run.

Big Step

The method used to describe the formal rules and specifications in ParaLang is big-step operational semantics. Big-step semantics evaluate an entire computation in a single step, producing a final value or state directly, rather than describing every intermediate transition as small-step semantics would. This results in a more concise and visually understandable representation of ParaLang's behavior.

The big-step semantics use two different forms of judgment. Expressions are evaluated

using judgments of the form:

$$(s, g) \vdash e \rightarrow (v, g')$$

Where s is the local state, g is the global state, e is the expression being evaluated, v is the resulting value, and g' is the updated global state after evaluation. Statements are evaluated using judgments of the form:

$$\langle \sigma, s, g \rangle \rightarrow (r, g')$$

where σ is the statement being executed, r is the result, and g' is the updated global state after execution. Expressions can modify the global state because function calls are expressions, and functions may interact with shared memory.

Statement Definition: Big-step makes use of a local state S and a global state G .

$$S = \left[\begin{array}{lcl} x & \mapsto & l \\ f & \mapsto & \text{func}(\bar{p}, \sigma) \\ \mu^O & \mapsto & \bar{\mu} \end{array} \right]$$

$$G = \left[\begin{array}{lcl} x & \mapsto & v \\ l & \mapsto & v \\ \tau & \mapsto & \text{thread}(b, \sigma, s) \\ \mu_x & \mapsto & \text{free} \mid \tau \end{array} \right]$$

The local state S maps variable names x to memory locations l , function names f to their parameters and body, and μ^O to the mutexes currently held by the thread. The global state G maps shared variable names x and memory locations l to values v , thread identifiers τ to their runtime state, and mutex identifiers μ_x to either `free` or a thread identifier τ .

μ^O maps to the mutexes already owned by the thread in the outer scope. This is needed for a critical section to know which mutexes not to release after executing. μ_x denotes the mutex associated with the shared variable x , where `free` means unlocked so it can be accessed again, and τ means it is locked by thread τ .

Initial State: The initial state for big-step denotes how the program looks before the first statement is executed.

$$(\text{PROGRAM}_{BS}) \frac{\langle \sigma, s_0, g_0 \rangle \rightarrow (s', g')}{\langle \sigma \rangle \rightarrow g'} \quad \begin{array}{l} s_0 = [\mu^O \mapsto \emptyset] \\ g_0 = [\tau_0 \mapsto \text{thread}(ff, \sigma, s_0)] \end{array}$$

The starting state of big-step takes the body of the program $\langle \sigma \rangle$ and uses it to update the global state g' . This is done by taking the full body $\langle \sigma, s_0, g_0 \rangle$, where σ is a valid statement, s_0 is the local state and g_0 is the global state. s_0 is defined to say that no mutexes exist on startup, and g_0 says that the only thread that exists is the main thread τ_0 , which starts with ff indicating it has not yet finished execution.

4.2.2 Standard Type System & Big Step Rules

This section presents the standard type system and the big-step rules of ParaLang. A selection of the most influential constructs is presented and explained, starting with the most fundamental expressions and progressing to some of the more complex statements.

Arithmetic & Binary Operations

ParaLang uses a combined set of binary operators covering arithmetic, logical, and comparison operations. All operators are handled through a Single Type rule $[\star_{TS}]$ and a single BS rule $(\star_{BS})$. By defining $\star$ as a placeholder representing all supported operators, only one rule is needed instead of one per operator. The type rule requires both operands e_1 and e_2 to be well-typed under the same type t in the combined typing context (γ_s, γ_g) . It then concludes that the full expression $e_1 \star e_2$ also has type t , reflecting that all supported operators are homogeneous, they do not mix types across operands.

$$(\star_{TS}) \frac{(\gamma_s, \gamma_g) \vdash e_1 : t \quad (\gamma_s, \gamma_g) \vdash e_2 : t}{(\gamma_s, \gamma_g) \vdash e_1 \star e_2 : t} \quad \star \in \left\{ \begin{array}{l} +, -, *, /, \%, ^, \&\&, ||, \\ <, >, <=, >=, ==, != \end{array} \right\}$$

The $(\star_{TS})$ rule is kept general, meaning both operands must share the same type, and the result then inherits that same type. This reflects that all operators are treated uniformly and do not mix types across operands. The rule does not distinguish between arithmetic operators such as $+$ and $-$, logical operators such as $\&\&$ and $||$, or comparison operators such as $<$ and $==$. Instead, all operators are handled in the same way. Type correctness for specific combinations is enforced implicitly through the value domain rather than at the syntactic type level. For instance, $\&\&$ is only meaningful with boolean, but this is not captured by the rule itself.

$$(\star_{BS}) \frac{(s, g) \vdash e_1 \rightarrow (v_1, g') \quad (s, g') \vdash e_2 \rightarrow (v_2, g'')}{(s, g) \vdash e_1 \star e_2 \rightarrow (v, g'')} \quad \star \in \left\{ \begin{array}{l} +, -, *, /, \%, \wedge, \&\&, ||, \\ <, >, <=, >=, ==, != \end{array} \right\} \\ v = v_1 \star v_2$$

The $(\star_{BS})$ rule defines how binary expressions are evaluated at runtime. The left expression e_1 is evaluated first in the state (s, g) . This produces the value v_1 and potentially updates the global state to g' . The right expression e_2 is then evaluated in the updated state (s, g') , producing v_2 and g'' . The final value v is computed as $v = v_1 \star v_2$. This gives all binary expressions a consistent left-to-right evaluation order. For purely arithmetic expressions, the order does not matter, but when expressions contain function calls that interact with the global state, the order becomes important since it determines when side effects occur.

Together, the two rules explain the use of binary expressions throughout ParaLang. The type rule $(\star_{TS})$ ensures that both operands are well-typed and share the same type before proceeding, while the big-step rule $(\star_{BS})$ explains how the expression is evaluated at runtime, including the evaluation order and how the global state is carried through the execution.

Variables

In this section the different rules for variables is shown and how they work in tandem with each other.

Literals: All values have their own rule for how a literal value is assigned a type t .

$$\begin{array}{ll} [\text{INT}_{TS}] \frac{}{(\gamma_s, \gamma_g) \vdash z : \text{Int}} & (\text{FLOAT}_{TS}) \frac{}{(\gamma_s, \gamma_g) \vdash \phi : \text{Float}} \\ (\text{BOOL}_{TS}) \frac{}{(\gamma_s, \gamma_g) \vdash b : \text{Bool}} & (\text{CHAR}_{TS}) \frac{}{(\gamma_s, \gamma_g) \vdash 'c' : \text{Char}} \\ (\text{STRING}_{TS}) \frac{}{(\gamma_s, \gamma_g) \vdash \text{"str"} : \text{String}} & (\text{THREAD}_{TS}) \frac{}{(\gamma_s, \gamma_g) \vdash \tau : \text{Bool}} \end{array}$$

The literal type rules have no premises, meaning they hold unconditionally. Each rule simply assigns the corresponding type to the given literal value in any typing context (γ_s, γ_g) .

$$\begin{array}{c}
(\text{INT}_{BS}) \frac{}{(s, g) \vdash n \rightarrow (n, g)} \quad n \in \mathbb{Z} \qquad (\text{FLOAT}_{BS}) \frac{}{(s, g) \vdash \phi \rightarrow (\phi, g)} \quad \phi \in \mathbb{R} \\
(\text{BOOL}_{BS}) \frac{}{(s, g) \vdash b \rightarrow (b, g)} \quad b \in \{tt, ff\} \qquad (\text{CHAR}_{BS}) \frac{}{(s, g) \vdash 'c' \rightarrow (c, g)} \quad c \in \text{Char} \\
(\text{STRING}_{BS}) \frac{}{(s, g) \vdash \text{"str"} \rightarrow (str, g)} \quad str \in \text{String} \\
(\text{THREAD}_{BS}) \frac{}{(s, g) \vdash \tau \rightarrow (b, g)} \quad b \in \{tt, ff\}
\end{array}$$

The big-step rules evaluate a literal in the local state s and global state g , returning the value itself together with the unchanged global state. The only outlier is the thread τ , since a thread is not a value that can be used directly. Instead, the returned value is a boolean that signals the state of the thread, indicating whether it is running or completed, using tt for completed and ff for still running.

Variable Lookup: When retrieving a value from a given variable x , the rules determine whether the variable exists in the local typing context γ_s and retrieves its type from there, or whether it is a shared variable stored in the global typing context γ_g .

$$\begin{array}{c}
[\text{VAR}_{TS}] \frac{\Delta(x) = t}{(\gamma_s, \gamma_g) \vdash x : t} \quad \gamma_s = (\Delta, b_l, (b_f, t_f)) \qquad [\text{VAR-GLOBAL}_{TS}] \frac{\gamma_g(x) = t}{(\gamma_s, \gamma_g) \vdash x : t} \quad x \notin \gamma_s
\end{array}$$

Given the typing contexts (γ_s, γ_g) , the rules attempt to retrieve a variable with the name x . As described in the typing context section [Section 4.2.1], Δ maps variable names to their types within the current scope, and it is this mapping that is used to look up the type of x .

$$\begin{array}{c}
(\text{VAR}_{BS}) \frac{s(x) = l \quad g(l) = v}{(s, g) \vdash x \rightarrow (v, g)} \qquad (\text{VAR-GLOBAL}_{BS}) \frac{g(x) = v}{(s, g) \vdash x \rightarrow (v, g)} \quad x \notin s
\end{array}$$

After the input has been checked in the typing context, the big-step rules are used at runtime, where a given local state s and a given global state g are used to evaluate a variable x .

x is used in the local state $s(x)$ to find the location in memory l , which is then used in the global state $g(l)$ to find the associated value. If the variable is a shared variable, $x \notin s$, the

global variable rule is used instead, where it maps directly to the global state $g(x)$ instead of going through the local state first.

Declaration and Assignment: When creating variables, two things need to happen: the variable needs to be declared and it needs to be assigned a value for a given location.

$$\begin{array}{c}
[\text{DEC}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : t \quad (\gamma'_s, \gamma_g) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash m \ t \ x := e; \sigma : \text{ok}} \quad \begin{array}{l} m \neq \text{shared} \\ \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \gamma'_s = (\Delta[x \mapsto t], b_l, (b_f, t_f)) \end{array} \\
[\text{DEC-SHARED}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : t \quad (\gamma_s, \gamma_g[x \mapsto t]) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash m \ t \ x := e; \sigma : \text{ok}} \quad \begin{array}{l} m = \text{shared} \\ x \notin \gamma_g \end{array}
\end{array}$$

In ParaLang, declaring a variable takes the typing contexts (γ_s, γ_g) , a modifier m , a type t , and a variable name x , and checks that the declaration is well-typed. The difference between the local and shared versions is that the local version updates γ'_s by mapping x to its type, while the shared version updates $\gamma_g[x \mapsto t]$, reflecting where the variable is stored.

$$\begin{array}{c}
(\text{DEC}_{BS}) \frac{(s, g) \vdash e \rightarrow (v, g') \quad \langle \sigma, s[x \mapsto l], g'[l \mapsto v] \rangle \rightarrow (s', g'')}{\langle m \ t \ x := e; \sigma, s, g \rangle \rightarrow (s', g'')} \quad m \neq \text{shared} \\
(\text{DEC-SHARED}_{BS}) \frac{(s, g) \vdash e \rightarrow (v, g') \quad \langle \sigma, s, g'[x \mapsto v][\mu_x \mapsto \text{free}] \rangle \rightarrow (s', g'')}{\langle m \ t \ x := e; \sigma, s, g \rangle \rightarrow (s', g'')} \quad m = \text{shared}
\end{array}$$

Declaration in the big-step semantics works with the given state body $\langle m \ t \ x := e; \sigma, s, g \rangle$. The first symbol m indicates whether the variable is shared or local, the second symbol t denotes the type of the variable, and the third symbol x is the variable name. The result of the operation is (s', g'') , where s' is the updated local state and g'' is the updated global state.

In both versions, the first premise evaluates the expression e to a value v and updates the global state to g' . In the local version, a fresh location l is created in s , and the value v is stored in the global state g' at that location, resulting in updated states s' and g'' . In the shared version, the mapping happens directly in g' using the variable name x , which is mapped to the value v , and a fresh mutex μ_x is created for the variable, after which the states are updated in a similar way to the local case.

$$[\text{Ass}_{TS}] \frac{\Delta(x) = t \quad (\gamma_s, \gamma_g) \vdash e : t}{(\gamma_s, \gamma_g) \vdash x = e : \text{ok}} \quad \gamma_s = (\Delta, b_l, (b_f, t_f))$$

The assignment works in a similar way to (Var_{TS}) . It uses the given state (γ_s, γ_g) and concludes that the variable is assigned a value from an expression e , and that the assignment is well-typed. This works by extracting the variable x from Δ , which is inside γ_s . With x , the corresponding type can be extracted and used to ensure that the type of the expression e matches.

$$(\text{Ass}_{BS}) \frac{s(x) = l \quad (s, g) \vdash e \rightarrow (v, g')}{\langle x = e, s, g \rangle \rightarrow (s, g'[l \mapsto v])}$$

Assignment takes $\langle x = e, s, g \rangle$ and evaluates it to $(s, g'[l \mapsto v])$. x is used to store the result of the evaluated expression, and s and g are used to represent the local and global states, respectively, indicating where values are stored. The first premise finds the location l for the variable x stored in the local state s . The second premise uses the local and global states (s, g) to evaluate the expression to the value v and produces the updated global state g' . In the conclusion, the location l is then updated with the value v .

Composition

When executing multiple statements, there needs to be an order of operations. This is what composition defines. It works as a fundamental part of the order of execution when statements are evaluated.

$$(\text{Comp}_{TS}) \frac{(\gamma_s, \gamma_g) \vdash \sigma_1 : \text{ok} \quad (\gamma_s, \gamma_g) \vdash \sigma_2 : \text{ok}}{(\gamma_s, \gamma_g) \vdash \sigma_1; \sigma_2 : \text{ok}}$$

$$(\text{Comp-Abort}_{TS}) \frac{(\gamma_s, \gamma_g) \vdash \sigma_1 : \text{ok} \quad (\gamma_s, \gamma_g) \vdash \sigma_2 : \text{ok}}{(\gamma_s, \gamma_g) \vdash \sigma_1; \sigma_2 : \text{ok}}$$

Both (Comp_{TS}) and (Comp-Abort_{TS}) take (γ_s, γ_g) and check that both σ_1 and σ_2 are well-typed statements. The two rules are identical at the type level since the type system

does not distinguish between normal and aborted composition, this distinction only becomes relevant at runtime.

$$(\text{COMP}_{BS}) \frac{\langle \sigma_1, s, g \rangle \rightarrow (s', g') \quad \langle \sigma_2, s', g' \rangle \rightarrow (s'', g'')}{\langle \sigma_1; \sigma_2, s, g \rangle \rightarrow (s'', g'')}$$

$$(\text{COMP-ABORT}_{BS}) \frac{\langle \sigma_1, s, g \rangle \rightarrow r(s', g')}{\langle \sigma_1; \sigma_2, s, g \rangle \rightarrow r(s', g')} \quad r \in \{\text{cnt}, \text{brk}, \text{ret}\}$$

In the semantics of composition, the input $\langle \sigma_1; \sigma_2, s, g \rangle$ is evaluated. σ_1 is executed first, and its result updates the local state s' and the global state g' , which are then passed to σ_2 . The difference between the two rules is that (Comp-Abort_{BS}) is used when σ_1 produces a control signal $r \in \{\text{cnt}, \text{brk}, \text{ret}\}$, in which case σ_2 is skipped and the control signal is propagated directly.

Expression as Statement

- Exp

$$[\text{EXP}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : t}{(\gamma_s, \gamma_g) \vdash e : \text{ok}}$$

$$(\text{EXP}_{BS}) \frac{(s, g) \vdash e \rightarrow (v, g')}{\langle e, s, g \rangle \rightarrow (s, g')}$$

Block

- Block

$$[\text{BLOCK}_{TS}] \frac{(\gamma'_s, \gamma_g) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash \{\sigma\} : \text{ok}} \quad \begin{array}{l} \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \gamma'_s = (\Delta, b_l, (b_f, t_f)) \end{array}$$

$$(\text{BLOCK}_{BS}) \frac{\langle \sigma, s, g \rangle \rightarrow (s', g')}{\langle \{\sigma\}, s, g \rangle \rightarrow (s, g')}$$

Functions

Functions as defined in [Requirement 8] are important for ParaLang. The implementation requires that they are split into two parts: function definition and function call. Function definitions declare the body and formal parameters of used to execute a sequence of statements, while function call executes these statements giving values to the parameters. These are shown using Fun-Def and Fun-Call as shown below.

$$(\text{FUN-DEF}_{TS}) \frac{(\gamma_s'', \gamma_g) \vdash \sigma_f : \text{ok} \quad (\gamma_s', \gamma_g) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash t f(m_1 t_1 p_1, \dots, m_n t_n p_n) \sigma_f; \sigma : \text{ok}}$$

$$\begin{array}{l} \forall i \in \{1, \dots, n\} \quad m_i \neq \text{shared} \\ \bar{m} = (m_1, \dots, m_n) \\ \bar{t} = (t_1, \dots, t_n) \\ \gamma_s' = \gamma_s[f \mapsto \text{func}((\bar{m}, \bar{t}), t, \sigma_f)] \\ \gamma_s'' = ([\bar{p} \mapsto \bar{t}], ff, (tt, t)) \end{array}$$

The syntax of the rules require a function name f with return type t , along with a set of parameters, p_i , each with a corresponding modifier, m_i , and type, t_i . Following this is the function body, σ_f , and the scope in which this function is defined, σ .

The type system rule must first check that the function body type checks ok in the state (γ_s'', γ_g) . This means that the function is run in a local static state that only maps the parameters to types, and that keeps track of the fact that the body is being executed inside of a function with return type t . This is seen in the side condition $\gamma_s'' = ([\bar{p} \mapsto \bar{t}], ff, (tt, t))$, where the ff indicates that body inside of the function is not by default a loop, and the (tt, t) indicates that it is within a function with return type t .

Afterwards it checks that all statements in the scope in which the function is defined type checks ok in the state (γ_s', γ_g) . Here γ_s' is the state in which the function is defined with its parameter modifiers and types, return type and function body.

$$(\text{FUN-DEF}_{BS}) \frac{\langle \sigma, s[f \mapsto \text{func}(\bar{p}, \sigma_f)], g \rangle \rightarrow (s', g')}{\langle t f(m_1 t_1 p_1, \dots, m_n t_n p_n) \sigma_f; \sigma, s, g \rangle \rightarrow (s', g')}$$

Using the same syntax, the BS rule runs σ under the state in which the function is defined and evaluates to (s', g') , which is the conclusion of the rule. This rule declares the function and executes the rest of the scope in which the function is defined.

$$(\text{FUN-CALL}_{TS}) \frac{\Delta(f) = \text{func}((\bar{m}, \bar{t}), t, \sigma) \quad (\gamma_s, \gamma_g) \vdash (e_1, \dots, e_n) : \bar{t}}{(\gamma_s, \gamma_g) \vdash f(e_1, \dots, e_n) : t} \quad \gamma_s = (\Delta, b_l, (b_f, t_r))$$

The type system rule for function call gets the function definition from Δ in γ_s , such that it knows the type of all the parameters of the function. Then it evaluates the type of all given expressions $(e_1, \dots, e_n)$ using auxiliary rule *Args* or *Args-Empty* if there were no parameters of the function, and ensures that the types match. If they do, the rule evaluates to t , which is the return type of the function fetched from Δ . Since the body of the function is type checked at declaration, it is ignored here.

$$(\text{FUN-CALL}_{BS}) \frac{s(f) = \text{func}(\bar{p}, \sigma) \quad (s, g) \vdash (e_1, \dots, e_n) \rightarrow (\bar{v}, g') \quad \langle \sigma, s_\emptyset[\bar{p} \mapsto \bar{l}], g'[\bar{l} \mapsto \bar{v}] \rangle \rightarrow (\text{ret}(v), g'')}{(s, g) \vdash f(e_1, \dots, e_n) \rightarrow (v, g''')} \quad \begin{array}{l} g''' = g'' \setminus \bar{l} \\ \forall l \in \bar{l} \quad l \notin g' \end{array}$$

$$(\text{FUN-CALL-EMPTY}_{BS}) \frac{s(f) = \text{func}(\varepsilon, \sigma) \quad \langle \sigma, s_\emptyset, g \rangle \rightarrow (\text{ret}(v), g')}{(s, g) \vdash f() \rightarrow (v, g')}$$

The big step rule for function call is separated into two, where the *Fun-Call-Empty_BS* is a function call to a function with no parameters. Here the rule looks up the body of the function and executes it under a fresh local state $s_\emptyset$.

The *Fun-Call_BS* rule is slightly more complicated, as it also needs to use the parameters of the function called, $\bar{p}$. After looking up the function, it evaluates all input expressions $(e_1, \dots, e_n)$ to values in $\bar{v}$. These values are then stored in the state using both s and g , where s maps to the locations in the global state, g , which maps these location to the values. After the function call, it removes the new locations from g again.

Both of the rules expect to receive *ret* when executing the body of the function. These are flags used to signal out from the function that the end of the function has been reached, and if they carry a value, it should be caught by this rule and used in the conclusion as v .

Control-Flow Statements

Control-flow statements like `continue` and `break` are really useful when working with loops, since they allow the programmer to determine when to go to next iteration of a loop or break out of it. Furthermore, `return` is a necessity for functions, especially if they need to compute something, as it can return a value.

$$\begin{aligned}
& (\text{CONTINUE}_{TS}) \frac{}{(\gamma_s, \gamma_g) \vdash \text{continue} : \text{ok}} \quad \gamma_s = (\Delta, tt, (b_f, t)) \\
& (\text{BREAK}_{TS}) \frac{}{(\gamma_s, \gamma_g) \vdash \text{break} : \text{ok}} \quad \gamma_s = (\Delta, tt, (b_f, t)) \\
& (\text{RETURN-VOID}_{TS}) \frac{}{(\gamma_s, \gamma_g) \vdash \text{return} : \text{ok}} \quad \gamma_s = (\Delta, b_l, (tt, \text{Void})) \\
& (\text{RETURN}_{TS}) \frac{(\gamma_s, \gamma_g) \vdash e : t}{(\gamma_s, \gamma_g) \vdash \text{return } e : \text{ok}} \quad \begin{array}{l} \gamma_s = (\Delta, b_l, (tt, t)) \\ t \neq \text{Void} \end{array}
\end{aligned}$$

All statements have similar TS rules, where they all evaluate to `ok`. Noteworthy details are, that `Continue_TS` and `Break_TS` require γ_s to have `tt` as the first boolean, which is seen in the side conditions of the rules. This ensures that the statements are evaluated inside of a loop, as this is the only case where the first boolean is equal to `tt`.

Similarly `Return-Void_TS` and `Return_TS` must have the second boolean be `tt`, as this indicates that the statements are within functions. `Return-Void_TS` also shows that the type `t` saved in γ_s is `Void`. `Return_TS` needs to evaluate the expression `e` which is returned by the function, since it needs to match the function return type `t` stored in γ_s .

$$\begin{aligned}
& (\text{CONTINUE}_{BS}) \frac{}{\langle \text{continue}, s, g \rangle \rightarrow (\text{cnt}, g)} \quad (\text{BREAK}_{BS}) \frac{}{\langle \text{break}, s, g \rangle \rightarrow (\text{brk}, g)} \\
& (\text{RETURN-VOID}_{BS}) \frac{}{\langle \text{return}, s, g \rangle \rightarrow (\text{ret}, g)} \quad (\text{RETURN}_{BS}) \frac{(s, g) \vdash e \rightarrow (v, g')}{\langle \text{return } e, s, g \rangle \rightarrow (\text{ret}(v), g')}
\end{aligned}$$

The BS rules are also quite similar and unlike most of the statement rules produce `cnt`, `brk`, `ret` and `ret(v)` contrary to `s` as described previously in [Section 4.2.1]. These are control signals or flags, that are used in the loop rules, which can be seen in the appendix [Section 9.2], and in the `Fun-Call_BS` and `Fun-Call-Empty_BS` rules as described in the previous section.

The Return_BS rule unlike the others has to evaluate expression, e , to a value, v , that is returned in the statement. The fact that the syntax uses e instead of just v allows developers to write expressions, such as "return (2 + 3)" as well as returning variables.

4.2.3 Concurrency Semantics

The semantic rules that make ParaLang stand out from other languages, are the rules that allow for concurrency in parallel programming. These range from creating and using threads, to having critical sections that automatically and safely acquires and releases mutex locks in sections where shared variables are used.

Threads

Using threads is the key to concurrency in ParaLang, and are a must have for the language according to [Requirement 13]. The implementation of threads in ParaLang follows [Requirement 18], where the threads resolve to a boolean value, that cannot be manually reassigned.

$$\begin{array}{c} (\text{THREAD}_{TS}) \frac{}{(\gamma_s, \gamma_g) \vdash \tau : \text{Bool}} \quad (\text{THREAD}_{BS}) \frac{}{(s, g) \vdash \tau \rightarrow (b, g)} \quad b \in \{tt, ff\} \end{array}$$

These two rules are the expression evaluation of a thread. The TS rule ensures that the thread, τ , evaluates to a boolean value, and the BS rule shows that τ when interpreted does in fact evaluate to a boolean value. There is however more to threads, as they need to be able to execute a block of statements concurrently. This is shown using the rules Thread-Spawn_TS and Thread-Spawn_BS, which show the creation of threads.

$$(\text{THREAD-SPAWN}_{TS}) \frac{(\gamma_s, \gamma_g) \vdash \sigma_b : \text{ok} \quad (\gamma_s, \gamma'_g) \vdash \sigma_c : \text{ok}}{(\gamma_s, \gamma_g) \vdash \text{Thread } \tau \Rightarrow \sigma_b; \sigma_c : \text{ok}} \quad \begin{array}{l} \tau \notin \gamma_g \\ \gamma'_g = \gamma_g[\tau \mapsto \text{thread}] \end{array}$$

Threads are stored in the global state, and the TS rule for spawning threads must therefore ensure that τ is not in γ_g already. It then checks that the body of the thread, σ_b , type checks okay with the current global state, and that the continuation, σ_c , type checks okay when σ_g maps τ to a thread.

$$\begin{array}{c}
\text{(THREAD-SPAWN}_{BS}\text{)} \frac{\langle \sigma_c, s, g[\tau \mapsto \text{thread}(ff, \sigma_b, s)] \rangle \rightarrow (s', g')}{\langle \text{Thread } \tau \Rightarrow \sigma_b; \sigma_c, s, g \rangle \rightarrow (s', g')}
\end{array}
\begin{array}{l}
\tau \notin g \\
\text{thread } \tau \text{ runs parallel} \\
\text{when } \tau \text{ reaches a final state,} \\
g'' \text{ is the global state, and} \\
g''' = g''[\tau \mapsto \text{thread}(tt, \sigma_b, s)]
\end{array}$$

The BS rule is a little more complex. It also checks that the thread is not in the global runtime state, and then stores the thread in g , and then proceeds with the scope in which the thread is declared in the global scope. This is seen in the premise of the rule, where it runs σ_c under the scope, $s, g[\tau \mapsto \text{thread}(ff, \sigma_b, s)]$. Here, ff means that the thread has not finished running.

Because of the fact that BS semantics struggle with describing small steps, and that concurrency is very difficult to describe with rules, especially within the scope of this project, the side conditions try to explain it simply. Here it states that the thread, τ , runs parallel, and when it reaches a final state, it is assumed that the global state is g'' , and the global state is updated, such that τ maps to the same thread, but with tt saved instead of ff , which indicates that the thread has finished. This is used by the Await rules.

Await

In a programming language that revolves around different threads running simultaneously, the ability to wait for one- or more threads is very useful as described in [Requirement 12]. This feature has been split into two functions in the program: `awaitAll` and `awaitAny`. The key difference between the two functions, is that `awaitAll` waits until all the thread arguments are done running, while `awaitAny` waits only until any of them is finished.

$$\begin{array}{c}
\text{(AWAIT-ALL}_{TS}\text{)} \frac{\forall i \in \{1, \dots, n\} \quad \gamma_g(\tau_i) = \text{thread}}{(\gamma_s, \gamma_g) \vdash \text{awaitAll}(\tau_1, \dots, \tau_n) : \text{ok}} \\
\text{(AWAIT-ANY}_{TS}\text{)} \frac{\forall i \in \{1, \dots, n\} \quad \gamma_g(\tau_i) = \text{thread}}{(\gamma_s, \gamma_g) \vdash \text{awaitAny}(\tau_1, \dots, \tau_n) : \text{ok}}
\end{array}$$

The only difference between the TS rules, for the two functions, is the syntax. Both check that all arguments are threads, and if so they both evaluate to `ok`.

$$\begin{aligned}
& (\text{AWAIT-ALL-DONE}_{BS}) \frac{\forall i \in \{1, \dots, n\} \quad g(\tau_i) = \text{thread}(tt, \sigma_t, s_t)}{\langle \text{awaitAll}(\tau_1, \dots, \tau_n), s, g \rangle \rightarrow (s, g)} \\
& (\text{AWAIT-ANY-DONE}_{BS}) \frac{\exists i \in \{1, \dots, n\} \quad g(\tau_i) = \text{thread}(tt, \sigma_t, s_t)}{\langle \text{awaitAny}(\tau_1, \dots, \tau_n), s, g \rangle \rightarrow (s, g)}
\end{aligned}$$

For the BS rules, there is a key difference that separates them, apart from just the syntax. `Await-All` must check that all threads given as arguments are marked as "done" in the global state, while `Await-Any` checks that any τ_i is "done". This is checked through the mapping of τ_i , which must evaluate to a thread, which has tt as the first argument, thus showing that the thread has finished.

Critical

The critical sections are the most complex semantic rules of ParaLang, and because of this, along with the use of BS-semantics, some of the details are explained in text instead of the rules. This will be discussed further in (reference to discussion).

The critical sections need to acquire the mutex locks that are given as arguments in the critical function. These need to be acquired in hierarchical order according with the "Acquire Needed Locks First" deadlock avoidance strategy [Section 2.2.2]. As mentioned in the strategy, if a lock is required, which is of lower hierarchical order than any of the locks already acquired, all locks should be dropped and acquired again. An example of this can happen when a critical section requires lock 2, and another critical section, that requires lock 1, is within this section.

$$(\text{CRITSEC}_{TS}) \frac{(\gamma_s, \gamma_g) \vdash (x_1, \dots, x_n) : \bar{t} \quad (\gamma_s, \gamma_g) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash \text{critical}(x_1, \dots, x_n) \sigma : \text{ok}} \quad x_i \notin \gamma_s \quad \forall i \in \{1, \dots, n\}$$

The TS rule for critical sections need to ensure, that all variable arguments successfully evaluate to a type, and that none of them are within γ_s , which essentially means that they all are in γ_g , and thus are shared variables. It also needs to ensure that the statement σ evaluates to ok.

$$\begin{array}{c}
 \forall x_i \in N, \forall x_j \in s(\mu^O) \quad \text{ord}(x_i) > \text{ord}(x_j) \\
 \forall x_i \in N \quad g(\mu_{x_i}) = \text{free} \\
 \langle \sigma, s, g' \rangle \rightarrow (s', g'') \\
 \text{(CRIT-SEC-NO-REORDER}_{BS}) \frac{}{\langle \text{critical}(x_1, \dots, x_n) \sigma, s, g \rangle \rightarrow (s', g''')}
 \end{array}
 \begin{array}{l}
 \tau = \text{current thread} \\
 N = \{x_1, \dots, x_n\} \setminus s(\mu^O) \\
 g' = g[\mu_{x_i} \mapsto \tau \mid x_i \in N] \\
 g''' = g''[\mu_{x_i} \mapsto \text{free} \mid x_i \in N]
 \end{array}$$

$$\begin{array}{c}
 \exists x_i \in N, x_j \in s(\mu^O) \quad \text{ord}(x_i) < \text{ord}(x_j) \\
 \forall x_i \in \bar{x} \quad g'(\mu_{x_i}) = \text{free} \\
 \langle \sigma, s, g'' \rangle \rightarrow (s', g''') \\
 \text{(CRIT-SEC-REORDER}_{BS}) \frac{}{\langle \text{critical}(x_1, \dots, x_n) \sigma, s, g \rangle \rightarrow (s', g''')}
 \end{array}
 \begin{array}{l}
 \tau = \text{current thread} \\
 N = \{x_1, \dots, x_n\} \setminus s(\mu^O) \\
 \bar{x} = \text{sort}(N \cup s(\mu^O)) \\
 g' = g[\mu_{x_j} \mapsto \text{free} \mid x_j \in s(\mu^O)] \\
 g'' = g'[\mu_{x_i} \mapsto \tau \mid x_i \in \bar{x}] \\
 g''' = g''[\mu_{x_i} \mapsto \text{free} \mid x_i \in \bar{x}]
 \end{array}$$

Text describing Reorder and No-Reorder

$$\begin{array}{c}
 \forall x_i \in \bar{x} \quad g(\mu_{x_i}) = \tau \\
 \langle \sigma, s, g \rangle \rightarrow (s', g') \\
 \text{(CRIT-SEC-RUN}_{BS}) \frac{}{\langle \text{critical}(x_1, \dots, x_n) \sigma, s, g \rangle \rightarrow (s', g'')}
 \end{array}
 \begin{array}{l}
 \tau = \text{current thread} \\
 \bar{x} = \text{sort}(\{x_1, \dots, x_n\} \cup s(\mu^O)) \\
 g'' = g'[\mu_{x_i} \mapsto \text{free} \mid x_i \in \bar{x} \setminus s(\mu^O)]
 \end{array}$$

Text describing Run

Chapter 5

Development

As specified by requirement [2], the ParaLang code must be transpiled into Java code. This process happens through several steps; First, the syntax rules must be implemented as a grammar, such that a lexer can recognize the logical components of the ParaLang code as tokens. This then lets a parser structure the tokens into an Abstract Syntax Tree (AST) using the grammar. To check the correctness of the ParaLang code and subsequently translate it into Java code, a series of visitors are used to move through the AST and handle different aspects of the transpiler pipeline.

This section will go through how the grammar, lexer and parser is implemented, the general purpose of each visitor in the pipeline, and how some more complex parts of ParaLang are handled in each phase.

5.1 Grammar and ANTLR4

Writing a lexer and parser from the bottom up is a relatively large undertaking, so a common practice is to use a parser generator. Since the target language is Java according to [Requirement 2], ANTLR4 was chosen for this project, as it is an established and well-documented parser generator for Java [42].

5.1.1 ANTLR4

ANTLR4 (Another Tool for Language Recognition) takes a grammar file as input and automatically generates a lexer, parser, and visitor pattern infrastructure in the target

language. The grammar file defines two kinds of rules: lexer rules, which describe how raw characters are grouped into tokens, and parser rules, which describe how tokens are combined into meaningful structures. By convention, lexer rules are written in `UPPER_CASE` and parser rules in `lower_case`.

From a single grammar file, ANTLR4 generates several Java classes. The lexer and parser are generated automatically, along with a base visitor class containing an empty `visit` method for every parser rule. These visitor methods can then be selectively overridden to implement the compilation phases without modifying the generated ANTLR4 code.

5.1.2 Lexer Rules

The lexer is responsible for reading the raw character stream and grouping characters into tokens, which the parser then operates on. Here are some of ParaLang's lexer rules:

```

175 NEGATIVE : '-';
176 INT : [0-9]+ ;
177 FLOAT : [0-9]+ '.' [0-9]* ;
178 BOOL : 'true' | 'false' ;
179 CHAR : '\'' . '\'' ;
180 STRING : '"' .*? '"' ;
181 THREAD : BOOL ;
182 PREFIX : 'shared' 'static'? | 'const' 'static'? | ('shared' | 'const')? 'static' ;
183 TYPE : 'int' | 'float' | 'bool' | 'char' | 'string' | 'void' | 'thread' ;
184 ID : [a-zA-Z_] [a-zA-Z0-9_]* ;

```

Code 6: Lexer rules.

An important property of ANTLR4 is that when two lexer rules can match the same input, the rule defined first takes priority. This is used deliberately with `TYPE` and `ID`: the keyword `int` would technically match `ID`, but since `TYPE` is defined first, it is always recognized as a type keyword rather than an identifier. This means reserved words do not need to be handled separately, as their ordering in the grammar file is sufficient.

Similarly, whitespace and comments are discarded before the parser ever sees them, using the `skip` directive:

```

186 LINE_COMMENT : '//' ~[\r\n]* -> skip ;
187 BLOCK_COMMENT : '/*' .*? '*/' -> skip ;
188 WS : [ \t\r\n]+ -> skip ;

```

Code 7: Lexer comment & whitespace skip rules.

Note how `.*?` is used in `BLOCK_COMMENT` to denote zero or more of any character. The question mark makes the match *lazy*, meaning it stops as soon as it can. To illustrate why

```
1  /* first comment */ code /* second comment */
```

this matters, consider the following input:

A lazy match consumes only up to the first `*/`, correctly producing two separate comments. A *greedy* match, by contrast, would consume as far as possible, swallowing everything from the opening `/*` all the way to the final `*/`, meaning `code` would be treated as part of the comment rather than as a token for the parser.

5.1.3 Parser Rules

Parser rules define the syntactic structure of ParaLang by describing how tokens combine into statements, expressions, and other constructs. ANTLR4 uses *LL(*)* parsing[43], meaning it reads tokens from left to right and constructs a leftmost derivation, looking ahead as many tokens as necessary to resolve ambiguities. The parser outputs a nested tree structure of all rules it navigates through, and the tokens at the end of each branch:

```
(statement (statementPrime (declaration (typeRef int) x)) ;)
```

Program and Statements

The top-level rule is `program`, which simply states that a program is a sequence of zero or more statements followed by an end-of-file marker:

```
3  program
4    : statement* EOF
5    ;
```

Code 8: Program top-level rule in `OurGrammar.g4`[9.1].

This structure makes it a very functional language, and allows for a bare bones ParaLang file with functional code, which can run without a class or main function containing it. These are automatically generated when transpiling to Java code. Statements are split into two rules. The outer `statement` rule handles constructs that manage their own termination, such as blocks and control flow, while `statementPrime` groups constructs that require an explicit trailing semicolon:

```

7  statement
8      : statementPrime ';'
9      | assignment
10     | threadAssignment
11     | ifStatement
12     | forStatement
13     | whileStatement
14     | criticalSection
15     | block
16     ;
17
18  statementPrime
19      : reassignment
20      | declaration
21      | awaitStatement
22      | continueStatement
23      | breakStatement
24      | returnStatement
25      | printStatement
26      | expr
27      ;

```

Code 9: Statement and statementPrime rules in OurGrammar.g4[9.1].

This separation avoids redundancy. Constructs like `ifStatement` and `block` end with a closing brace `}`, so they do not need a semicolon. By contrast, a `declaration` such as `int x` has no natural terminator, so it is placed under `statementPrime` to ensure statements are ended by semicolon at a single point in the grammar rather than in every individual rule.

Assignments and Function Declarations

One of the key design decisions in the grammar is that variable assignments and function declarations both stem from a single `assignment` rule, which contains the `PREFIX?` `typeRef ID` prefix they both have in common:


```

29 assignment
30 : PREFIX? typeRef ID (assVar ';' | assFunc)
31 ;
32
33 assVar
34 : '=' expr
35 ;
36
37 assFunc
38 : '(' (PREFIX? typeRef ID (',' PREFIX? typeRef ID)*)? ')' block
39 ;

```

Code 10: Assignment and its sub-components rules in OurGrammar.g4[9.1].

For example, the variable assignment `int x = 5;` matches `assignment` via `assVar`, while the function declaration:

```

1 int add(int a, int b) {
2     return a + b;
3 }

```

Code 11: Java example for assFunc grammar.

Matches `assignment` via `assFunc`. The parser reads the shared prefix `int add` and then uses the next token to disambiguate: a `'='` token leads to `assVar`, while a `'('` token leads to `assFunc`. Functions are declared the same way variables are, differing only in what follows the identifier. This was partly done to assist ANTLR4 in understanding which is which, as both function assignment and variable assignment would need to begin with `PREFIX? typeRef ID`. This would make ANTLR4 unsure which token to parse, so they were joined together in the same base function.

Type References and Arrays

The `typeRef` rule describes both primitive types and array types of arbitrary dimension:

```

63 typeRef
64 : TYPE ('[' (ID | INT)? ']')*
65 ;

```

Code 12: Typeref rules in OurGrammar.g4[9.1].

The `*` quantifier allows zero or more bracket pairs, so `int` matches as a scalar, `int[]` as a one-dimensional array, and `int[2][10]` as a two-dimensional array with explicit sizes. Dimension sizes are optional and can be either an integer literal or an identifier, allowing both static and dynamic sizing to be expressed in the type declaration. This single rule is reused across variable declarations, function parameters, and return types, keeping the

grammar consistent.

Expressions and Operator Precedence

Operator precedence is encoded directly in the grammar by splitting expressions into a hierarchy of rules, each handling one precedence level:

```

115 expr
116   : equal ('&&' | '||') expr
117   | equal
118   ;
119
120 equal
121   : comp ('==' | '!=') equal
122   | comp
123   ;
124
125 comp
126   : additive ('<' | '>' | '<=' | '>=') comp
127   | additive
128   ;
129
130 additive
131   : mult ('+' | '-') additive
132   | mult
133   ;
134
135 mult
136   : power ('*' | '/' | '%') mult
137   | power
138   ;
139
140 power
141   : factor '^' power
142   | factor
143   ;

```

Code 13: Boolean and arithmetic expression rules in [OurGrammar.g4](#) [9.1].

Each rule can either apply its operator and recurse, or fall through to the next level. Since a rule can only reach its operator after descending through all higher-priority rules, lower rules in the hierarchy bind more tightly. Consider the expression `3 * 2 + 1`. The lexer first tokenizes this into:

```
(additive (mult (power (factor 3)) * (mult (power (factor 2)))) + (additive
(mult (power (factor 1)))))
```

The parser then enters at `expr`, falling through to `additive`, which delegates its left operand downward. `mult` claims `3 * 2` before `additive` ever sees the `+`, returning a subtree to `additive`, which then matches `+` and consumes `1` on the right. The resulting parse tree is:

```
1      additive (+)
2      /      \
3      mult (*)  factor (1)
4      /      \
5      factor(3) factor(2)
```

The leaf of the expression hierarchy is `factor`, which handles everything that can resolve to a value, meaning literals, identifiers, function calls, array accesses, casts, and parenthesized expressions:

```
154 factor
155   : NEGATIVE factor
156   | functionCall
157   | arrayIndex
158   | arrayLiteral
159   | ID
160   | INT
161   | FLOAT
162   | BOOL
163   | CHAR
164   | STRING
165   | THREAD
166   | '(' expr ')'
167   | castExpression
168   | read
169   ;
```

Code 14: Factor rules in `OurGrammar.g4`[9.1].

Unary negation is handled here via the `NEGATIVE` token, which is defined as a separate lexer rule rather than reusing the `-` literal from `additive`. This avoids ambiguity between the binary minus in `additive` and the unary minus in `factor`, since ANTLR4 would otherwise be unable to distinguish them during lexing.

Explicit Casting

ParaLang allows for explicit conversions between compatible data types using the syntactic form illustrated in [Code 15]. The `castExpression` consists of three main elements: The `cast` keyword, the parenthesized `TYPE` to cast to, and finally an `expr`, which can turn into any valid expression, e.g., a function call, a variable, amongst others. While the grammar permits any datatype in a cast, the types: `void`, `thread` and `char` are rejected during type-checking, as they are not valid cast targets. `void` and `thread` are also rejected as expression types.

```

59 castExpression
60   : 'cast' '(' TYPE ')' expr
61   ;

```

Code 15: `castExpression` rule in *OurGrammar.g4* [Section 9.1].

An example of `castExpression` could be: `int myInt = cast(int) myFloat` where the expression `cast(int) myFloat` converts the value of `myFloat` to an `int`.

The `castExpression` is reached through the `factor` rule, as seen in [Code 14]. The `factor` rule represents the highest precedence level in the hierarchy of expressions. The high precedence is important because the cast is performed, for example, before any arithmetic expression or comparisons, among others.

Read Operation

ParaLang allows for read operations, as illustrated in [Code 16]. The `read` rule parses if the parser recognizes the format of: `read` followed by a parenthesized `TYPE`. The grammar allows any datatype when reading input, but `void`, `thread` and `char` are rejected in the type checker, because they are not valid read operations.

```

111 read
112   : 'read' '(' TYPE ')'
113   ;

```

Code 16: `read` rule in *OurGrammar.g4* [Section 9.1].

The read operation is also reached through the `factor` rule, as shown in [Code 14]. Since the `read` rule is treated as a factor, it can occur wherever a factor is allowed in an expression.

Concurrency Constructs

ParaLang provides concurrency through three constructs: `threadAssignment`, `awaitStatement`, and `criticalSection`.

A thread is created by assigning a block to an identifier using the `=>` operator:

```
50 threadAssignment
51   : typeRef ID '=>' block
52   ;
```

Code 17: Thread assignment rule in *OurGrammar.g4*[9.1].

For example, `thread t => { doWork(); }` spawns a concurrent task and binds its handle to `t`. The `thread` type in `typeRef` distinguishes thread handles from ordinary variables at the type level.

Synchronisation on one or more threads is expressed with `awaitStatement`:

```
54 awaitStatement
55   : 'awaitAll' '(' ID (',' ID)* ')'
56   | 'awaitAny' '(' ID (',' ID)* ')'
57   ;
```

Code 18: `awaitAll` and `awaitAny` rules in *OurGrammar.g4*[9.1].

, as described in [Section 4.1.3.

Mutual exclusion over shared variables is handled by `criticalSection`:

```
103 criticalSection
104   : 'critical' '(' ID (',' ID)* ')' block
105   ;
```

Code 19: Critical section rule in *OurGrammar.g4*[9.1].

The identifiers in the parentheses name the shared variables that must be protected. The code generator uses these to acquire the corresponding locks in a globally consistent ascending order before entering the block, and releases them after exiting the block, preventing deadlocks from inconsistent lock ordering. The specifics of locking and releasing of the mutexes happens according to the Acquire Needed Locks First deadlock avoidance strategy [Section 2.2.2, Requirement 24]. The specifics of how critical sections are usable is further described in [Section 5.3.5, 5.4].

5.2 Compilation Context

NEW CHANGE HERE!



When parsing the input and checking whether the input is valid, then generating the corresponding java code, multiple passes over the input will be required. To ensure that the information gained by the first visitor is carried over to the next, an instance of a class is given as an argument to every visitor, alongside the AST. This class is [CompilationContext.java](#). It is fairly compact in itself, containing only 5 data constructs, as seen in [Code 20].

```

10 public class CompilationContext {
11     // Phase 1: the symbol table, which supports scopes (stack of hashmaps)
12     public final SymbolTable symbolTable = new SymbolTable();
13
14     // Phase 2: every identifier-use node → the Symbol it refers to
15     // Use LinkedHashMap to preserve insertion order (deterministic across runs)
16     public final Map<ParseTree, Symbol> resolvedSymbols = new LinkedHashMap<>();
17
18     // Phase 3: every expr/factor node → its resolved TypeSymbol
19     public final Map<ParseTree, TypeSymbol> nodeTypes = new LinkedHashMap<>();
20
21     // Phase 4: function name → FunctionSymbol (the ftable)
22     // Use LinkedHashMap to keep a stable order when iterating over functions.
23     public final Map<String, FunctionSymbol> ftable = new LinkedHashMap<>();
24
25     // Concurrency: track which variables are declared 'shared' - Phase 3 uses this
26     // to enforce that shared vars are only mutated inside critical sections
27     public final List<String> sharedVariables = new ArrayList<>();
28 }

```

Code 20: Compilation Context class.

The data constructs in the class are gradually filled throughout the visitor pipeline, transferring the necessary key data to allow the next visitors to utilize the data for their functionality. The class itself instantiates every data structure upon creation, where the deeper functionality lies in the classes associated with each of them.

5.2.1 SymbolTable

The [SymbolTable](#) is by far the most utilized data construct in [CompilationContext](#). As shown in []

```

10 public class SymbolTable {
11     private Scope currentScope = null;
12     private final Map<ParseTree, Scope> nodeScopes = new HashMap<>();
13     private int depth = -1; // global scope starts at 0, so -1 means no scopes yet
14
15     public void pushScope(ParseTree node) {
16         currentScope = new Scope(currentScope);
17         nodeScopes.put(node, currentScope);
18         depth++;
19     }

```

Code 21: Symbol table class.

5.3 Visitor Pipeline

Now, the visitor pipeline will be presented, as well as briefly explained to understand what happens where. After the quick introduction on what happens in each visitor, specific features will be presented and explained in depth across visitors. An illustration of the visitor pipeline has been created to further assist in the understanding on the program:

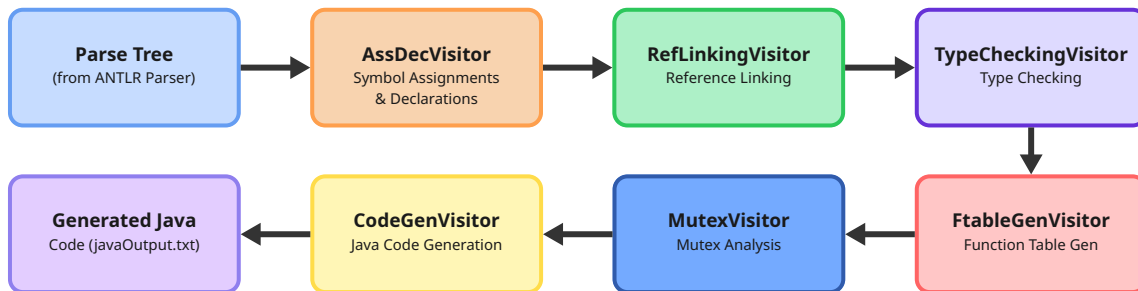


Figure 5.1: Illustration over the visitor pipeline that handles semantic checks and code generation.

5.3.1 Assignment Declaration Visitor

The visitors are all extensions of `OurGramaBaseVisitor`, which is generated by ANTLR and made to traverse the parse tree. The first visitor phase is the assignment and declaration visitor. This visitor's job is to handle symbol declarations, assignments, prefixes, and scope management. It looks through the entire parse tree one node at a time, checking for any matches to the parser generated node types defined in the context free grammar [Section 5.1.3]. It creates `VariableSymbol` objects and `FunctionSymbol`

objects and inserts them into the `SymbolTable` while tracking scope depth for `blocks`, `for loops`, and `while loops`, etc. [Section 5.10]. It also checks for valid combinations of prefixes, so no `const` and `shared` combinations, as they are not compatible.

5.3.2 Reference Linking Visitor

In phase 2, the reference linking visitor resolves identifier references to their declared symbols in the `SymbolTable`. It inserts the newly linked IDs from assignments, reassignments, array accesses, function calls, factors, threads, and await statements into the `resolvedSymbols` map. It checks if variables are declared before referencing them and throws errors if not. The reference linking visitor also handles scope restoration for blocks and loops [Section 5.10] as well as validating critical section [Section 5.4] usage for shared variables. In this phase, the program also updates array dimension sizes/bounds when literals or reassignments occur. It links array accesses and declarations so later compiler phases know the exact symbol and validated sizes [Section 5.8].

5.3.3 Type Checker Visitor

Phase 3 is the type checking visitor, and its job is to check data types of expressions and statements in the `ParseTree`. When type checking it verifies type compatibility for arithmetic on int, float and boolean logic and conditions etc. It also checks for valid types for assignments, function bodies, loop conditions, read / print, thread and await constructs. The type checker also ensures that all references to arrays are within the scope of the arrays dimensions [Section 5.8]. It throws `RuntimeExceptions` when detecting a type mismatch and tracks loop depth to tell if break and continue is within a loop.

5.3.4 Function Table Visitor

In phase 4, the Function Table Generation Visitor builds the function table in the `CompilationContext`. When detecting a function assignment node in the `ParseTree`, the visitor populates the `FunctionSymbol` object with parameters, declared depth and context reference. It also checks if the function contains a critical section for mutex handling in phase 5.

5.3.5 Mutex Visitor

In phase 5, the `MutexVisitor` analyses critical sections and ensures any function calls do not call code that itself contains a critical section. It helps avoid deadlocks and invalid nesting in concurrent execution [Section 5.4].

5.3.6 Code Generation Visitor

In the last phase, phase 6, the Code Generation Visitor traverses the `ParseTree` going through each node, generating equivalent Java source code. It first of all creates an output string with static import libraries for all supported keywords in ParaLang. Then it looks for a `main` keyword in the `ParseTree`. If not found it creates a `Main` class structure around the source code in the correct Java structure. It also creates the functions as methods in said `Main` class. All of this gets appended into the output string of the visitor. Finally, the equivalent Java code is saved in an output file, ready to compile.

5.4 Critical Sections and Mutexes

Critical sections are mythical creatures, understood by none. The underlying mechanisms are constructed using fairy dust, hopes n' dreams, and millennia old duct tape. Do also note, that if you poke it, it will have a non-zero chance of blowing up.

5.5 Thread declaration and Usage

NEW CHANGE HERE! 

Threads support in the compiler is implemented across the semantic analyses and code generation phases. ParaLang allows for declaring and using threads which are ultimately translated into Java `CompletableFuture` constructs for asynchronous execution. The `CompletableFuture` construct spawns a thread and runs a block of code and terminates after code execution. The reason for using `CompletableFuture` instead of the `Future` construct is the ability to use other synchronization methods instead of just `threadName.get()`, which is the only way to pause until completed for `Future`.

The `AssDecVisitor` handles declaration by using its `visitThreadAssignment` method to assign the thread to the symbol table. Following this the `RefLinkingVisitor` looks for the

thread symbol in the `SymbolTable` and throws an error if it has not been declared. The `TypeCheckingVisitor` then validates the thread construct by checking for valid syntax and confirming that the variable is of thread type. Finally the `CodeGenVisitor` generates the corresponding Java output as seen in [Code 22]. It generates a `CompletableFuture` construction that incorporates the thread identifier and the associated code block.

```

283 @Override
284 public String visitThreadAssignment(OurGrammarParser.ThreadAssignmentContext
    ↪ context) {
285     String id = context.ID().getText();
286     String blockCode = visit(context.block());
287     if (blockCode.startsWith(indent())) blockCode =
    ↪ blockCode.substring(indent().length());
288     return indent() + "CompletableFuture<Void> " + id + " =
    ↪ CompletableFuture.runAsync(() -> " + blockCode + ");\n";
289 }

```

Code 22: `visitThreadAssignment` method in `CodeGenVisitor.java`.

This design enables clean expression of concurrent operations in the source language while mapping them efficiently to Java concurrency utilities.

5.6 Await Constructions

NEW CHANGE HERE!

The await statement has two purposes: `AwaitAll`: to pause the program until all defined background threads have completed, or `AwaitAny` to pause the program until any one of the defined background threads has completed. Their syntax are based on the semantics in [Section 5.1.3] and is defined and described in [Section 18]

In `RefLinkingVisitor`, the `visitAwaitStatement` method verifies that every identifier passed to `awaitAll` or `awaitAny` has been declared in the current scope. This step is important because the compiler must ensure that the statement only refers to known background tasks before any further analysis is performed. If an identifier is missing, the program throws an error and an error message.

After reference linking, the `TypeCheckingVisitor` processes the same construct in `visitAwaitStatement` and checks that each provided identifier refers to a thread variable. This guarantees that `awaitAll` and `awaitAny` are only used with threads and not with unrelated variables.

Finally, `CodeGenVisitor` translates the statement into Java code. In `visitAwaitStatement`, it inspects whether the statement is `awaitAll` or `awaitAny` and translates to the corresponding Java implementation. For `awaitAll`, the code waits for every thread to finish execution. For `awaitAny`, the code waits until one of the threads completes, which requires a different coordination strategy than the blocking behavior used by `awaitAll`.

This setup keeps the syntax, semantic validation, and code generation phases clearly separated, which makes the implementation of `awaitStatement` consistent with the rest of the compiler architecture. By resolving identifiers first, then validating their types, and only then generating code, the compiler ensures that invalid concurrency usage is caught early and that the resulting Java output matches the intended behavior of `awaitAll` and `awaitAny`. Below is an example of how the output code looks when `awaitAll` is translated to Java code [Code 23]:

```
1 try {
2     CompletableFuture.allOf(t1, t2).get();
3 } catch (InterruptedException | ExecutionException e) {
4     e.printStackTrace();
5 }
```

Code 23: `awaitAll()` example in `javaOutput.txt`.

It has to be wrapped in a `TRY / catch` construction for good error handling. The Code Generator Visitor translates the `awaitAll(t1, t2)` to the Java-compatible `allOf().get()` method in the `CompletableFuture` library.

5.7 Read Construction

NEW CHANGE HERE!

This section describes the implementation of read operations in ParaLang. The full semantic rules are covered in [Section ??], and the grammar is defined in [Section 16].

The read operation allows the program to obtain user input at runtime. It uses the syntax `read(TYPE)`, where `TYPE` specifies the expected input type.

The Type Checker Visitor is responsible for verifying that the data type trying to be read is valid. The valid data types for a read operation are: `int`, `float`, `bool`, `string`. In case the data type is not one of the valid data types, a runtime exception is thrown. Finally, it returns the data type chosen to be read.

For all read operations, ParaLang generates the equivalent Java code for handling console input. The `import java.util.Scanner` library is imported at the very top of the Java output. The `Scanner scanner = new Scanner(System.in)` object is initialized at the beginning of main, and uses `System.in` to obtain inputs from the console. Finally, at the end of main, the scanner object is closed to avoid resource leaks.

In the code generator, a switch case is used to determine which of the four valid data types is in `TYPE`. Instead of having a `readInt`, `readFloat`, etc., ParaLang uses just one `read` operation, with the parenthesized type determining which of the corresponding Java Scanner methods to use. In [Code 24] is the output from using the `read` operation as input, with the types: `int`, `float`, `bool`, and `string`.

```
1 scanner.nextInt();
2 scanner.nextFloat();
3 scanner.nextBoolean();
4 scanner.nextLine();
```

Code 24: Example of Java output when using read operations in ParaLang.

ParaLang keeps the reading operation simple by using a single `read` operation instead of different functions for each data type. The parenthesized type decides which scanner method to use, keeping ParaLang small and simple, while supporting different input types.

5.8 Array Declaration and Usage

Grammar **NEW CHANGE HERE!**  EVERYTHING HAS BEEN REVISED

The grammar of the array has been split up into two different working parts, `arrayIndex` and `arrayLiteral`, which together support each other completing the entirety of the grammar for the array. `arrayIndex` is used in the grammar to affiliate a specific index of an array with a variable of said array's type. The number of dimensions is illustrated with an open and closing square bracket per dimension at the end of the type. Between the open and closing square brackets is an integer that must be within the specific size of each dimension. This size is allotted with either an integer literal or an identifier, as described in [Section 5.1.3].

The other grammar rule, `arrayLiteral`, is used on the right side of an equation when an array has to be assigned or reassigned. The grammar rule has two options that can be chosen. The first option uses curly brackets, which are used to give the array's indexes specific values and are used both when assigning and reassigning an array. The other

option uses square brackets and can, like the first option, also be used when assigning or reassigning an array. The only way for an array to be reassigned is either if the array's dimensions were not assigned a size when the array was declared or if the array is set equal to another array with both the same dimensions and the same sizes for each dimension. Both of these grammar rules are placed under the factor grammar rule that is tied all the way to the top of the grammar rules.

ArrayValidator

The class named `ArrayValidator` is implemented with two almost identical functions to check if everything within the array is valid. The functions are both called `validate`, and which of the functions is called is determined by the parameters that are used to call the function. The first function only takes a string input to determine if the input is correctly setup and hereby making it a valid array. The other function also uses a string input along with a control input that both has the number of dimensions and the size of each dimension.

Both functions start with a setup of two integer arrays that are going to store the size of each dimension; therefore, the lengths of these arrays are set to the number of dimensions. The first function does not have a control input to determine the number of dimensions, and because of that, it is determined by counting the number of open curly brackets from the start to the first closing curly bracket, as illustrated on line 18 in [Code 25]. The `arrLiteral` array is used to uphold each dimension's rightful size based on the first input, and the `dimensionSize` array is used to store the current dimension's size, which is compared with `arrLiteral`. If the string does not contain any closing curly brackets, it will throw a runtime exception as seen on line 16 in [Code 25]. The last part of the setup involves initializing an integer called `currentDepth`, which is used to keep track of which dimension is being handled. The code is illustrated below:

```

14  int firstCloseBracket = input.indexOf("}");
15  if (firstCloseBracket == -1) {
16      throw new RuntimeException("You have an uneven amount of curly brackets in
    ↪   context: " + input);
17  }
18  int dimensions = (int) input.substring(0, firstCloseBracket).chars().filter(ch ->
    ↪   ch == '{').count();
19
20  int[] arrLiteral = new int[dimensions];
21  int[] dimensionSize = new int[dimensions];
22
23  Arrays.fill(arrLiteral, -1);
24  Arrays.fill(dimensionSize, 1);

```

Code 25: Initialization of the function `validate`'s setup from line 14 to 24 - `ArrayValidator.java`

The main part of the code from the first function is a for loop that goes through the input one character at a time, and depending on the character, it determines what to do. It will first check if the character is an open curly bracket, and if so, it will add one to both `depth` and `currentDepth`, as can be seen on line 29 in [Code 26]. If the character is a comma, it will add one to one of the indexes of `dimensionSize`; this is illustrated on line 30 in [Code 26]. Which index depends on the value of `currentDepth`, and it will always need to subtract one from that result because if the depth is 0, it would mean that it is either in the beginning or at the end of the string input. The last character being checked is the closing curly bracket, and if this is the case, it will check if the latest dimension that was exited has been registered within `arrLiteral`. If the index is equal to minus one, then it has not been registered, and it will register the `currentDepth` value subtracted by one index of `dimensionSize` to be the size of said dimension, which can also be seen on lines 33 to 34 in [Code 26]. Otherwise, if the index of `arrLiteral` has been registered, it will check if it is the same as the index of `dimensionSize`. If not, a runtime exception will be thrown; this is illustrated on lines 35 to 36 in [Code 26]. It will continue if the exception is not thrown by resetting the `dimensionSize` index to one and subtracting one from the `currentDepth` this is seen on lines 38 and 39 in [Code 26]. All of this can be seen in the code below.

```

27 for (int i = 0; i < input.length(); i++) {
28     char c = input.charAt(i);
29     if (c == '{') {
30         currentDepth++;
31     } else if (c == ',') dimensionSize[currentDepth - 1]++;
32     else if (c == '}') {
33         if (arrLiteral[currentDepth - 1] == -1) {
34             arrLiteral[currentDepth - 1] = dimensionSize[currentDepth - 1];
35         } else if (arrLiteral[currentDepth - 1] != dimensionSize[currentDepth - 1])
36             → {
37             throw new RuntimeException("Uneven array construction." + input);
38         }
39         dimensionSize[currentDepth - 1] = 1;
40         currentDepth--;
41     }
42 }

```

Code 26: Main part of the validate function from line 27 to 41 - ArrayValidator.java

The last check will determine if the depth has reached zero, return the `arrLiteral` to the place where the function was called, and if not, a runtime exception will be thrown. The main thing that is different in the code between the two functions is that it determines whether the `dimensionSize` index is the same as the dimension size in the control.

Visitors

In order to gain a better understanding of how arrays are managed through the pipelines [Section 5.3], a grammar rule for each visitor will be chosen to help provide insight into that specific grammar rule. In each visitor, the implementation is not going to be the same for each grammar rule; therefore, using different grammar rules is meant to provide a general idea of what is happening within each visitor. The first visitor that ParaLang is going to use is `AssDecVisitor` or phase 1; "Symbol assignments and declarations", depending on which part of the grammar rule is being used, it will choose a path. This path will have filters throughout each visitor in order to check that everything is correct.

For this visitor, the grammar rule for assignment is used in order to get the gist of how arrays are treated in this visitor. Before the arrays are being handled, the visitor makes sure that the variable's prefix does not contain `const` and `shared` together, because it should not be possible to have both of these prefixes together, see [Section 5.3.1]. After the context is confirmed to be the assigning of a variable, it will count for the amount of open square brackets there exists on the right-hand side of the equal sign.

If the number of open square brackets is above zero, that means the variable is an array with that number of dimensions. A one dimensional string array named `dimensions` is created with the number of dimensions as its size, and each index is filled with null as its value, as shown on line 254 in [Code 27]. It will then, through a for loop, check if there is an open square bracket within `beforeEquals`.

If there is an open square bracket, it will get what is within the square brackets and set that to be the size of the current index of `dimensions`, as illustrated on lines 257 to 261 in [Code 27], if the content from within the square brackets is not empty. After this, it will set `beforeEquals` to whatever comes after the first closing square bracket and go through the next iteration, see line 263 in [Code 27]. It will then check in another if statement whether the value of the current index of `dimensions` is null, and if true, it will set the index to "0". Lastly, after the for loop, it will set the symbol to a new `VariableSymbol`. This is all illustrated by the code below:

```

252 if (dims > 0) {
253     String[] dimensions = new String[dims];
254     Arrays.fill(dimensions, null);
255     for (int i = 0; i < dims; i++) {
256         if (beforeEquals.contains("(")) {
257             int sqBracket1 = beforeEquals.indexOf("(");
258             int sqBracket2 = beforeEquals.indexOf(")");
259             String insideBrackets = beforeEquals.substring(sqBracket1 + 1,
260                 ↪ sqBracket2).trim();
261             if (!insideBrackets.isEmpty()) {
262                 dimensions[i] = insideBrackets;
263             }
264             beforeEquals = beforeEquals.substring(sqBracket2 + 1);
265         }
266         if (dimensions[i] == null) {
267             dimensions[i] = "0";
268         }
269     }
270     symbol = new VariableSymbol(id, TypeSymbol.fromString(typeStr), dimensions);
271 }

```

Code 27: Assign the size of the array with the provided context lines 252 to 269 - AssDecVisitor.java

After the for loop, it will add the prefixes that was within the context to the symbol and then try to define the symbol, by adding it to the `CompilationContext`'s `SymbolTable`. If it was a success and the variable was not a duplicate of a known variable it will add the symbol to the `SharedVariables` list if one of the prefixes was `shared`.

RefLinkingVisitor

For the next visitor, `RefLinkingVisitor`, a different grammar rule is chosen to get the gist of how arrays are treated in this visitor, see [Section 5.3.2] for a general explanation of the visitor. The rule used is reassignment, and it is used in three different scenarios: the first is when a specific index is changed, the second scenario is when the array is set to a specific array using curly brackets, and the last scenario is when the size of the array is going to be changed using square brackets.

Before the scenarios a few setup variable is declared and the symbol is checked, whether it is of `arrType` or not. If it is then the context is split up into what is before and after the equal sign, in order to keep track of what scenario is sought after.

For the first scenario, which was to set a specific index to a value, an if statement would make sure that the context before the equal sign would contain an open square bracket, as well as make sure that the context after the equal sign does not contain either a curly

or square bracket. Then just as it was implemented in [AssDecVisitor](#) on lines 256 to 259 in [Code 27], the value within the set of square brackets would be extracted. A check would be implemented to see if this value is within the size of dimension. It would then set up for the next iteration, like as illustrated on line 263 in [Code 27].

Within the second scenario, which was to set an array to a specific array, it would check for open curly brackets in the context after the equal sign. It would then use the [validate](#) function from [ArrayValidator](#) and if the array is valid, it would set the array's indexes to the corresponding indexes of the result of [validate](#).

Now for the last scenario, which was to set the size of an array, it would first check for an open square bracket within the given context. This scenario would only be possible if the array has only been declared or the array is set to another array with the same dimensions, as described in [Section 5.8]. Much like the first scenario, the value within the set of square brackets would be extracted and checked if it is above zero. Then the setup for the next iteration would be implemented and after the for loop the new array would be cloned to prevent pass by reference and to use copy by value instead.

TypeCheckingVisitor

For the next visitor, [TypeCheckingVisitor](#), the same grammar rule from the visitor above is used get the gist of how arrays are treated in this visitor, see [Section 5.3.3] for a general explanation of the visitor. A few variable is declared in order to get the symbol corresponding to the context's id. If the symbol is of [arrType](#) and therefore an array, it checks there is at least one dimension by getting the context on the right-hand side of the equation and counting the amount of open curly bracket there is. If there is at least one dimension it will use the [validate](#) function from [ArrayValidator](#), to make sure that the array is valid. This is also done in this visitor because in [RefLinkingVisitor](#), it would use the function if an array was set to another array.

Other checks would also have to be done. This entailing; check if both sides of the equal sign is an array, check that the type for the array is the same as the value type, check if both array's has the same size for each dimension and lastly, make an exception when assigning a specific index of an array to a value of any type. It would need to override a runtime exception, because when getting a specific index, it would always get type int, but the value type and the array type could and should be able to be of any type.

CodeGenVisitor

For the last visitor, [CodeGenVisitor](#), the grammar rule that was used for the first visitor is used again in order to get the gist of how arrays are treated in this visitor, see [Section 5.3.6] for a general explanation of the visitor. A few variables are declared before the main part that is essential for arrays and after confirming that the context is assigning a

variable, a check that used to see if the depth of the `SymbolTable` is zero and thereby also a global variable would be in order.

If the depth is zero, then the first prefix would have to be `static` in order for it to be used in other functions. The next part is checking if the symbol is an array, and if this is the case, it will make a string specific for arrays named `arrayPrefix`, as illustrated on lines 139 and 140 in [Code 28]. The prefix from earlier is then added to `arrayPrefix` along with the name of the symbol type. The next part is a for loop that for each iteration adds a set of square brackets to the prefix, as shown on lines 141 to 143 in [Code 28]. Then after the for loop is done it will return the code in such a manner that it would be able to run in java. This is all illustrated by the code snippet below:

```

139 if (symbol.arrType != null) {
140     String arrayPrefix = prefix + " " + javaType(symbol.type.name.toLowerCase());
141     for (String i : symbol.arrType.dimSize) {
142         arrayPrefix += "[]";
143     }
144     return indent() + prefix + arrayPrefix + " " + id + " = " + "new " +
        ↪ arrayPrefix + exprCode + ";\n";
145 }

```

Code 28: Generate the java code for arrays lines 139 to 145 - CodeGenVisitor.java

5.9 Casting Construction

NEW CHANGE HERE!

This section describes the implementation of casting in ParaLang, which exclusively supports explicit casting, fulfilling [Requirement 14]. ParaLang has no support for implicit casting. The full semantic rules are covered in [Section ??], and the grammar is defined in [Section 5.1.3].

Casting allows for explicit conversion between specific data types in ParaLang. In [Figure 5.2], the legal conversions are listed. The left side of the arrow represents the source type, while the right side of the arrow denotes which types it may be cast into. The various conversion type rules are handled in the type checker.

The type checker is responsible for verifying that the data types of a cast are valid. First, it checks all of the valid data types that can be cast, such that it excludes `void` and `thread`. After the initial check, a switch case is utilized to map what each target type can be converted to. For example, when the target type is a `float`, the expression being cast must be either an `int` or a `bool`. In case of any illegal type conversions, the type checker

throws a semantic error, which specifies the illegal conversion attempt.

When ParaLang casts between the datatypes: `int` and `float`, it translates to Java's way of explicit casting, for example: `myInt = (int) myFloat;`. For the remaining conversions, which are not castable in Java, the translation relies on Java's built-in methods or ternary expressions, as detailed below.

<code>int</code>	→	<code>{string bool float}</code>
<code>float</code>	→	<code>{string bool int}</code>
<code>bool</code>	→	<code>{string float int}</code>
<code>char</code>	→	<code>{string}</code>

Figure 5.2: Illustration of ParaLang's legal conversions.

For converting any of the legal data types into a `string`, the `String.valueOf();` method is used, which obtains a `string` representation of the value used, as seen in [Code 29] line 5.

For converting either an `int` or a `float` into a `bool`, logic was added, such that everything other than the values of "0" and "0.0f" will result in changing it to "true" to match Java's Boolean value. The resulting Java code uses a ternary operator to determine if the result is true or false, shown in [29] line 3-4 for an `int` and a `float`.

Lastly, it is possible to cast a `bool` into an `int` or a `float`. This is done by using another ternary operator, as seen in [Code 29]. If the `bool` value is true, it will return either `1f` or `1.0f`, depending on whether it is cast to an `int` or a `float`. If the boolean is false, it turns to `0f` or `0.0f`.

```

1      int i = varBool ? 1 : 0;
2      float f = varBool ? 1.0f : 0.0f;
3      Boolean bi = (varInt == 0) ? false : true;
4      Boolean bf = (varFloat == 0.0f) ? false : true;
5      String s = String.valueOf(varFloat);

```

Code 29: Example of Java output when using casting in ParaLang.

In ParaLang, a `string` cannot be cast to any other data type because it is an arbitrary series of characters, which is not guaranteed to represent a numeric or boolean value.

Overall, ParaLang's casting system utilizes a clear syntax, strict type checking, and well-defined translation rules to ensure that the conversion is legal, both in ParaLang and the output in Java.

5.10 Scopes and Indentation

NEW CHANGE HERE!



When developing a new programming language, issues related to scopes and indentation often arise. Without implemented scope handling, a programmer could declare the same variable twice in different scopes. The compiler would then detect the variable `ID` already present in the `SymbolTable` and throw a duplicate variable error. This is why scope handling is a critical aspect of language design. It establishes clear rules that allow programmers to write code effectively and avoid unintended conflicts.

5.10.1 Scope Handling

Scope management is primarily handled within the `SymbolTable`, which maintains a tree of `Scope` objects. These objects form a parent-child chain that supports properly nested blocks, functions, and loops.

As the Assignment Declaration Visitor (`AssDecVisitor.java`) traverses the parse tree, it detects language constructs that introduce new scopes (functions, for loops, while loops, blocks, etc.). When entering such a construct, the visitor pushes a new scope onto the chain and pops it upon exiting the block. This approach ensures clean, hierarchical nesting of scopes.

When the `AssDecVisitor` finds a variable declaration, it calls the `define(Symbol s)` function (delegated through `SymbolTable` to `Scope`). Here is the implementation [Code 30]:

```

23 public boolean define(Symbol s) {
24     if (symbols.containsKey(s.ID)) return false;
25     symbols.put(s.ID, s);
26     return true;
27 }
```

Code 30: `define(Symbol s)` method in `scope.java`.

The method checks whether the symbol already exists in the current scope only. If present, it returns `false` leading to a duplicate declaration error. Otherwise, it adds the symbol and returns `true`.

The way the program manages the scope stack and maps parse tree nodes to scopes is handled in the `SymbolTable.java` class. It has several methods that help with this:

- `pushScope(ParseTree node)` creates a new scope with the current one as parent and stores the mapping.
- `popScope()` closes the current scope.
- `restoreScope(ParseTree node)` is used by later visitors to restore the correct scope associated with a node.

An example of scope handling in `AssDecVisitor.java` [Code 31]:

```
139 @Override
140 public Void visitForStatement(OurGrammarParser.ForStatementContext context) {
141     this.ctx.symbolTable.pushScope(context);
142     visitChildren(context);
143     this.ctx.symbolTable.popScope();
144     return null;
145 }
```

Code 31: `visitforvar` method in `AssDecVisitor.java`.

Similar patterns (push -> visitChildren -> pop) appear for blocks, functions, while loops, and other constructs. Scopes saved via node mappings allow other visitors (`RefLinkingVisitor`, `TypeCheckingVisitor`, etc.) to reliably restore scopes without recreating them.

5.10.2 Variable Shadowing

Variable shadowing occurs when an existing variable is declared again in a deeper nested scope. In such cases, references to that variable name within the inner scope resolve to the innermost declaration. The outer variable remains accessible only after exiting the inner scope.

The mechanism relies on the recursive `resolve` method in `Scope.java`, which first checks the current scope and then walks up the parent chain until it finds a match or reaches the global scope [Code 32].

```

33 public Symbol resolve(String ID) {
34     if (symbols.containsKey(ID)) {
35         return symbols.get(ID);
36     } else if (parent != null) {
37         return parent.resolve(ID);
38     } else {
39         return null;
40     }
41 }

```

Code 32: `resolve(String s)` method in `scope.java`.

5.10.3 Indentation

When it comes to indentation it is actually not important for either the requirements nor for the generated Java code to compile. In fact, the parser ignores whitespace entirely and only looks at the tokens. However, there are advantages in having correct indentation in the output Java code. It makes it way more readable to the human eye and makes debugging easier. This is how indentation is handled in the Code Generation Visitor (`CodeGenVisitor.java`):

When the Code Generation Visitor calls the `restoreScope(ParseTree node)` it increments a depth tracker [Code 33]:

```

46 public void restoreScope(ParseTree node) {
47     Scope stored = nodeScopes.get(node);
48     if (stored != null) {
49         currentScope = stored;
50         depth++;
51     }
52 }

```

Code 33: `restoreScope(ParseTree node)` method in `SymbolTable.java`.

The `indent()` method [Code 34] uses this depth tracker each time the code generator needs to write a new line of code into the output string.

```

46 private String indent() {
47     int depth = Math.max(0, ctx.symbolTable.getDepth());
48     return INDENT.repeat(Math.max(0, depth + 1));
49 }

```

Code 34: `indent()` method in `CodeGenVisitor.java`.

This method [Code 34] first gets the depth, while making sure to not go into the negatives by picking the largest number between zero and `ctx.symbolTable.getDepth()`. It then returns the amount of indentation that corresponds with the current depth level. This method ensures clean and readable Java output, that the programmer does not have to think about.

5.11 Implementation and demo of the Solution

NEW CHANGE HERE!

This section demonstrates how to use the transpiler program. The main method for the project is in `ParserDriver.java` and calls the entire pipeline from start to finish, as well as saving the generated Java code to a file.

When running the main method, the program reads the source code written in ParaLang. Then it feeds that string into the lexer, after which the parser generates the `parseTree`. Next, each semantics visitor (phase 1 - 5) traverses the `parseTree` and populates the `CompilationContext`. Once all the semantics visitors have visited the `parseTree`, the Code Generator Visitor generates equivalent code and saves the generated code in the `javaOutput.txt` file. When the program has run, the emitted code can then compile in a Java compiler. Let us take a look at an example that can take an input from the user and calculate the factorial of that number in parallel:

5.11.1 Factorial example in ParaLang

```
1 shared int counter = 1;
2 const int LIMIT = 16;
3 int[4] results;
4 int input;
5 int[] arr;
6 bool ready = false;
```

...

```

21 void main() {
22     print("Enter a number:");
23     input = read(int);
24     arr = [input];
25     int index = 0;
26
27     if (input > LIMIT) {
28         print("Input too large, clamping to the LIMIT:", LIMIT, "\n");
29         input = LIMIT;
30         ready = true;
31     } else {
32         ready = true;
33         print("The input was accepted as is.\n")
34     }

```

Code 35: `main` method in `sample-input.txt` [Code ??].

This is only the first part of the `main` function. Here, the user is prompted to enter a number. It is specified on line 23 in [Code 35] that the user input must be of type `int`. This line corresponds to the Java code `scanner.nextInt()` that throws a type mismatch error if anything other than a positive int is submitted. The array is then resized to match the length specified by the user input. If the input exceeds 12, which is the maximum value before the result surpasses the largest number that can be stored in an int, the program displays the message on line 28 and sets the input to the `LIMIT` value of 12. If the input does not surpass 12, it will simply proceed to the next part of the program, as seen below [Code 36]:


```

35     if (ready) {
36         for(int i = 0; i < input; i = i + 1) {
37             arr[i] = i + 1;
38         }
39         while(true) {
40             index = index + 1;
41             if (index == 67) {
42                 print("SIX SEVEN!!! SIX SEVEN!!! SIX SEVEN!!! SIX SEVEN!!!\n");
43                 continue;
44             }
45             thread t1 => {
46                 critical(counter) {
47                     counter = worker(0, input / 4);
48                 }
49             }
50             thread t2 => {
51                 // the rest of the code for thread creation
52
53                 ...
54
55                 awaitAll(t1, t2, t3, t4);
56                 print("The result of ", index, " is: ", counter, "\n");
57                 counter = 1;
58                 if (index >= 100) break;
59             }
60         }
61     }
62 }

```

Code 36: Array population and thread creation in [sample-input.txt](#) [Code ??].

Once the input has been validated, it sets the `ready` flag as true to allow the rest of the code to run. On line 36 - 38, the array gets populated with numbers from 1 to the user input. As seen on line 39, 40, and 69 in [Code 36], the program runs the threads a hundred times to show that the program gets the right answer all one hundred times without any race conditions or deadlocks. To show the use of `continue`, when the 67th iteration of the code is reached, the program prints "Six Seven!" and continues to the next iteration. Inside the while loop, the array is split into four pieces and fed to the `worker` function inside the four threads `t1`, `t2`, `t3` and `t4`. The worker function is called inside a critical section, meaning before acquiring the lock for `counter`, it has to be unlocked. The worker function can be seen below [Code 37]:

```

9  int worker(int start, int end) {
10     int i = start;
11     int result = counter;
12
13     while (i < end) {
14         result = result * arr[i];
15         i = i + 1;
16     }
17
18     return result;
19 }

```

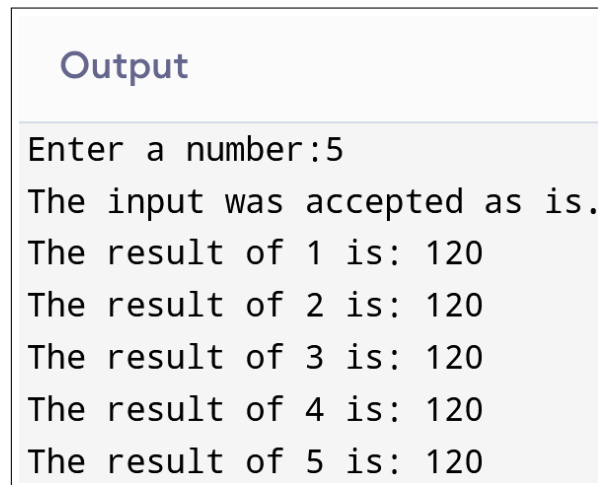
Code 37: Worker function in `sample-input.txt` [Code ??].

The worker function has the parameters `start` and `end` of type `int` [Code 37]. It uses these as indexers for the array to know where to start and end its calculations. Factorial is calculated by multiplying all elements of the array by each other. This can be done in any order, so multithreading is a good tool for this, as it can do more calculations simultaneously [Section 2.3.2]. A simple while loop inside the function takes the result and multiplies it by the next value of the array. Once the while loop is done, it returns the result.

As seen on line 66 in [Code 36], the program waits for all threads to return completed, then continues the program to the print statement that prints the result. After the result has been printed, it resets the counter to 1 and runs another iteration until the 100th iteration.

5.11.2 Running the Java Code

The above code example [Code 37] makes use of all ParaLang's unique features, making it a great example of how the code should work in Java. Once the program has translated the code, the line length goes from 72 lines in ParaLang to 130 lines in Java. This shows how simple and readable ParaLang is and how the programmer does not have to know a lot of advanced syntax to code with mutexes and threads. The full output Java code can be seen here: [Appendix ??]. The finished Java code will be run inside an online Java compiler to show how it is able to run without any prerequisites or extensions. Just some standard Java libraries are needed to compile and run. The online Java compiler in question is "<https://www.programiz.com/java-programming/online-compiler/>," and on that page, there is a code input section where the output code from the translator will be placed and run.



```
Output
Enter a number:5
The input was accepted as is.
The result of 1 is: 120
The result of 2 is: 120
The result of 3 is: 120
The result of 4 is: 120
The result of 5 is: 120
```

Figure 5.3: Console output of the Factorial implementation example.

The user has typed 5 into the console when prompted, and the program accepted the value as it is smaller than the limit of 16. The program then split up the array of five cells and calculated the factorial in parallel, and got the result 120. As seen in the [Figure 5.3], the program got the right answer all five times, but also all 100 times, if the screenshot was longer.

Chapter 6

Testing

When developing systems, each component must be thoroughly tested, both by itself and as part of a larger system. Using a combination of unit testing, integration testing, and acceptance testing increases the confidence that individual functions work as intended, that subsystems interact with each other as intended, and that the final product satisfies the requirements.

Testing the system continuously during development helps identify errors and bugs earlier in the development process. Detecting these errors and bugs sooner rather than later can reduce the potential for cascading changes that require other parts of the system to be modified to accommodate them.

On one hand, unit testing and integration testing are performed using the JUnit testing framework for Java. JUnit makes it easier to run tests automatically and repeatably throughout the development process, ensuring that new changes do not break already existing functionalities.

On the other hand, the acceptance test is based on the requirements that were defined for the system, ensuring that the final product fulfills the expectations set for it.

6.1 Unit Testing

Unit testing is for testing individual functions or modules in a program. These are the smallest modules of a program. When unit testing, one should give the function valid inputs as well as edge case inputs and faulty inputs to check if all cases are handled as expected. The reason for using unit testing is to be able to identify where in the code

something breaks when changing or developing the program. It saves time debugging and makes a more robust program.

Due to time constraints, the use of stubs and drivers to isolate the unit tests was not implemented. As a result, the unit tests are not fully isolated, but instead a part of a larger pipeline, including the lexer, parser, and multiple visitors. While it is not ideal, as it is difficult to exactly pinpoint where certain errors or bugs occur, it still provides valuable verification of components.

The unit tests are performed using the Arrange-Act-Assert (AAA) pattern. Each test case begins with arranging the preconditions, such as initializing objects. After arranging the preconditions, the code that actually needs testing is executed. Lastly, the results must be verified in the assertion phase, where any errors are caught.

Testing every single function in a compiler is impractical and time-consuming. Instead, the unit testing strategy for this project focuses on testing the full compiler pipeline. This includes the lexer, parser, and all visitor phases. Each test case includes multiple input programs, some that are expected to compile successfully, and others that intentionally contain errors to verify that the compiler rejects them correctly. These inputs are arranged right above the specific method being tested using JUnit's `ParameterizedTest` and `CsvSource`. This allows the inputs to the unit tests to be written in a CSV format, and then `ParameterizedTest` goes through every input in the CSV file. If time had permitted it, these inputs would have been moved to separate files in order to remove clutter and make it easier to identify the various tests.

To verify that the output of a test meets the expected results, assertions are used. Depending on the specific test case and what exactly needs to be validated, different assertion methods are used, such as `assertTrue` to check boolean values or `assertNotNull` to ensure values are not `null`.

To illustrate how the individual components were tested, the lexer unit test is shown in [Code 38].

```

61 @ParameterizedTest(name = "Testing lexer tokenization: {0}")
62 @CsvSource({
63     "int i = 5;', 5",
64     "float radius = 3.14;', 5",
65     "if (x > 5) {}", 8"
66 })
67 void testLexer(String input, int expectedTokenCount) {
68     System.out.println("==== Running testLexer for: " + input + " =====");
69     CharStream charStream = CharStreams.fromString(input);
70     OurGrammarLexer lexer = new OurGrammarLexer(charStream);
71     CommonTokenStream tokens = new CommonTokenStream(lexer);
72
73     tokens.fill();
74     int actualCount = tokens.getTokens().size() - 1;
75
76     try {
77         assertAll("Lexer Token Count Check",
78             () -> assertEquals(expectedTokenCount, actualCount, "Token count should
79                 ↳ match expected length."
80         );
81         System.out.println("testLexer = success! Found exactly " +
82             ↳ expectedTokenCount + " tokens.");
83     } catch (AssertionError e) {
84         System.out.println("testLexer = failure! Error: " + e.getMessage());
85         throw e;
86     }
87 }

```

Code 38: Lexer token unit test extracted from `UnitTest.java`.

Below are listed some of the other core unit tests that have been done.

- **Lexer** – tested to ensure that the correct number of tokens are produced for a given input.
- **Parser** – tested to verify that programs with syntactically valid input produce the correct parse tree, and that incorrect input is caught.
- **AssDecVisitor** – tested to ensure correct handling of assignments and declarations.
- **RefLinkingVisitor** – tested to guarantee that identifiers are resolved to the correct declaration or assignment.
- **TypeCheckingVisitor** – tested to verify that types follow ParaLang’s type rules.
- **FtableVisitor** – tested to ensure correct construction of functions.

- **CodeGenVisitor** – tested to verify that valid programs generate the correct Java code output.

Overall, the unit tests provided valuable information, even though the specific test cases were not fully isolated due to not implementing stubs or drivers. This approach still verified both successful compilations and error handling across the pipeline.

6.2 Integration Testing

Integration testing is the phase that follows unit testing. While unit testing verifies that individual functions or modules work correctly in isolation, integration testing examines how these modules interact with each other when combined.

The main goal of integration testing is to detect defects in the interfaces and interactions between components. Even if two modules pass their unit tests perfectly, problems frequently arise when they communicate, such as incorrect data formats, wrong parameters, timing issues, mismatched return values, or unexpected side effects. Identifying these issues early makes debugging significantly easier compared to discovering them later in the development process [44].

6.2.1 Integration Testing Strategies

There are two primary strategies for integration testing: Bottom-Up and Top-Down integration.

Bottom-Up Integration begins with the lowest-level modules (leaf modules), then integrates higher and higher-level modules. This strategy often uses drivers to simulate calls from higher-level modules that have not yet been integrated. It is generally easier to debug because errors can be traced from the bottom up. Once clusters of low-level modules are verified, they are combined with mid-level and eventually high-level modules [44].

Top-Down Integration starts with the highest-level modules (main control modules) and works downward. Lower-level modules that are not yet ready are replaced with stubs. This approach allows early validation of the overall system design and control flow. However, it may delay the testing of critical low-level functionality until the stubs are replaced with real modules [44].

Many projects use a hybrid (or Sandwich) approach that combines elements of both

Top-Down and Bottom-Up strategies to balance their respective strengths and weaknesses. In this project, in the interest of time, the group chose to focus on the Bottom-Up approach to integration testing. A demonstration of how the project uses the Bottom-Up approach will be presented below [44].

6.2.2 Bottom-Up integration testing demonstration

The way the group chose to handle Bottom-Up integration testing was fairly easy to implement, as the program is a very linear pipeline of things happening in order. First, the lexer, then the parser, then the semantic visitors, and so on until the program prints the finished Java code, which is fully checked and tested.

The first step in all of the integration tests is to create a series of test case files with code examples written in ParaLang, then feed the code examples to different test cases that have been set up. An example of a test input file can be seen below [Code 39].

```
1 int x = 10;  
2 float y = cast(float) x;  
3 print(y, " ", x);
```

Code 39: Contents of `test6.txt` written in ParaLang.

Lexer Parser Integration Test

The first integration test case tests the interaction between the lexer and the parser. The above code snippet [Code 39] can be seen using the custom explicit casting syntax of ParaLang. The lexer should tokenize the string into tokens using the lexer rules from the projects context free grammar [Appendix 9.1], which the parser should recognize as valid syntaxes and generate a parse tree. If a missing parenthesis or semicolon is detected, the parser should report an error, failing the integration test. Also, if the lexer generates a faulty token stream, the parser should throw an error.


```

126 public static String runLexerParserPipeline(String input) {
127     CharStream charStream = CharStreams.fromString(input);
128     OurGrammarLexer lexer = new OurGrammarLexer(charStream);
129     CommonTokenStream tokens = new CommonTokenStream(lexer);
130     OurGrammarParser parser = new OurGrammarParser(tokens);
131
132     parser.removeErrorListeners();
133     parser.addErrorListener(new org.antlr.v4.runtime.BaseErrorListener() {
134         @Override
135         public void syntaxError(org.antlr.v4.runtime.Recognizer<?, ?> recognizer,
136             ↪ Object offendingSymbol, int line, int charPositionInLine, String msg,
137             ↪ org.antlr.v4.runtime.RecognitionException e) {
138             throw new RuntimeException("Syntax error at line " + line + ":" +
139                 ↪ charPositionInLine + " - " + msg);
140         }
141     });
142
143     ParseTree tree = parser.program();
144     return tree.toStringTree(parser);
145 }

```

Code 40: Lexer + Parser pipeline integration test from ParserDriver.java

Above, a code snippet of the lexer + parser integration pipeline can be seen [Code 40]. Here the input is tokenized by the lexer, then parsed into a convenient token stream by ANTLR. Finally, the parser gets the tokens, checks the amount and order of the tokens and generates the parse tree if no errors occur. If no errors occur the integration test will check the expected results to see if they match. The expected results have been manually checked carefully to see if they are correct. If a match is made, the lexer + parser integration test will succeed.

To get an overview of which test cases failed and succeeded, a series of Bottom-Up integration tests has been created with various test inputs, some that are designed to fail, others using the "happy path" also known as valid input. The tests have been designed to cover all areas of ParaLang, aiming to cover as much as possible in these tests. Each test case covered a small amount of the grammar, making sure to cover all aspects like a checklist. Below is an overview of the tests that have been conducted and the results.

```

Running all integration tests via parameterized methods...
=== Testing Lexer + Parser for: test1.txt ===
test1.txt - Lexer + Parser pipeline SUCCESS - matched expected output
=== Testing Lexer + Parser for: test2.txt ===
test2.txt - Lexer + Parser pipeline SUCCESS - matched expected output
=== Testing Lexer + Parser for: test3.txt ===
test3.txt - Lexer + Parser pipeline SUCCESS - matched expected output
=== Testing Lexer + Parser for: test4.txt ===
test4.txt - Lexer + Parser pipeline SUCCESS - matched expected output
=== Testing Lexer + Parser for: test5.txt ===
test5.txt - Lexer + Parser pipeline SUCCESS - matched expected output
=== Testing Lexer + Parser for: test6.txt ===
test6.txt - Lexer + Parser pipeline SUCCESS - matched expected output
=== Testing Lexer + Parser for: test7.txt ===
test7.txt - Lexer + Parser pipeline SUCCESS - matched expected output
=== Testing Lexer + Parser for: test8.txt ===
test8.txt - Lexer + Parser pipeline SUCCESS - matched expected output
=== Testing Lexer + Parser for: test9.txt ===
test9.txt - Lexer + Parser pipeline SUCCESS - matched expected output

```

Figure 6.1: Lexer + Parser test results from `IntegrationTest.java`.

Parser + Semantic Integration Test

Now, the next integration test in the Bottom-Up strategy is the Parser + Semantics integration test. Here, the parse tree must be correct so that the semantic visitors can do their job correctly. This is what will be tested using the same input ParaLang files that already cover all aspects of the language. This test, however, will generate a parse tree, then feed that to `AssDecVisitor.java`, `RefLinkingVisitor.java`, and `TypeCheckingVisitor.java`. These visitors will populate the `SymbolTable`, `ResolvedSymbols`, and `nodeTypes` inside the `CompilationContext` object. These visitors will check the semantics and only if no errors are found while populating and traversing the parse tree will they succeed. Once successfully populating the `CompilationContext`, the integration test will compare the `SymbolTable`, `ResolvedSymbols` and `nodeTypes` with the expected results. If they match, the test will succeed. If any errors in this whole pipeline occur, the test will fail and report the error in the console. Below is an overview of the tests and their results [Fig 6.2.2]:

```
Integration test setup complete. Output directory ready.\n
=== Testing Parser + Semantic Analysis for: test1.txt ===
test1.txt - Parser + Semantic pipeline SUCCESS - matched expected output
=== Testing Parser + Semantic Analysis for: test2.txt ===
test2.txt - Parser + Semantic pipeline SUCCESS - matched expected output
=== Testing Parser + Semantic Analysis for: test3.txt ===
test3.txt - Parser + Semantic pipeline SUCCESS - matched expected output
=== Testing Parser + Semantic Analysis for: test4.txt ===
test4.txt - Parser + Semantic pipeline SUCCESS - matched expected output
=== Testing Parser + Semantic Analysis for: test5.txt ===
test5.txt - Semantic error caught: Variable 'x' not declared.
=== Testing Parser + Semantic Analysis for: test6.txt ===
test6.txt - Parser + Semantic pipeline SUCCESS - matched expected output
=== Testing Parser + Semantic Analysis for: test7.txt ===
test7.txt - Parser + Semantic pipeline SUCCESS - matched expected output
=== Testing Parser + Semantic Analysis for: test8.txt ===
test8.txt - Parser + Semantic pipeline SUCCESS - matched expected output
=== Testing Parser + Semantic Analysis for: test9.txt ===
test9.txt - Parser + Semantic pipeline SUCCESS - matched expected output
```

Figure 6.2: Parser + Semantic test results from `IntegrationTest.java`.

These integration tests show that the modules work well with each other, catching the error in test case 5 in the semantics checker while succeeding all the other ones that were designed with valid input. Once all the tests ran they were all compared to the expected outputs and succeeded.

6.3 Acceptance Testing

Acceptance testing is where the success of the project is determined. The requirements can be compared to the final product, checking which features or capabilities have been implemented. If only all of the most important requirements have been met (Must have requirements), the product is a Minimal Viable Product. If more requirements have been implemented and successfully tested, the project can be described as a viable product.

6.3.1 Testable Requirements

Each requirement that was deemed to be testable and implemented was verified through one dedicated test case. A requirement is considered testable if its expected behavior can be expressed through an observable input and output.

In [Table 6.1] is the complete set of requirements identified in [Chapter 3]. For each requirement, its corresponding test case is listed, including the test status. Linking each requirement to a concrete test prevents the case of core untested functionality.

Requirements that are not testable or not implemented are noted as "N/A" in the Test Case column. For these requirements, the Status column provides a short description, indicating whether the requirement is not testable or not implemented.

Each of the test cases executes the entire pipeline to validate that the ParaLang input results in the expected Java output.

As shown in [Table 6.1], all testable must have requirements (1-2, 5-14) have been implemented and passed their corresponding test cases. As the must-have requirements have been verified as complete, the project qualifies as an MVP. In addition to these, two requirements were considered non-testable, because they cannot be verified through observable inputs and outputs. These requirements are [Requirement 3] and [Requirement 4], which concern the project being testing using unit, integration, and acceptance testing, and ParaLang uses formal and correct syntax.

As visible in [Table 6.1], the should-have requirements have also been completed, except for [Requirement 18] and [Requirement 19]. [Requirement 18] states that a thread should evaluate to a boolean value, but this was not implemented in ParaLang. [Requirement 19] states that ParaLang will have support for these modifiers: shared, const, and static. It was realized late in the project's timeline that using static as a prefix does not make sense in ParaLang, because it does not support classes. Instead, ParaLang applies the static keyword when code is transpiled to Java. Static is applied in the generated code for the main class, main method, every function, and all global variables, except arrays.

Requirement	Test Case	Status
Requirement 1	TC-01	Passed
Requirement 2	TC-02	Passed
Requirement 3	N/A	Not Testable
Requirement 4	N/A	Not Testable
Requirement 5	TC-03	Passed
Requirement 6	TC-04	Passed
Requirement 7	TC-05	Passed
Requirement 8	TC-06	Passed
Requirement 9	TC-07	Passed
Requirement 10	TC-08	Passed
Requirement 11	TC-09	Passed
Requirement 12	TC-10	Passed
Requirement 13	TC-11	Passed
Requirement 14	TC-12	Passed
Requirement 15	TC-13	Passed
Requirement 16	TC-14	Passed
Requirement 17	TC-15	Passed
Requirement 18	N/A	Not Implemented
Requirement 19	TC-16	Passed
Requirement 20	TC-17	Passed
Requirement 21	N/A	Not Implemented
Requirement 22	N/A	Not Implemented
Requirement 23	N/A	Not Implemented
Requirement 24	TC-18	Passed
Requirement 25	N/A	Not Implemented
Requirement 26	N/A	Not Implemented
Requirement 27	N/A	Not Implemented
Requirement 28	TC-19	Passed
Requirement 29	N/A	Not Implemented
Requirement 30	N/A	Not Testable/Implemented

Table 6.1: Acceptance Test Case Table.

Color Guide: ■ Must Have ■ Should Have
■ Could Have ■ Won't Have

As shown in [Table 6.1], [Requirement 24] and [Requirement 28], both marked as could have, have been completed. While implementing [Requirement 10] to prevent deadlocks, a hierarchical lock order was introduced, which fulfilled both requirements. At the beginning of the project, this deadlock prevention strategy was not yet planned, causing

[Requirement 10] and [Requirement 24] to overlap. [Requirement 28] was not strictly necessary, but it proved very useful for identifying if the Java output was in its expected format.

Across the 30 requirements, 19 have been passed, 2 are classified as not testable, and 9 remain not implemented, resulting in an overall 63% pass rate for the requirement set.

Each of the tests used the same format, starting with a single input. These test inputs vary in length, depending on the specific requirement. For example, [Requirement 7] outlined a series of expressions, each of which was included in the input.

After the input, the `ParserDriver.runFullPipeline` is called using that input. The `runFullPipeline` method goes through the entire transpiler from start to finish. Firstly, the input is turned into tokens using the lexer, after which it is parsed through the parser into an AST. `runFullPipeline` then runs through every visitor in the following order: `AssDecVisitor`, `RefLinkingVisitor`, `TypeCheckingVisitor`, `FtableGenVisitor`, `MutexVisitor`, and `CodeGenVisitor` as described in [Section 5.3]. This order is required because each visitor populates the AST with information that subsequent visitor phases depend on. After all phases have been executed, the final generated code is returned as a string.

To ensure whitespace does not change the expected format of the output, it is removed after executing `ParserDriver.runFullPipeline`, and before testing with assertions.

The final part of each test includes assertions to verify that the input code produces the expected output. Using the same example as [Requirement 7], there is a `assertTrue` for each included operator. `assertTrue` is primarily used for these, because the tests are supposed to test if valid input results in the expected output.

6.3.2 End-2-End Testing Demonstration

To run the End-2-End acceptance test the whole program pipeline will be called multiple times using different inputs. It effectively runs the `ParserDriver.java` over and over and saves any thrown errors and displays them in the end. This testing method is good to check if there are any errors, but it is not good to identify all errors. If the first visitor reports an error it will effectively stop that trial and not report any further errors. So only the first error will be reported. This is why running multiple test approaches is helpful to get a clearer idea of where an error might be.

```

415 try {
416     String actualOutput = ParserDriver.runFullPipeline(input);
417
418     Files.writeString(Paths.get(outputPath), actualOutput);
419
420     String expected = Files.readString(Paths.get(expectedPath))
421         .trim().replace("\r\n", "\n");
422     actualOutput = actualOutput.trim().replace("\r\n", "\n");
423
424     if (actualOutput.equals(expected)) {
425         System.out.println(testFileName + " - Full pipeline SUCCESS");
426     } else {
427         System.out.println(testFileName + " - Mismatch. Check " + outputPath);
428         System.out.println("Expected length: " + expected.length() + ", Actual: " +
429             ↪ actualOutput.length());
430         assertEquals(expected, actualOutput,
431             testFileName + " full pipeline output did not match expected");
432     }
433 } catch (RuntimeException e) {
434     System.out.println("Runtime error in full pipeline: " + e.getMessage());
435     Files.writeString(Paths.get(outputPath), "ERROR: " + e.toString());
436 } catch (Exception e) {
437     fail("Unexpected exception in full pipeline for " + testFileName + ": " +
438         ↪ e.getMessage());
439 }

```

Code 41: End-2-End test from [AcceptanceTest.java](#)

In this code snippet [Code 41], the integration test function gets the raw string of each test input in ParaLang, runs the lexer, parser and the visitors ending up with java code. If it records an error it will print it as shown below [Figure 6.3]:

```

=== Acceptance Test Setup Complete ===
=== Running Full Pipeline for: test1.txt ===
test1.txt - Full pipeline SUCCESS
=== Acceptance Test Setup Complete ===
=== Running Full Pipeline for: test2.txt ===
test2.txt - Full pipeline SUCCESS
=== Acceptance Test Setup Complete ===
=== Running Full Pipeline for: test3.txt ===
test3.txt - Full pipeline SUCCESS
=== Acceptance Test Setup Complete ===
=== Running Full Pipeline for: test4.txt ===
test4.txt - Full pipeline SUCCESS
=== Acceptance Test Setup Complete ===
=== Running Full Pipeline for: test5.txt ===
Runtime error in full pipeline: Variable 'x' not declared.
=== Acceptance Test Setup Complete ===
=== Running Full Pipeline for: test6.txt ===
test6.txt - Full pipeline SUCCESS
=== Acceptance Test Setup Complete ===
=== Running Full Pipeline for: test7.txt ===
test7.txt - Full pipeline SUCCESS
=== Acceptance Test Setup Complete ===
=== Running Full Pipeline for: test8.txt ===
test8.txt - Full pipeline SUCCESS
=== Acceptance Test Setup Complete ===
=== Running Full Pipeline for: test9.txt ===
test9.txt - Full pipeline SUCCESS

```

Figure 6.3: End-2-End test results from `AcceptanceTest.java`.

As seen in the test results above, the fifth test which is deliberately faulty input shows that the error has been found and what the error is. On line 114 the test compares the actual output of the test with the expected output file. Only if they match will trigger a success, and then there will be no errors in the pipeline for that particular test.

The acceptance testing confirms that the delivered transpiler satisfies all testable must have requirements and successfully converts valid ParaLang code into semantically equivalent Java code. Combined with the end-to-end testing results, the project is considered to be accepted.

Chapter 7

Discussion

Discussion intro...

7.1 Discussion: Design

7.1.1 Design Decisions

7.1.2 Syntax and Semantics

Big Step Limitations

Concurrency:

- Global State

7.2 Discussion: Implementation

7.2.1 Arrays

NEW CHANGE HERE! 

As of the current version of how the arrays are being handled, either an integer literal or

an identifier being used to define the size of a dimension, as mentioned in both [Section 5.1.3] and [Section 5.8]. There is, however, a huge flaw in this way of handling it. When there is an instance of using the identifier to set the size of each dimension, it stores the name of the identifier instead of the identifier's value.

At the beginning of the implementation, the `dimSize` parameter from `ArrayTypeSymbol` was an integer array storing each dimension's size. This was later changed to a string array to allow the identifiers to be used in setting the size of each dimension. This change will not ruin the translation of ParaLang to the java language; however, the logic behind `ArrayValidator` and other methods would crumble when having to compare an integer with a string.

Therefore, one of the problems that would occur is when a single index is changed, and it tries to check if the size of each dimension that is trying to be accessed is within the range for that specific array. It would fail to make a runtime exception, if the index is outside the array's range. Everything related to the integer literal value is the perfect way of setting the size of each dimension.

An implementation would be instead of setting the name of the identifier, then it would take the actual value instead. This is, however, not a feature supported by the current methods. One way to support a feature like this would be to implement a helper function that would keep track of the values of each variable, just like the current `CompilationContext`, where the scope and symbols is being tracked. This functionality could either be integrated within the current `CompilationContext` or be its own class.

If it was its own class it might be able to help fix another big problem there is with the arrays. The problem lies in the way each visitor is used. If two arrays were to be initialized with the same amount of dimensions, but the size of the dimensions is not the same, then it would pass

```
1  int[10] x;
2  int[5] y;
3  x[9] = 3;
4  x = y;
```

Code 42: Array example of visitor flaw.

the first visitor fine. The problem lies with the three other visitors that manage and influence the arrays. When `RefLinkingVisitor` is used it will think that the size of the first array `x` is 5, the same as the size of `y`, instead of 10. It will then try to get the 9th index, in order to set it to three, which is not possible, this illustrated in the example below [Code 42].

This would be an issue further down the pipeline, in visitors `TypeCheckingVisitor` and `CodeGenVisitor`, where it would fail by the same logic as in `RefLinkingVisitor`. The current solution to this problem is that only two arrays with the same amount of dimensions and identical sizes of each dimension are accepted by the `TypeCheckingVisitor` in `reassignment`.

One way to change this would be every time an array is detected in both [RefLinkingVisitor](#) and [TypeCheckingVisitor](#), it would ignore it. Then, in [CodeGenVisitor](#), for all code containing arrays, it would instead call methods from the separate array class. Here it would execute logic similar to [RefLinkingVisitor](#), [TypeCheckingVisitor](#) and [CodeGenVisitor](#) all at once. This would mean no leftover states from multiple passes can be interpreted at the wrong points in the code.

All of the implementations described above is not feasible to have within the scope of the project, due to the time constraints there have to be uphold.

7.2.2 Thread Lambda Expressions

NEW CHANGE HERE! 

One significant limitation of this project relates to how lambda expressions are used when generating Java code for threads and critical sections containing shared variables. The program translates ParaLang thread blocks into Java's [CompletableFuture.runAsync\(\(\) -> ... \)](#) lambdas, which has strict rules about variable access and modification. Namely the shared variable inside a critical section inside a thread block must be effectively final, meaning it cannot be reassigned after its initial declaration. Below, is an example of a shared variable being declared inside the `void main` and modified inside a critical section inside of a thread [Code 43].

```

1 void main() {
2     shared int counter = 1;
3     thread t1 => {
4         critical(counter) {
5             counter = counter + 5;
6         }
7     }
8 }

```

Code 43: A ParaLang code example showing a limitation with lambda expressions.

This structure [Code 43] fails to produce valid compilable Java output because the shared int `counter` is treated as a local variable within `void main()`. Java rejects the modification of `counter` since it is neither final nor effectively final. However, declaring the shared variable globally outside `void main()` allows the [CodeGenVisitor](#) to give it the `static` prefix, bypassing the lambda restriction and enabling proper locking and access via [ReentrantLock](#). Another workaround is to change the variable to a single cell array, which makes it call by reference. This is a worse workaround because then the logic for that variable will be changed everywhere else in the code, both inside and outside the

critical section. For future development a better solution will be presented below:

Changing the `CodeGenVisitor` to automatically wrap locally declared shared variables in an `AtomicReference<T>` would solve this problem. Upon detecting a shared declaration inside the `void main()` that is later used in a critical section, the translator could rewrite it to something like `AtomicReference<Int> counter = new AtomicReference<>(1);` and replace assignments with atomic operations such as `counter.set(counter.get() + 5)`. This approach would allow the programmer to declared local shared variables inside the `void main()` while ensuring thread safety and compatibility with lambda expressions.

7.2.3 Mutex Visitor

NEW CHANGE HERE!



As described in [Section 5.3.5], the mutex visitor currently ensures that functions containing critical sections cannot be called from within a critical section. While this was the simplest way to avoid deadlocks arising from nested locking, it is a significant limitation. Ideally, the visitor should trace through each function call, resolve which shared variables flow into any nested critical sections, and handle the corresponding locking at the original call site.

To illustrate what a complete implementation would require, consider [Code 44].

```

1 Shared int x = 100; //mutex m0 generated
2 Shared int y = 10;  //mutex m1 generated
3 Shared int z = 400; //mutex m2 generated
4
5 critical(z, y) {
6     y = x - y;
7     criticalFunc1(y, x, z);
8 }
9
10 criticalFunc1(int a, int b, int c) {
11     critical(a, c) {
12         a = a + 10;
13         c = c - a;
14     }
15     criticalFunc2(b);
16 }
17
18 criticalFunc2(int a) {
19     critical(a) {
20         a = a * 2;
21     }
22 }
23
24 print("x: " + x + ", y: " + y + ", z: " + z);

```

Code 44: ParaLang complex nested critical code example.

When the critical section on line 5 is entered, the mutexes corresponding to `z` and `y`, namely `m2` and `m1`, must be acquired in ascending order: `m1 -> m2`. Inside the section, `criticalFunc1` is called with `y`, `x`, `z` as arguments, mapped to parameters `a`, `b`, `c` respectively. The visitor would need to inspect the function body and determine that its internal critical section operates on `a` and `c`, which resolve back to the shared variables `y` and `z`. Since the corresponding mutexes are already held by the parent scope, no new locks need to be acquired, and crucially, they must not be released when the function returns, as the parent critical section still holds them.

The situation becomes more complex with the nested call to `criticalFunc2`, which receives `b`, resolving to the shared variable `x`. Its internal critical section therefore requires `m0`, which is not currently held. According to the deadlock avoidance strategy in [Section 2.2.2], all required locks must be re-evaluated and acquired in ascending order. However, since `m0` ranks below the already-held `m1` and `m2`, the full lock set `m0 -> m1 -> m2` would need to be acquired from the beginning. This re-acquisition logic cannot be placed inside `criticalFunc2` itself, as the required context, which locks are already held and which shared variables are in scope, only exists at the original call site on line 7.

This means that a complete mutex visitor would need to recursively trace all nested function calls originating from within a critical section, resolve the shared variables flowing into any critical sections encountered along the way, and accumulate all of the resulting locking logic back at the original call site. Given the complexity of this and the time constraints of the project, it was decided to instead disallow calling functions that contain critical sections from within a critical section in the current implementation.

7.2.4 Threads

NEW CHANGE HERE! 

As described in Section [4.1.3] and [Requirement 13], threads in ParaLang are created by binding a block of code to an identifier using the `=>` operator.

```
1  thread t1 => {  
2      doWork();  
3  }  
4  
5  thread t2 => {  
6      doOtherWork();  
7  }  
8  
9  awaitAll(t1, t2);
```

Code 45: ParaLang thread assignment and synchronisation example.

Once spawned, threads can be synchronized using `awaitAll` and `awaitAny`. As shown in [Code 45], `awaitAll` blocks until every listed thread has completed, while `awaitAny` unblocks as soon as any one of them finishes. These two constructs form the entirety of how the programmer interacts with a thread after it has been spawned in the current implementation.

Boolean Thread Status

The original design intention, as outlined in requirement [Requirement 18] and Section [4.1.3], was for a thread identifier to be usable directly in boolean expressions, evaluating to `false` while the thread is still running and `true` once it has terminated. This would allow for constructs as seen in [Code 46].

```
1  thread t1 => {  
2      doWork();  
3  }  
4  
5  if(t1) {  
6      print("t1 has finished");  
7  }
```

Code 46: ParaLang intended boolean thread status example.

This is not the case in the current implementation. While `awaitAll` and `awaitAny` provide a means of synchronization, the boolean status of an individual thread cannot be queried directly. Implementing this would require the thread identifier to carry a reference to the underlying `CompletableFuture` mentioned in [Section 5.5] or similar construct in the generated Java code, such that its completion status could be resolved at runtime. This was planned during development, but was ultimately left as future work due to time constraints.

Thread Overhead

ParaLang currently spawns a new thread for every thread assignment, with no use of thread pools. In a thread pool, a fixed set of worker threads are reused across tasks, avoiding the overhead of repeatedly creating and terminating threads. Without this, each spawned thread has the potential to slow down the program if the granularity is too fine, as described in [Section 2.3.2].

While this has not been benchmarked directly, the theoretical implication is that programs with fine-grained parallelism could suffer significant overhead. Introducing a thread pool in a future implementation would allow for improved performance for fine-grained use cases.

7.2.5 Why Java and not C++?

NEW CHANGE HERE! 

In the early stages of the project, after the analysis, the project aimed to be very efficient and to provide a strong performance for large tasks. This naturally made C++ a very appealing choice for translating ParaLang, as it is very low level, has great performance for large tasks, and offers a relatively modern or high level coding experience. The problem, however, was that the time schedule was already slipping, and none of the group members knew C++. An important decision had to be made.

Since none of the members had any previous knowledge of C++, a

Create a low level MVP that barely met requirements written in and translates to C++ or create a better product written in java? The group was already used to write in Java and it supports a wide variety of parallelization and synchronization libraries that could be used to bring the MVP much closer in terms of time. Some good arguments for C++ were made, though: C++ is a very valuable language to know, as it is widely used in the real world because of its performance benefits.

7.3 Something about antlr

NEW CHANGE HERE! 

7.3.1 Grammar rule for assignment

When the grammar was being developed, there were a few bumps that had to be overcome in order to get a working grammar. One of these bumps was in the assignment grammar rule. Before the assignment grammar rule was defined, as illustrated in [Code 10], it was written as shown in the code below [Code 47].

```
1 assignment
2 : PREFIX? typeRef ID ('=' expr ';' | '(' (PREFIX? typeRef ID (',' PREFIX?
  ↳ typeRef ID)*)? ')' block)
3 ;
```

Code 47: Assignment grammar rule before change.

The reason for this change was mainly that ANTLR could not differentiate whether a branch in the tree was assigning a variable or a function. This was because the first part of each functionality was nearly identical to one another. The ANTLR will only read a few tokens and make a decision about what to do next based on the input. Based on the design, it would only be able to assign a variable and never a function. This is due to the hierarchy that will take the first right fit based on the input; this is also described in [Section 5.1.2]. In the end, the change not only works but is also more readable for humans.

7.3.2 Lexar rule for prefix

In the beginning the lexar rule for prefixes was designed to be either `shared`, `const` or `static`. However, this was quickly replaced with the code below [Code 48].

```
1 PREFIX : 'shared' 'const'? 'static'? | 'shared'? 'const' 'static'? | 'shared'?  
    ↪ 'const'? 'static' ;
```

Code 48: The first version of PREFIX.

It was realized that a prefix could entail more than one prefix; however, it could only be two prefixes combined and not every single combination that would work. `shared` and `const` would not be a viable combination because a `shared` variable that is meant to be changed should not be paired with a `const` that is not meant to be changed. Then, finally, a more optimized version of the prefix combination was created and is illustrated below [Code 49].

```
1 PREFIX : 'shared' 'static'? | 'const' 'static'? | ('shared' | 'const')? 'static' ;
```

Code 49: The final version of PREFIX.

Chapter 8

Conclusion

Must Have				Should Have		Could Have	
ID	Done	ID	Done	ID	Done	ID	Done
[1]	✓	[8]	✓	[15]	✓	[21]	
[2]	✓	[9]	✓	[16]	✓	[22]	
[3]	✓	[10]	✓	[17]	✓	[23]	
[4]	✓	[11]	✓	[18]		[24]	✓
[5]	✓	[12]	✓	[19]	✓	[25]	
[6]	✓	[13]	✓	[20]	✓	[26]	
[7]	✓	[14]	✓			[27]	
						[28]	✓

Table 8.1: All Requirements [Section 3].

Color Guide:
Must Have Should Have Could Have

Bibliography

- [1] Moore Gordon. *Understanding Moore's Law: Four Decades of Innovation*. Ed. by David C. Brock. Chemical Heritage Foundation, 2006. ISBN: 9780941901413.
- [2] Karl Rupp. *Transistors per Microprocessor*. Our World in Data. 2022. URL: <https://ourworldindata.org/grapher/transistors-per-microprocessor>.
- [3] Thomas N. Theis and H.-S. Philip Wong. "The End of Moore's Law: A New Beginning for Information Technology". In: (2017), pp. 41–50. DOI: 10.1109/MCSE.2017.29.
- [4] Audrey Woods. *The Death of Moore's Law: What it means and what might fill the gap going forward*. URL: <https://cap.csail.mit.edu/death-moores-law-what-it-means-and-what-might-fill-gap-going-forward>.
- [5] Redha Gouicem. *Thread scheduling in multi-core operating systems: how to understand, improve and fix your scheduler*. Feb. 2023. URL: <https://theses.hal.science/tel-03987730/>.
- [6] Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts 10th Edition*. Wiley, 2021. ISBN: 978-1-118-06333-0.
- [7] Inderpal Singh. *Review on Parallel and Distributed Computing*. 2013. URL: <https://saspublishers.com/media/articles/SJET14218-225.pdf>.
- [8] Matthew Hennessy and Robin Milner. *Algebraic laws for nondeterminism and concurrency*. Jan. 1985. URL: <https://doi.org/10.1145/2455.2460>.
- [9] Ajay D Kshemkalyani and Mukesh Singhal. *Distributed Computing*. Cambridge, England: Cambridge University Press, Apr. 2008.
- [10] Aisha Mushtaq. *Fault Tolerance in Distributed Systems*. 2022. URL: https://kjdk-aub.primo.exlibrisgroup.com/permalink/45KBDK_AUB/1hibt6n/cdi_cdl_escholarship_oai_escholarship_org_ark_13030_qt2k53r3sz.

Bibliography

- [11] Richard P. Martin et al. "Effects of communication latency, overhead, and bandwidth in a cluster architecture". In: New York, NY, USA: ACM, 1997. ISBN: 9780897919012. URL: https://kjdk-aub.primo.exlibrisgroup.com/permalink/45KBDK_AUB/1hibt6n/cdi_acm_books_10_1145_264107_264146.
- [12] Paul E. McKenney. *Is Parallel Programming Hard, And, If So, What Can You Do About It?* June 2023. URL: <https://arxiv.org/abs/1701.00854>.
- [13] Dean M. Tullsen, Susan J. Eggers, and Henry M. Levy. *Simultaneous Multithreading: Maximizing On-Chip Parallelism*. June 1995. URL: <https://www.eecs.harvard.edu/cs146-246/smt.pdf>.
- [14] Jun Nakajima and Venkatesh Pallipadi. *Enhancements for Hyper-Threading Technology in the Operating System: Seeking the Optimal Scheduling*. Nov. 2002. URL: https://www.usenix.org/legacy/event/wiess02/tech/full_papers/nakajima/nakajima.pdf.
- [15] Susan J. Eggers et al. *Simultaneous Multithreading: A Platform for Next-generation Processors*. Sept. 1997. URL: <https://cse.msu.edu/~cse820/readings/eggers97simultaneous.pdf>.
- [16] Microsoft Corporation. C++ Standard Library header files: <mutex>. Mar. 2022. URL: <https://learn.microsoft.com/da-dk/cpp/standard-library/mutex?view=msvc-170>.
- [17] Python Software Foundation. *The Python Standard Library (3.14.4 Documentation) » Concurrent Execution » threading - Thread-based parallelism*. Apr. 2026. URL: <https://docs.python.org/3/library/threading.html>.
- [18] Jonathan Corbet. *Finding race conditions with KCSAN*. Oct. 2019. URL: <https://lwn.net/Articles/802128/>.
- [19] Jonathan Corbet. *The kernel lock validator*. May 2006. URL: <https://lwn.net/Articles/185666/>.
- [20] Mahdi Eslamimehr. *Efficient detection and validation of atomicity violations in concurrent programs*. Mar. 2018. URL: <https://www.sciencedirect.com/science/article/pii/S0164121217301085>.
- [21] Sara Abbaspour Asadollah. *Concurrency bugs in open source software: a case study*. Apr. 2017. URL: https://link.springer.com/article/10.1186/s13174-017-0055-2?utm_source=chatgpt.com.
- [22] E A Goncharenko, A A Paznikov, and A V Tabakov. "Evaluating the performance of atomic operations on modern multicore systems". In: *Journal of Physics: Conference Series* 1399.3 (Dec. 2019), p. 033107. doi: 10.1088/1742-6596/1399/3/033107. URL: <https://doi.org/10.1088/1742-6596/1399/3/033107>.

Bibliography

- [23] Wim H. Hesselink. “An assertional proof for a construction of an atomic variable”. English. In: *Formal Aspects of Computing* 16.4 (Nov. 2004). Copyright - Copyright Springer-Verlag 2004; Last updated - 2023-11-24; CODEN - FACME5, pp. 387–393. ISSN: 0934-5043. URL: <https://www.proquest.com/scholarly-journals/assertional-proof-construction-atomic-variable/docview/203588135/se-2>.
- [24] David Dice, Virendra J. Marathe, and Nir Shavit. *Lock Cohorting: A General Technique for Designing NUMA Locks*. Feb. 2012. URL: <https://dl.acm.org/doi/10.1145/2370036.2145848>.
- [25] Xian-He Sun and Xiaoyang Lu. *The Memory-Bounded Speedup Model and Its Impacts in Computing*. Oct. 2022. URL: <https://link.springer.com/article/10.1007/s11390-022-2911-1>.
- [26] Mike Bailey. *Parallel Programming: Speedups and Amdahl's law*. Mar. 2025. URL: <https://web.engr.oregonstate.edu/~mjb/cs575/Handouts/speedups.and.amdahls.law.1pp.pdf>.
- [27] Sasko Ristov and Marjan Gusev. *Superlinear Speedup for Matrix Multiplication*. 2012. URL: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6308001>.
- [28] Maurice Herlihy and Nir Shavit. *The Art of Multiprocessor Programming*. 2nd ed. Morgan Kaufmann, 2021, pp. 59–61. ISBN: 978-0124159501.
- [29] Brian Goetz. *Java theory and practice: Going atomic*. Mar. 2007. URL: https://courses.cs.vt.edu/~cs5204/fall07-kafura/Papers/TransactionalMemory/Java%20theory%20and%20practice_%20Going%20atomic.pdf.
- [30] Jesper Madsen. *Lock-based versus non-blocking algorithms*. Master's thesis, Aalborg University, 10th term, EVU. Oct. 2006. URL: <https://projekter.aau.dk/projekter/files/61067775/1149254542.pdf>.
- [31] Kiko Fernandez-Reyes et al. *Dala: A Simple Capability-Based Dynamic Language Design For Data Race-Freedom*. 2021. URL: <https://michael.homer.nz/Publications/Onward2021/SimpleCapabilityBased-FernandezReyes2021.pdf>.
- [32] Joe Armstrong. *Communications of the ACM*. 2010. URL: <https://dl.acm.org/doi/fullHtml/10.1145/1810891.1810910>.
- [33] Francesco Cesarini and Simon Thompson. *Erlang Programming: A Concurrent Approach to Software Development*. O'Reilly Media, Inc., 2009. ISBN: 9780596555856.
- [34] Natalia Chechina et al. *Improving the network scalability of Erlang*. Apr. 2016. URL: <https://www.sciencedirect.com/science/article/abs/pii/S0743731516000034>.
- [35] Emil Gustavsson Gregor Ulm and Mats Jirstrand. *Functional Federated Learning in Erlang*. Mar. 2019. URL: <https://arxiv.org/pdf/1808.08143>.

Bibliography

- [36] Sylvan W. Clebsch. *Pony: Co-designing a Type System and a Runtime*. Oct. 2017. URL: <https://www.ponylang.io/media/papers/codesigning.pdf>.
- [37] Steve Klabnik et al. *The Rust Programming Language*. Online edition covering Rust 1.90.0 (as of 2025) and later. No Starch Press, 2025. URL: <https://doc.rust-lang.org/book/>.
- [38] Ralf Jung et al. *Safe Systems Programming in Rust*. Apr. 2021. URL: <https://cacm.acm.org/research/safe-systems-programming-in-rust/>.
- [39] Aaron Turon. *Fearless Concurrency*. Apr. 2015. URL: <https://blog.rust-lang.org/2015/04/10/Fearless-Concurrency.html>.
- [40] Zeming Yu, Linhai Song, and Yiyang Zhang. *Fearless Concurrency? Understanding Concurrent Programming Safety in Real-World Rust Software*. Feb. 2019. URL: <https://arxiv.org/pdf/1902.01906>.
- [41] *AAU Learning Goals*. URL: <https://moduler.aau.dk/course/2024-2025/DSNSWB410>.
- [42] Terrence Parr. *The definitive ANTLR 4 reference*. 2013. URL: <https://www.torrossa.com/en/resources/an/5241753>.
- [43] Torben Ægidius Mogensen. *Introduction to Compiler Design, 3rd edition*. Springer, 2024. ISBN: 9783031464607.
- [44] Paul C. Jorgensen. *Object-Oriented Integration Testing*. Sept. 1994. URL: <https://dl.acm.org/doi/pdf/10.1145/182987.182989>.

Chapter 9

Appendices

9.1 Context Free Grammar

```
1  grammar OurGrammar;
2
3  program
4      : statement* EOF
5      ;
6
7  statement
8      : statementPrime ';'
9      | assignment
10     | threadAssignment
11     | ifStatement
12     | forStatement
13     | whileStatement
14     | criticalSection
15     | block
16     ;
17
18 statementPrime
19     : reassignment
20     | declaration
21     | awaitStatement
22     | continueStatement
23     | breakStatement
24     | returnStatement
25     | printStatement
26     | expr
```

```

27     ;
28
29 assignment
30     : PREFIX? typeRef ID (assVar ';' | assFunc)
31     ;
32
33 assVar
34     : '=' expr
35     ;
36
37 assFunc
38     : '(' (PREFIX? typeRef ID (',' PREFIX? typeRef ID)*)? ')' block
39     ;
40
41 reassignment
42     : ID '=' expr
43     | arrayIndex '=' expr
44     ;
45
46 declaration
47     : PREFIX? typeRef ID
48     ;
49
50 threadAssignment
51     : typeRef ID '=>' block
52     ;
53
54 awaitStatement
55     : 'awaitAll' '(' ID (',' ID)* ')'
56     | 'awaitAny' '(' ID (',' ID)* ')'
57     ;
58
59 castExpression
60     : 'cast' '(' TYPE ')' expr
61     ;
62
63 typeRef
64     : TYPE ('[' (ID | INT)? ']')*
65     ;
66
67 ifStatement
68     : 'if' '(' expr ')' statement
69     ('else if' '(' expr ')' statement)*
70     ('else' statement)?
71     ;
72
73 forStatement

```



```

74     : 'for' '(' forVar ';' expr ';' reassignment ')' statement
75     ;
76
77 forVar
78     : PREFIX? typeRef ID assVar
79     | reassignment
80     | ID
81     ;
82
83 whileStatement
84     : 'while' '(' expr ')' statement
85     ;
86
87 continueStatement
88     : 'continue'
89     ;
90
91 breakStatement
92     : 'break'
93     ;
94
95 block
96     : '{' statement* '}'
97     ;
98
99 returnStatement
100    : 'return' expr?
101    ;
102
103 criticalSection
104    : 'critical' '(' ID (',' ID)* ')' block
105    ;
106
107 printStatement
108    : 'print' '(' expr (',' expr)* ')'
109    ;
110
111 read
112    : 'read' '(' TYPE ')'
113    ;
114
115 expr
116    : equal ('&&' | '||') expr
117    | equal
118    ;
119
120 equal

```

```

121     : comp ('==' | '!=') equal
122     | comp
123     ;
124
125 comp
126     : additive ('<' | '>' | '<=' | '>=') comp
127     | additive
128     ;
129
130 additive
131     : mult ('+' | '-') additive
132     | mult
133     ;
134
135 mult
136     : power ('*' | '/' | '%') mult
137     | power
138     ;
139
140 power
141     : factor '^' power
142     | factor
143     ;
144
145 arrayLiteral
146     : ('{' expr (',' expr)* '}'')+
147     | ('[' expr ']'')+
148     ;
149
150 arrayIndex
151     : ID ('[' expr ']'')+
152     ;
153
154 factor
155     : NEGATIVE factor
156     | functionCall
157     | arrayIndex
158     | arrayLiteral
159     | ID
160     | INT
161     | FLOAT
162     | BOOL
163     | CHAR
164     | STRING
165     | THREAD
166     | '(' expr ')'
167     | castExpression

```

```

168     | read
169     ;
170
171 functionCall
172     : ID '(' (expr (',' expr)*)? ')'
173     ;
174
175 NEGATIVE : '-';
176 INT : [0-9]+ ;
177 FLOAT : [0-9]+ '.' [0-9]* ;
178 BOOL : 'true' | 'false' ;
179 CHAR : '\'' . '\'' ;
180 STRING : '"' .*? '"' ;
181 THREAD : BOOL ;
182 PREFIX : 'shared' 'static'? | 'const' 'static'? | ('shared' | 'const')? 'static' ;
183 TYPE : 'int' | 'float' | 'bool' | 'char' | 'string' | 'void' | 'thread' ;
184 ID : [a-zA-Z_][a-zA-Z0-9_]* ;
185
186 LINE_COMMENT : '//' ~[\r\n]* -> skip ;
187 BLOCK_COMMENT : '/*' .*? '*/' -> skip ;
188 WS : [\t\r\n]+ -> skip ;

```

9.2 Syntax and Semantics

Syntactic Categories

$e \in E$ (Exp)	$\sigma \in \Sigma$ (Stm)
$v \in V$ (Val)	$x \in X$ (Var)
$t \in T$ (Types)	$f \in F$ (Func)
$s \in S$ (States)	$g \in G$ (Global States)
$\gamma_s \in \Gamma_s$ (Local Type Environment)	$\gamma_g \in \Gamma_g$ (Global Type Environment)
$l \in L$ (Loc)	$a \in A$ (Array)
$\tau \in \mathcal{T}$ (Thread)	$r \in \mathcal{R}$ (Result)
$k \in K$ (Array Size)	$m \in M$ (Modifier)
$\mu \in \mathcal{M}$ (Mutex)	

$$\mathcal{R} ::= S \mid \text{cnt} \mid \text{brk} \mid \text{ret} \mid \text{ret}(V)$$

For any meta-variable x , $\bar{x}$ will be shorthand for a finite sequence $x_1, \dots, x_n$ for some $n \geq 0$. Sequence concatenation is written $x_1 \oplus \bar{x}'$ and ε denotes the empty sequence.

Transition Functions

$$\begin{aligned} S \times G \times E &\rightarrow V \times G \\ S \times G \times \Sigma &\rightarrow \mathcal{R} \times G \end{aligned}$$

Functions

$\text{ord} : \mathcal{M} \rightarrow \mathbb{N}$	(returns the order of creation)
$\text{sort} : \bar{\mathcal{M}} \rightarrow \bar{\mathcal{M}}$	(sorts using "ord" function)
$\text{min}_{\text{ord}} : \bar{\mathcal{M}} \rightarrow \mathcal{M}$	(returns the mutex with lowest "ord")

Formation Rules

$$e ::= v \mid x \mid \tau \mid e_1 + e_2 \mid e_1 - e_2 \mid e_1 * e_2 \mid e_1 / e_2 \mid e_1 \% e_2 \mid e_1 \hat{=} e_2 \mid e_1 \&\& e_2 \mid e_1 \mid\mid e_2 \mid \\ e_1 == e_2 \mid e_1 != e_2 \mid e_1 < e_2 \mid e_1 > e_2 \mid e_1 \leq e_2 \mid e_1 \geq e_2 \mid (e) \mid f() \mid \\ f(e_1, \dots, e_n) \mid \{e_1, \dots, e_n\} \mid x[e_1] \dots [e_n] \mid \text{read}(t) \mid \text{cast}(t) e$$

$$\sigma ::= \sigma; \sigma \mid \{\sigma\} \mid e \mid m \ t \ x; \sigma \mid m \ t \ x := e; \sigma \mid x = e \mid \text{Thread } \tau \Rightarrow \sigma_b; \sigma_c \mid t \ f() \sigma_b; \sigma \mid \\ t \ f(m_1 \ t_1 \ p_1, \dots, m_n \ t_n \ p_n) \sigma_b; \sigma \mid \text{print}(e_1, \dots, e_n) \mid \text{if } e \text{ then } \sigma_1 \text{ else } \sigma_2 \mid \\ \text{if } e \text{ then } \sigma \mid \text{while } e \text{ do } \sigma \mid \text{for}(m \ t \ x := e_1; e_c; \sigma_r) \sigma_b \mid \text{for}(x = e_1; e_c; \sigma_r) \sigma_b \mid \\ \text{for}(x; e; \sigma_r) \sigma_b \mid m \ t[] \dots [] \ x; \sigma \mid m \ t[e_1] \dots [e_n] \ x; \sigma \mid m \ t[] \dots [] \ x := e; \sigma \mid \\ m \ t[e_1] \dots [e_n] \ x := e_v; \sigma \mid x = [e_1] \dots [e_n] \mid x[e_1] \dots [e_n] = e_v \mid \\ \text{awaitAll}(\tau_1, \dots, \tau_n) \mid \text{awaitAny}(\tau_1, \dots, \tau_n) \mid \text{critical}(x_1, \dots, x_n) \sigma \mid \\ \text{continue} \mid \text{break} \mid \text{return} \mid \text{return } e$$

$$t ::= \text{Int} \mid \text{Float} \mid \text{Bool} \mid \text{Char} \mid \text{String} \mid t[k] \mid \text{Void}$$

$$k ::= \varepsilon \mid e$$

$$m ::= \varepsilon \mid \text{Shared} \mid \text{Const}$$

Typing Contexts

$$\gamma_s ::= (\Delta, b_l, (b_f, t))$$

$$\gamma_g ::= (X \multimap T) \times (\mathcal{T} \multimap \text{thread})$$

$$\Delta ::= (X \multimap T) \times (F \multimap \text{func}((\bar{m}, \bar{t}), t, \sigma))$$

$$b_l, b_f \in \{tt, ff\}$$

where $b_l = tt$ in a loop, and $b_f = tt$ in a function

Statement Definition

$$S = \left[\begin{array}{lcl} x & \mapsto & l \\ f & \mapsto & \text{func}(\bar{p}, \sigma) \\ \mu^O & \mapsto & \bar{\mu} \end{array} \right]$$

$$G = \left[\begin{array}{lcl} x & \mapsto & v \\ l & \mapsto & v \\ \tau & \mapsto & \text{thread}(b, \sigma, s) \\ \mu_x & \mapsto & \text{free} \mid \tau \end{array} \right]$$

τ_0 denotes the main thread

μ^O maps to the mutexes already owned by the thread in the outer section. This is needed for a critical section to know which mutexes not to drop after executing.

μ_x denotes the mutex associated with the shared variable x , where **free** means unlocked, and τ means locked by thread τ .

Type System

$$(\text{PROGRAM}_{TS}) \frac{(\gamma_{s_0}, \gamma_{g_0}) \vdash \sigma : \text{ok}}{\vdash \sigma : \text{ok}} \quad \begin{array}{l} \gamma_{s_0} = (\emptyset, ff, (ff, \text{Void})) \\ \gamma_{g_0} = (\emptyset, \emptyset) \end{array}$$

Expressions

Typing Judgement: $(\gamma_s, \gamma_g) \vdash e : t \quad e \in E, t \in T$

$$\begin{array}{ll} [\text{INT}_{TS}] \frac{}{(\gamma_s, \gamma_g) \vdash z : \text{Int}} & (\text{FLOAT}_{TS}) \frac{}{(\gamma_s, \gamma_g) \vdash \phi : \text{Float}} \\ (\text{BOOL}_{TS}) \frac{}{(\gamma_s, \gamma_g) \vdash b : \text{Bool}} & (\text{CHAR}_{TS}) \frac{}{(\gamma_s, \gamma_g) \vdash 'c' : \text{Char}} \\ (\text{STRING}_{TS}) \frac{}{(\gamma_s, \gamma_g) \vdash \text{"str"} : \text{String}} & (\text{THREAD}_{TS}) \frac{}{(\gamma_s, \gamma_g) \vdash \tau : \text{Bool}} \\ \\ [\text{VAR}_{TS}] \frac{\Delta(x) = t}{(\gamma_s, \gamma_g) \vdash x : t} \quad \gamma_s = (\Delta, b_l, (b_f, t_f)) & \\ [\text{VAR-GLOBAL}_{TS}] \frac{\gamma_g(x) = t}{(\gamma_s, \gamma_g) \vdash x : t} \quad x \notin \gamma_s & [\text{PAR}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : t}{(\gamma_s, \gamma_g) \vdash (e) : t} \\ [\star_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e_1 : t \quad (\gamma_s, \gamma_g) \vdash e_2 : t}{(\gamma_s, \gamma_g) \vdash e_1 \star e_2 : t} & \star \in \left\{ \begin{array}{l} +, -, *, /, \%, \wedge, \&\&, ||, \\ <, >, <=, >=, ==, != \end{array} \right\} \end{array}$$

$$\begin{array}{c}
\text{[ARGS}_{TS}\text{]} \frac{(\gamma_s, \gamma_g) \vdash e_1 : t_1 \quad (\gamma_s, \gamma_g) \vdash (e_2, \dots, e_n) : \bar{t}'}{(\gamma_s, \gamma_g) \vdash (e_1, \dots, e_n) : \bar{t}} \quad \bar{t} = t_1 \oplus \bar{t}' \\
\\
\text{[ARGS-EMPTY}_{TS}\text{]} \frac{}{(\gamma_s, \gamma_g) \vdash () : \varepsilon} \\
\\
\text{[FUN-CALL}_{TS}\text{]} \frac{\Delta(f) = \mathbf{func}((\bar{m}, \bar{t}), t, \sigma) \quad (\gamma_s, \gamma_g) \vdash (e_1, \dots, e_n) : \bar{t}}{(\gamma_s, \gamma_g) \vdash f(e_1, \dots, e_n) : t} \quad \gamma_s = (\Delta, b_l, (b_f, t_r)) \\
\\
\text{[FUN-CALL-EMPTY}_{TS}\text{]} \frac{\Delta(f) = \mathbf{func}(\varepsilon, t, \sigma)}{(\gamma_s, \gamma_g) \vdash f() : t} \quad \gamma_s = (\Delta, b_l, (b_f, t_r)) \\
\\
\text{[ARR-LIT}_{TS}\text{]} \frac{(\gamma_s, \gamma_g) \vdash e_1 : t \quad (\gamma_s, \gamma_g) \vdash \{e_2, \dots, e_n\} : t[n-1]}{(\gamma_s, \gamma_g) \vdash \{e_1, \dots, e_n\} : t[n]} \\
\\
\text{[ARR-LIT-EMPTY}_{TS}\text{]} \frac{}{(\gamma_s, \gamma_g) \vdash \{ \} : t[0]} \\
\\
\text{[ARR-INDEX}_{TS}\text{]} \frac{(\gamma_s, \gamma_g) \vdash (e_1, \dots, e_n) : \bar{\mathbf{Int}}}{(\gamma_s, \gamma_g) \vdash x[e_1] \dots [e_n] : t} \quad \begin{array}{l} \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \Delta(x) = t[k] \end{array} \\
\\
\text{[ARR-INDEX-SHARED}_{TS}\text{]} \frac{(\gamma_s, \gamma_g) \vdash (e_1, \dots, e_n) : \bar{\mathbf{Int}}}{(\gamma_s, \gamma_g) \vdash x[e_1] \dots [e_n] : t} \quad \begin{array}{l} x \notin \gamma_s \\ \gamma_g(x) = t[k] \end{array} \\
\\
\text{[READ}_{TS}\text{]} \frac{}{(\gamma_s, \gamma_g) \vdash \mathbf{read}(t) : t} \\
\\
\text{[CAST-FI}_{TS}\text{]} \frac{(\gamma_s, \gamma_g) \vdash e : \mathbf{Float}}{(\gamma_s, \gamma_g) \vdash \mathbf{cast}(\mathbf{Int}) e : \mathbf{Int}} \\
\\
\text{[CAST-BI}_{TS}\text{]} \frac{(\gamma_s, \gamma_g) \vdash e : \mathbf{Bool}}{(\gamma_s, \gamma_g) \vdash \mathbf{cast}(\mathbf{Int}) e : \mathbf{Int}}
\end{array}$$

$$[\text{CAST-IF}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : \text{Int}}{(\gamma_s, \gamma_g) \vdash \text{cast}(\text{Float}) e : \text{Float}}$$

$$[\text{CAST-BF}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : \text{Bool}}{(\gamma_s, \gamma_g) \vdash \text{cast}(\text{Float}) e : \text{Float}}$$

$$[\text{CAST-IB}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : \text{Int}}{(\gamma_s, \gamma_g) \vdash \text{cast}(\text{Bool}) e : \text{Bool}}$$

$$[\text{CAST-FB}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : \text{Float}}{(\gamma_s, \gamma_g) \vdash \text{cast}(\text{Bool}) e : \text{Bool}}$$

$$[\text{CAST-CS}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : \text{Char}}{(\gamma_s, \gamma_g) \vdash \text{cast}(\text{String}) e : \text{String}}$$

$$[\text{CAST-IS}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : \text{Int}}{(\gamma_s, \gamma_g) \vdash \text{cast}(\text{String}) e : \text{String}}$$

$$[\text{CAST-FS}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : \text{Float}}{(\gamma_s, \gamma_g) \vdash \text{cast}(\text{String}) e : \text{String}}$$

$$[\text{CAST-BS}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : \text{Bool}}{(\gamma_s, \gamma_g) \vdash \text{cast}(\text{String}) e : \text{String}}$$

Statements

Typing Judgement: $(\gamma_s, \gamma_g) \vdash \sigma : \text{ok}$ where $\sigma \in \Sigma$

$$[\text{COMP}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash \sigma_1 : \text{ok} \quad (\gamma_s, \gamma_g) \vdash \sigma_2 : \text{ok}}{(\gamma_s, \gamma_g) \vdash \sigma_1; \sigma_2 : \text{ok}}$$

$$[\text{COMP-ABORT}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash \sigma_1 : \text{ok} \quad (\gamma_s, \gamma_g) \vdash \sigma_2 : \text{ok}}{(\gamma_s, \gamma_g) \vdash \sigma_1; \sigma_2 : \text{ok}}$$

$$[\text{BLOCK}_{TS}] \frac{(\gamma'_s, \gamma_g) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash \{\sigma\} : \text{ok}} \quad \begin{array}{l} \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \gamma'_s = (\Delta, b_l, (b_f, t_f)) \end{array}$$

$$[\text{EXP}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : t}{(\gamma_s, \gamma_g) \vdash e : \text{ok}}$$

$$[\text{DEC-EMPTY}_{TS}] \frac{(\gamma'_s, \gamma_g) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash m \ t \ x; \sigma : \text{ok}} \quad \begin{array}{l} m \neq \text{shared} \\ \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \gamma'_s = (\Delta[x \mapsto t], b_l, (b_f, t_f)) \end{array}$$

$$[\text{DEC}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : t \quad (\gamma'_s, \gamma_g) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash m \ t \ x := e; \sigma : \text{ok}} \quad \begin{array}{l} m \neq \text{shared} \\ \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \gamma'_s = (\Delta[x \mapsto t], b_l, (b_f, t_f)) \end{array}$$

$$[\text{DEC-SHARED-EMPTY}_{TS}] \frac{(\gamma_s, \gamma_g[x \mapsto t]) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash m \ t \ x; \sigma : \text{ok}} \quad \begin{array}{l} m = \text{shared} \\ x \notin \gamma_g \end{array}$$

$$[\text{DEC-SHARED}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : t \quad (\gamma_s, \gamma_g[x \mapsto t]) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash m \ t \ x := e; \sigma : \text{ok}} \quad \begin{array}{l} m = \text{shared} \\ x \notin \gamma_g \end{array}$$

$$[\text{ASS}_{TS}] \frac{\Delta(x) = t \quad (\gamma_s, \gamma_g) \vdash e : t}{(\gamma_s, \gamma_g) \vdash x = e : \text{ok}} \quad \gamma_s = (\Delta, b_l, (b_f, t_f))$$

$$[\text{ASS-SHARED}_{TS}] \frac{\gamma_g(x) = t \quad (\gamma_s, \gamma_g) \vdash e : t}{(\gamma_s, \gamma_g) \vdash x = e : \text{ok}} \quad x \notin \gamma_s$$

$$\begin{array}{c}
\text{[THREAD-SPAWN}_{TS}\text{]} \frac{(\gamma_s, \gamma_g) \vdash \sigma_b : \text{ok} \quad (\gamma_s, \gamma'_g) \vdash \sigma_c : \text{ok}}{(\gamma_s, \gamma_g) \vdash \text{Thread } \tau \Rightarrow \sigma_b; \sigma_c : \text{ok}} \quad \begin{array}{l} \tau \notin \gamma_g \\ \gamma'_g = \gamma_g[\tau \mapsto \text{thread}] \end{array} \\
\\
\text{[FUN-DEF}_{TS}\text{]} \frac{(\gamma''_s, \gamma_g) \vdash \sigma_f : \text{ok} \quad (\gamma'_s, \gamma_g) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash t f(m_1 t_1 p_1, \dots, m_n t_n p_n) \sigma_f; \sigma : \text{ok}} \quad \begin{array}{l} \forall i \in \{1, \dots, n\} \quad m_i \neq \text{shared} \\ \bar{m} = (m_1, \dots, m_n) \\ \bar{t} = (t_1, \dots, t_n) \\ \gamma'_s = \gamma_s[f \mapsto \text{func}((\bar{m}, \bar{t}), t, \sigma_f)] \\ \gamma''_s = ([\bar{p} \mapsto \bar{t}], ff, (tt, t)) \end{array} \\
\\
\text{[PRINT}_{TS}\text{]} \frac{(\gamma_s, \gamma_g) \vdash (e_1, \dots, e_n) : \bar{t}}{(\gamma_s, \gamma_g) \vdash \text{print}(e_1, \dots, e_n) : \text{ok}} \\
\\
\text{[IF-ELSE}_{TS}\text{]} \frac{(\gamma_s, \gamma_g) \vdash e : \text{Bool} \quad (\gamma_s, \gamma_g) \vdash \sigma_1 : \text{ok} \quad (\gamma_s, \gamma_g) \vdash \sigma_2 : \text{ok}}{(\gamma_s, \gamma_g) \vdash \text{if } e \text{ then } \sigma_1 \text{ else } \sigma_2 : \text{ok}} \\
\\
\text{[IF}_{TS}\text{]} \frac{(\gamma_s, \gamma_g) \vdash e : \text{Bool} \quad (\gamma_s, \gamma_g) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash \text{if } e \text{ then } \sigma : \text{ok}} \\
\\
\text{[WHILE}_{TS}\text{]} \frac{(\gamma_s, \gamma_g) \vdash e : \text{Bool} \quad (\gamma'_s, \gamma_g) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash \text{while } e \text{ do } \sigma : \text{ok}} \quad \begin{array}{l} \gamma_s = (\Delta, b_l, (b_f, t)) \\ \gamma'_s = (\Delta, tt, (b_f, t)) \end{array}
\end{array}$$

$$\begin{array}{c}
\text{[FOR-DEC}_{TS}\text{]} \frac{(\gamma_s, \gamma_g) \vdash e_1 : t \quad (\gamma'_s, \gamma_g) \vdash e_c : \text{Bool} \quad (\gamma'_s, \gamma_g) \vdash \sigma_b : \text{ok} \quad (\gamma'_s, \gamma_g) \vdash \sigma_r : \text{ok}}{(\gamma_s, \gamma_g) \vdash \text{for}(m \ t \ x := e_1; e_c; \sigma_r) \ \sigma_b : \text{ok}} \quad \begin{array}{l} m \neq \text{shared} \\ \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \gamma'_s = (\Delta[x \mapsto t], tt, (b_f, t_f)) \end{array} \\
\\
\text{[FOR-DEC-SHARED}_{TS}\text{]} \frac{(\gamma_s, \gamma_g) \vdash e_1 : t \quad (\gamma'_s, \gamma_g[x \mapsto t]) \vdash e_c : \text{Bool} \quad (\gamma'_s, \gamma_g[x \mapsto t]) \vdash \sigma_b : \text{ok} \quad (\gamma'_s, \gamma_g[x \mapsto t]) \vdash \sigma_r : \text{ok}}{(\gamma_s, \gamma_g) \vdash \text{for}(m \ t \ x := e_1; e_c; \sigma_r) \ \sigma_b : \text{ok}} \quad \begin{array}{l} m = \text{shared} \\ \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \gamma'_s = (\Delta, tt, (b_f, t_f)) \end{array} \\
\\
\text{[FOR-ASS}_{TS}\text{]} \frac{\Delta(x) = t \quad (\gamma_s, \gamma_g) \vdash e_1 : t \quad (\gamma'_s, \gamma_g) \vdash e_c : \text{Bool} \quad (\gamma'_s, \gamma_g) \vdash \sigma_b : \text{ok} \quad (\gamma'_s, \gamma_g) \vdash \sigma_r : \text{ok}}{(\gamma_s, \gamma_g) \vdash \text{for}(x = e_1; e_c; \sigma_r) \ \sigma_b : \text{ok}} \quad \begin{array}{l} \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \gamma'_s = (\Delta, tt, (b_f, t_f)) \end{array} \\
\\
\text{[FOR-ASS-SHARED}_{TS}\text{]} \frac{\gamma_g(x) = t \quad (\gamma_s, \gamma_g) \vdash e_1 : t \quad (\gamma'_s, \gamma_g) \vdash e_c : \text{Bool} \quad (\gamma'_s, \gamma_g) \vdash \sigma_b : \text{ok} \quad (\gamma'_s, \gamma_g) \vdash \sigma_r : \text{ok}}{(\gamma_s, \gamma_g) \vdash \text{for}(x = e_1; e_c; \sigma_r) \ \sigma_b : \text{ok}} \quad \begin{array}{l} x \notin \Delta \\ \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \gamma'_s = (\Delta, tt, (b_f, t_f)) \end{array} \\
\\
\text{[FOR-ID}_{TS}\text{]} \frac{(\gamma'_s, \gamma_g) \vdash e : \text{Bool} \quad (\gamma'_s, \gamma_g) \vdash \sigma_b : \text{ok} \quad (\gamma'_s, \gamma_g) \vdash \sigma_r : \text{ok}}{(\gamma_s, \gamma_g) \vdash \text{for}(x; e; \sigma_r) \ \sigma_b : \text{ok}} \quad \begin{array}{l} x \in \Delta \\ \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \gamma'_s = (\Delta, tt, (b_f, t_f)) \end{array} \\
\\
\text{[ARR-DEC-EMPTY}_{TS}\text{]} \frac{(\gamma'_s, \gamma_g) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash m \ t[\dots] x; \sigma : \text{ok}} \quad \begin{array}{l} m \neq \text{shared} \\ \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \gamma'_s = (\Delta[x \mapsto t[\varepsilon]], b_l, (b_f, t_f)) \end{array} \\
\\
\text{[ARR-DEC-SHARED-EMPTY}_{TS}\text{]} \frac{(\gamma_s, \gamma_g[x \mapsto t[\varepsilon]]) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash m \ t[\dots] x; \sigma : \text{ok}} \quad \begin{array}{l} m = \text{shared} \\ x \notin \gamma_g \end{array} \\
\\
\text{[ARR-DEC}_{TS}\text{]} \frac{(\gamma_s, \gamma_g) \vdash (e_1, \dots, e_n) : \bar{\text{Int}} \quad (\gamma'_s, \gamma_g) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash m \ t[e_1] \dots [e_n] x; \sigma : \text{ok}} \quad \begin{array}{l} m \neq \text{shared} \\ \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \gamma'_s = (\Delta[x \mapsto t[e_1] \dots [e_n]], b_l, (b_f, t_f)) \end{array} \\
\\
\text{[ARR-DEC-SHARED}_{TS}\text{]} \frac{(\gamma_s, \gamma_g) \vdash (e_1, \dots, e_n) : \bar{\text{Int}} \quad (\gamma_s, \gamma_g[x \mapsto t[e_1] \dots [e_n]]) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash m \ t[e_1] \dots [e_n] x; \sigma : \text{ok}} \quad \begin{array}{l} m = \text{shared} \\ x \notin \gamma_g \end{array}
\end{array}$$

$$[\text{ARR-DEC-ASS}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : t[?] \quad (\gamma'_s, \gamma_g) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash m \, t[\dots] x := e; \sigma : \text{ok}} \quad \begin{array}{l} m \neq \text{shared} \\ \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \gamma'_s = (\Delta[x \mapsto t[\varepsilon]], b_l, (b_f, t_f)) \\ ? : \text{size inferred from } e \end{array}$$

$$[\text{ARR-DEC-ASS-SHARED}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : t[\varepsilon] \quad (\gamma_s, \gamma_g[x \mapsto t[\varepsilon]]) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash m \, t[\dots] x := e; \sigma : \text{ok}} \quad \begin{array}{l} m = \text{shared} \\ x \notin \gamma_g \\ ? : \text{size inferred from } e \end{array}$$

$$[\text{ARR-DEC-SIZE-ASS}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash (e_1, \dots, e_n) : \bar{\text{Int}} \quad (\gamma_s, \gamma_g) \vdash e_v : t[?] \quad (\gamma'_s, \gamma_g) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash m \, t[e_1] \dots [e_n] x := e_v; \sigma : \text{ok}} \quad \begin{array}{l} m \neq \text{shared} \\ \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \gamma'_s = (\Delta[x \mapsto t[e_1] \dots [e_n]], b_l, (b_f, t_f)) \\ ? : \text{size inferred from } e \end{array}$$

$$[\text{ARR-DEC-SIZE-ASS-SHARED}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash (e_1, \dots, e_n) : \bar{\text{Int}} \quad (\gamma_s, \gamma_g) \vdash e_v : t[?] \quad (\gamma_s, \gamma_g[x \mapsto t[e_1] \dots [e_n]]) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash m \, t[e_1] \dots [e_n] x := e_v; \sigma : \text{ok}} \quad \begin{array}{l} m = \text{shared} \\ x \notin \gamma_g \\ ? : \text{size inferred from } e \end{array}$$

$$[\text{ARR-INIT}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash (e_1, \dots, e_n) : \bar{\text{Int}}}{(\gamma_s, \gamma_g) \vdash x = [e_1] \dots [e_n] : \text{ok}} \quad \begin{array}{l} \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \Delta(x) = t[\varepsilon] \end{array}$$

$$[\text{ARR-INIT-SHARED}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash (e_1, \dots, e_n) : \bar{\text{Int}}}{(\gamma_s, \gamma_g) \vdash x = [e_1] \dots [e_n] : \text{ok}} \quad \begin{array}{l} x \notin \gamma_s \\ \gamma_g(x) = t[\varepsilon] \end{array}$$

$$[\text{ASS-ARR}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash (e_1, \dots, e_n) : \bar{\text{Int}} \quad (\gamma_s, \gamma_g) \vdash e_v : t}{(\gamma_s, \gamma_g) \vdash x[e_1] \dots [e_n] = e_v : \text{ok}} \quad \begin{array}{l} \gamma_s = (\Delta, b_l, (b_f, t_f)) \\ \Delta(x) = t[k] \end{array}$$

$$[\text{ASS-ARR-SHARED}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash (e_1, \dots, e_n) : \bar{\text{Int}} \quad (\gamma_s, \gamma_g) \vdash e_v : t}{(\gamma_s, \gamma_g) \vdash x[e_1] \dots [e_n] = e_v : \text{ok}} \quad \begin{array}{l} x \notin \gamma_s \\ \gamma_g(x) = t[k] \end{array}$$

$$[\text{AWAIT-ALL}_{TS}] \frac{\forall i \in \{1, \dots, n\} \quad \gamma_g(\tau_i) = \text{thread}}{(\gamma_s, \gamma_g) \vdash \text{awaitAll}(\tau_1, \dots, \tau_n) : \text{ok}}$$

$$[\text{AWAIT-ANY}_{TS}] \frac{\forall i \in \{1, \dots, n\} \quad \gamma_g(\tau_i) = \text{thread}}{(\gamma_s, \gamma_g) \vdash \text{awaitAny}(\tau_1, \dots, \tau_n) : \text{ok}}$$

NEW CHANGE HERE! 

$$[\text{CRITSEC}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash (x_1, \dots, x_n) : \bar{t} \quad (\gamma_s, \gamma_g) \vdash \sigma : \text{ok}}{(\gamma_s, \gamma_g) \vdash \text{critical}(x_1, \dots, x_n) \sigma : \text{ok}} \quad x_i \notin \gamma_s \quad \forall i \in \{1, \dots, n\}$$

$$[\text{CONTINUE}_{TS}] \frac{}{(\gamma_s, \gamma_g) \vdash \text{continue} : \text{ok}} \quad \gamma_s = (\Delta, tt, (b_f, t))$$

$$[\text{BREAK}_{TS}] \frac{}{(\gamma_s, \gamma_g) \vdash \text{break} : \text{ok}} \quad \gamma_s = (\Delta, tt, (b_f, t))$$

$$[\text{RETURN-VOID}_{TS}] \frac{}{(\gamma_s, \gamma_g) \vdash \text{return} : \text{ok}} \quad \gamma_s = (\Delta, b_l, (tt, \text{Void}))$$

$$[\text{RETURN}_{TS}] \frac{(\gamma_s, \gamma_g) \vdash e : t}{(\gamma_s, \gamma_g) \vdash \text{return } e : \text{ok}} \quad \begin{array}{l} \gamma_s = (\Delta, b_l, (tt, t)) \\ t \neq \text{Void} \end{array}$$

Big Step

$$(\text{PROGRAM}_{BS}) \frac{\langle \sigma, s_0, g_0 \rangle \rightarrow (s', g')}{\langle \sigma \rangle \rightarrow g'} \quad \begin{array}{l} s_0 = [\mu^O \mapsto \emptyset] \\ g_0 = [\tau_0 \mapsto \text{thread}(ff, \sigma, s_0)] \end{array}$$

Expressions

$$\mathbf{Format:} \quad (s, g) \vdash e \rightarrow (v, g') \quad (s \in S, \quad g, g' \in G, \quad e \in E, \quad v \in V)$$

$$(\text{INT}_{BS}) \frac{}{(s, g) \vdash n \rightarrow (n, g)} \quad n \in \mathbb{Z} \qquad (\text{FLOAT}_{BS}) \frac{}{(s, g) \vdash \phi \rightarrow (\phi, g)} \quad \phi \in \mathbb{R}$$

$$(\text{BOOL}_{BS}) \frac{}{(s, g) \vdash b \rightarrow (b, g)} \quad b \in \{tt, ff\} \qquad (\text{CHAR}_{BS}) \frac{}{(s, g) \vdash 'c' \rightarrow (c, g)} \quad c \in \text{Char}$$

$$(\text{STRING}_{BS}) \frac{}{(s, g) \vdash \text{"str"} \rightarrow (str, g)} \quad str \in \text{String}$$

$$(\text{THREAD}_{BS}) \frac{}{(s, g) \vdash \tau \rightarrow (b, g)} \quad b \in \{tt, ff\}$$

$$(\text{VAR}_{BS}) \frac{s(x) = l \quad g(l) = v}{(s, g) \vdash x \rightarrow (v, g)} \qquad (\text{VAR-GLOBAL}_{BS}) \frac{g(x) = v}{(s, g) \vdash x \rightarrow (v, g)} \quad x \notin s$$

$$(\text{PAR}_{BS}) \frac{(s, g) \vdash e \rightarrow (v, g')}{(s, g) \vdash (e) \rightarrow (v, g')}$$

$$(\star_{BS}) \frac{(s, g) \vdash e_1 \rightarrow (v_1, g') \quad (s, g') \vdash e_2 \rightarrow (v_2, g'')}{(s, g) \vdash e_1 \star e_2 \rightarrow (v, g'')} \quad \begin{array}{l} \star \in \left\{ +, -, *, /, \%, \wedge, \&\&, ||, \right\} \\ v = v_1 \star v_2 \end{array}$$

$$\begin{array}{c}
(\text{ARGS}_{BS}) \frac{(s, g) \vdash e_1 \rightarrow (v_1, g') \quad (s, g') \vdash (e_2, \dots, e_n) \rightarrow (\bar{v}', g'')}{(s, g) \vdash (e_1, \dots, e_n) \rightarrow (\bar{v}, g'')} \quad \bar{v} = v_1 \oplus \bar{v}' \\
\\
(\text{ARGS-EMPTY}_{BS}) \frac{}{(s, g) \vdash () \rightarrow (\epsilon, g)} \\
\\
(\text{FUN-CALL}_{BS}) \frac{s(f) = \text{func}(\bar{p}, \sigma) \quad (s, g) \vdash (e_1, \dots, e_n) \rightarrow (\bar{v}, g') \quad \langle \sigma, s_\emptyset[\bar{p} \mapsto \bar{l}], g'[\bar{l} \mapsto \bar{v}] \rangle \rightarrow (\text{ret}(v), g'')}{(s, g) \vdash f(e_1, \dots, e_n) \rightarrow (v, g''')} \quad \begin{array}{l} g''' = g'' \setminus \bar{l} \\ \forall l \in \bar{l} \quad l \notin g' \end{array} \\
\\
(\text{FUN-CALL-EMPTY}_{BS}) \frac{s(f) = \text{func}(\epsilon, \sigma) \quad \langle \sigma, s_\emptyset, g \rangle \rightarrow (\text{ret}(v), g')}{(s, g) \vdash f() \rightarrow (v, g')} \\
\\
(\text{ARR-LIT}_{BS}) \frac{(s, g) \vdash e_1 \rightarrow (v_1, g') \quad (s, g') \vdash \{e_2, \dots, e_n\} \rightarrow (\bar{a}', g'')}{(s, g) \vdash \{e_1, \dots, e_n\} \rightarrow (\bar{a}, g'')} \quad \bar{a} = v_1 \oplus \bar{a}' \\
\\
(\text{ARR-LIT-EMPTY}_{BS}) \frac{}{(s, g) \vdash \{ \} \rightarrow (\epsilon, g)} \\
\\
(\text{ARR-INDEX}_{BS}) \frac{(s, g) \vdash (e_1, \dots, e_n) \rightarrow ((v_1, \dots, v_n), g')}{(s, g) \vdash x[e_1] \dots [e_n] \rightarrow (v, g')} \quad \begin{array}{l} s(x) = l \\ \forall i \in \{1, \dots, n\} \quad v_i \geq 0 \\ g'(l[v_1] \dots [v_n]) = v \end{array} \\
\\
(\text{ARR-INDEX-SHARED}_{BS}) \frac{(s, g) \vdash (e_1, \dots, e_n) \rightarrow ((v_1, \dots, v_n), g')}{(s, g) \vdash x[e_1] \dots [e_n] \rightarrow (v, g')} \quad \begin{array}{l} x \notin s \\ \forall i \in \{1, \dots, n\} \quad v_i \geq 0 \\ g'(x[v_1] \dots [v_n]) = v \end{array}
\end{array}$$

Arr-Index also uses Args and Args-Empty to evaluate the list of expressions to indices.

$$\begin{array}{c}
(\text{READ}_{BS}) \frac{}{(s, g) \vdash \text{read}(t) \rightarrow (v, g)} \\
\\
(\text{CAST-FI}_{BS}) \frac{(s, g) \vdash e \rightarrow (\phi, g')}{(s, g) \vdash \text{cast}(\text{Int}) e \rightarrow (n, g')} \quad \begin{array}{l} \phi \in \mathbb{R} \\ n \in \mathbb{Z} \\ n = \lfloor \phi \rfloor \end{array}
\end{array}$$

$$\begin{array}{c}
 (\text{CAST-BI-T}_{BS}) \frac{(s, g) \vdash e \rightarrow (tt, g')}{(s, g) \vdash \text{cast}(\text{Int}) e \rightarrow (1, g')} \quad \begin{array}{l} b \in \{tt, ff\} \\ n \in \mathbb{Z} \end{array} \\
 (\text{CAST-BI-F}_{BS}) \frac{(s, g) \vdash e \rightarrow (ff, g')}{(s, g) \vdash \text{cast}(\text{Int}) e \rightarrow (0, g')} \quad \begin{array}{l} b \in \{tt, ff\} \\ n \in \mathbb{Z} \end{array} \\
 (\text{CAST-IB-T}_{BS}) \frac{(s, g) \vdash e \rightarrow (n, g')}{(s, g) \vdash \text{cast}(\text{Bool}) e \rightarrow (tt, g')} \quad \begin{array}{l} n \in \mathbb{Z} \\ b \in \{tt, ff\} \\ n \neq 0 \end{array} \\
 (\text{CAST-IB-F}_{BS}) \frac{(s, g) \vdash e \rightarrow (0, g')}{(s, g) \vdash \text{cast}(\text{Bool}) e \rightarrow (ff, g')} \quad \begin{array}{l} n \in \mathbb{Z} \\ b \in \{tt, ff\} \end{array} \\
 (\text{CAST-FB-T}_{BS}) \frac{(s, g) \vdash e \rightarrow (\phi, g')}{(s, g) \vdash \text{cast}(\text{Bool}) e \rightarrow (tt, g')} \quad \begin{array}{l} \phi \in \mathbb{R} \\ b \in \{tt, ff\} \\ \phi \neq 0.0 \end{array} \\
 (\text{CAST-FB-F}_{BS}) \frac{(s, g) \vdash e \rightarrow (0.0, g')}{(s, g) \vdash \text{cast}(\text{Bool}) e \rightarrow (ff, g')} \quad \begin{array}{l} \phi \in \mathbb{R} \\ b \in \{tt, ff\} \end{array} \\
 (\text{CAST-IF}_{BS}) \frac{(s, g) \vdash e \rightarrow (n, g')}{(s, g) \vdash \text{cast}(\text{Float}) e \rightarrow (\phi, g')} \quad \begin{array}{l} n \in \mathbb{Z} \\ \phi \in \mathbb{R} \\ \phi = \text{float}(n) \end{array} \\
 (\text{CAST-BF-T}_{BS}) \frac{(s, g) \vdash e \rightarrow (tt, g')}{(s, g) \vdash \text{cast}(\text{Float}) e \rightarrow (1.0, g')} \quad \begin{array}{l} b \in \{tt, ff\} \\ \phi \in \mathbb{R} \end{array} \\
 (\text{CAST-BF-F}_{BS}) \frac{(s, g) \vdash e \rightarrow (ff, g')}{(s, g) \vdash \text{cast}(\text{Float}) e \rightarrow (0.0, g')} \quad \begin{array}{l} b \in \{tt, ff\} \\ \phi \in \mathbb{R} \end{array} \\
 (\text{CAST-CS}_{BS}) \frac{(s, g) \vdash e \rightarrow (c, g')}{(s, g) \vdash \text{cast}(\text{String}) e \rightarrow (str, g')} \quad \begin{array}{l} c \in \text{Char} \\ str \in \text{String} \\ str = \text{str}(c) \end{array}
 \end{array}$$

$$(\text{CAST-IS}_{BS}) \frac{(s, g) \vdash e \rightarrow (n, g')}{(s, g) \vdash \text{cast}(\text{String}) e \rightarrow (str, g')} \quad \begin{array}{l} n \in \mathbb{Z} \\ str \in \text{String} \\ str = \text{str}(n) \end{array}$$

$$(\text{CAST-FS}_{BS}) \frac{(s, g) \vdash e \rightarrow (\phi, g')}{(s, g) \vdash \text{cast}(\text{String}) e \rightarrow (str, g')} \quad \begin{array}{l} \phi \in \mathbb{R} \\ str \in \text{String} \\ str = \text{str}(\phi) \end{array}$$

$$(\text{CAST-BS-T}_{BS}) \frac{(s, g) \vdash e \rightarrow (tt, g')}{(s, g) \vdash \text{cast}(\text{String}) e \rightarrow (\text{"true"}, g')} \quad \begin{array}{l} b \in \{tt, ff\} \\ str \in \text{String} \end{array}$$

$$(\text{CAST-BS-F}_{BS}) \frac{(s, g) \vdash e \rightarrow (ff, g')}{(s, g) \vdash \text{cast}(\text{String}) e \rightarrow (\text{"false"}, g')} \quad \begin{array}{l} b \in \{tt, ff\} \\ str \in \text{String} \end{array}$$

Statements

Format: $\langle \sigma, s, g \rangle \rightarrow (r, g') \quad (s \in S, \quad g, g' \in G \quad \sigma \in \Sigma, \quad r \in \mathcal{R})$

$$(\text{COMP}_{BS}) \frac{\langle \sigma_1, s, g \rangle \rightarrow (s', g') \quad \langle \sigma_2, s', g' \rangle \rightarrow (s'', g'')}{\langle \sigma_1; \sigma_2, s, g \rangle \rightarrow (s'', g')}$$

$$(\text{COMP-ABORT}_{BS}) \frac{\langle \sigma_1, s, g \rangle \rightarrow r(s', g')}{\langle \sigma_1; \sigma_2, s, g \rangle \rightarrow r(s', g')} \quad r \in \{\text{cnt}, \text{brk}, \text{ret}\}$$

$$(\text{BLOCK}_{BS}) \frac{\langle \sigma, s, g \rangle \rightarrow (s', g')}{\langle \{\sigma\}, s, g \rangle \rightarrow (s, g')}$$

$$(\text{EXP}_{BS}) \frac{(s, g) \vdash e \rightarrow (v, g')}{\langle e, s, g \rangle \rightarrow (s, g')}$$

$$(\text{DEC-EMPTY}_{BS}) \frac{\langle \sigma, s[x \mapsto l], g[l \mapsto \varepsilon] \rangle \rightarrow (s', g')}{\langle m \text{ } t \text{ } x; \sigma, s, g \rangle \rightarrow (s', g')} \quad m \neq \text{shared}$$

$$(\text{DEC}_{BS}) \frac{(s, g) \vdash e \rightarrow (v, g') \quad \langle \sigma, s[x \mapsto l], g'[l \mapsto v] \rangle \rightarrow (s', g'')}{\langle m \text{ } t \text{ } x := e; \sigma, s, g \rangle \rightarrow (s', g'')} \quad m \neq \text{shared}$$

$$(\text{DEC-SHARED-EMPTY}_{BS}) \frac{\langle \sigma, s, g[x \mapsto \varepsilon][\mu_x \mapsto \text{free}] \rangle \rightarrow (s', g')}{\langle m \text{ } t \text{ } x; \sigma, s, g \rangle \rightarrow (s', g')} \quad m = \text{shared}$$

$$(\text{DEC-SHARED}_{BS}) \frac{(s, g) \vdash e \rightarrow (v, g') \quad \langle \sigma, s, g'[x \mapsto v][\mu_x \mapsto \text{free}] \rangle \rightarrow (s', g'')}{\langle m \text{ } t \text{ } x := e; \sigma, s, g \rangle \rightarrow (s', g'')} \quad m = \text{shared}$$

$$(\text{ASS}_{BS}) \frac{s(x) = l \quad (s, g) \vdash e \rightarrow (v, g')}{\langle x = e, s, g \rangle \rightarrow (s, g'[l \mapsto v])}$$

$$(\text{ASS-SHARED}_{BS}) \frac{(s, g) \vdash e \rightarrow (v, g')}{\langle x = e, s, g \rangle \rightarrow (s, g'[x \mapsto v])} \quad x \notin s$$

$$(\text{THREAD-SPAWN}_{BS}) \frac{\langle \sigma_c, s, g[\tau \mapsto \text{thread}(ff, \sigma_b, s)] \rangle \rightarrow (s', g')}{\langle \text{Thread } \tau \Rightarrow \sigma_b; \sigma_c, s, g \rangle \rightarrow (s', g')}$$

$\tau \notin g$
thread τ runs parallel
when τ reaches a final state,
 g'' is the global state, and
 $g''' = g''[\tau \mapsto \text{thread}(tt, \sigma_b, s)]$

$$(\text{FUN-DEF}_{BS}) \frac{\langle \sigma, s[f \mapsto \text{func}(\bar{p}, \sigma_f)], g \rangle \rightarrow (s', g')}{\langle t f(m_1 t_1 p_1, \dots, m_n t_n p_n) \sigma_f; \sigma, s, g \rangle \rightarrow (s', g')}$$

$$(\text{PRINT}_{BS}) \frac{(s, g) \vdash (e_1, \dots, e_n) \rightarrow (\bar{v}, g')}{\langle \text{print}(e_1, \dots, e_n), s, g \rangle \rightarrow (s, g')}$$

$$(\text{IF-ELSE-TRUE}_{BS}) \frac{(s, g) \vdash e \rightarrow (tt, g') \quad \langle \sigma_1, s, g' \rangle \rightarrow (s', g'')}{\langle \text{if } e \text{ then } \sigma_1 \text{ else } \sigma_2, s, g \rangle \rightarrow (s', g')}$$

$$(\text{IF-ELSE-FALSE}_{BS}) \frac{(s, g) \vdash e \rightarrow (ff, g') \quad \langle \sigma_2, s, g' \rangle \rightarrow (s', g'')}{\langle \text{if } e \text{ then } \sigma_1 \text{ else } \sigma_2, s, g \rangle \rightarrow (s', g')}$$

$$(\text{IF-THEN}_{BS}) \frac{(s, g) \vdash e \rightarrow (tt, g') \quad \langle \sigma, s, g' \rangle \rightarrow (s', g'')}{\langle \text{if } e \text{ then } \sigma, s, g \rangle \rightarrow (s, g')}$$

$$(\text{IF-SKIP}_{BS}) \frac{(s, g) \vdash e \rightarrow (ff, g')}{\langle \text{if } e \text{ then } \sigma, s, g \rangle \rightarrow (s, g')}$$

$$(WHILE-TRUE_{BS}) \frac{(s, g) \vdash e \rightarrow (tt, g') \quad \langle \sigma, s, g' \rangle \rightarrow (s', g'') \quad \langle \text{while } e \text{ do } \sigma, s', g'' \rangle \rightarrow (s'', g''')}{\langle \text{while } e \text{ do } \sigma, s, g \rangle \rightarrow (s'', g''')}$$

$$(WHILE-TRUE-CNT_{BS}) \frac{(s, g) \vdash e \rightarrow (tt, g') \quad \langle \sigma, s, g' \rangle \rightarrow \text{cnt}(s', g'') \quad \langle \text{while } e \text{ do } \sigma, s', g'' \rangle \rightarrow (s'', g''')}{\langle \text{while } e \text{ do } \sigma, s, g \rangle \rightarrow (s'', g''')}$$

$$(WHILE-TRUE-BRK_{BS}) \frac{(s, g) \vdash e \rightarrow (tt, g') \quad \langle \sigma, s, g' \rangle \rightarrow \text{brk}(s', g'')}{\langle \text{while } e \text{ do } \sigma, s, g \rangle \rightarrow (s', g'')}$$

$$(WHILE-TRUE-RET_{BS}) \frac{(s, g) \vdash e \rightarrow (tt, g') \quad \langle \sigma, s, g' \rangle \rightarrow \text{ret}(v, s', g'')}{\langle \text{while } e \text{ do } \sigma, s, g \rangle \rightarrow \text{ret}(v, s', g'')}$$

$$(WHILE-TRUE-RET-VOID_{BS}) \frac{(s, g) \vdash e \rightarrow (tt, g') \quad \langle \sigma, s, g' \rangle \rightarrow \text{ret}(s', g'')}{\langle \text{while } e \text{ do } \sigma, s, g \rangle \rightarrow \text{ret}(s', g'')}$$

$$(WHILE-FALSE_{BS}) \frac{(s, g) \vdash e \rightarrow (ff, g')}{\langle \text{while } e \text{ do } \sigma, s, g \rangle \rightarrow (s, g')}$$

$$(FOR-DEC_{BS}) \frac{(s, g) \vdash e_1 \rightarrow (v, g') \quad \langle \text{while } e_c \text{ do } (\sigma_b; \sigma_r), s', g' \rangle \rightarrow (s'', g'')}{\langle \text{for}(m \text{ } t \text{ } x := e_1; e_c; \sigma_r) \sigma_b, s, g \rangle \rightarrow (s'', g'')} \quad \begin{array}{l} m \neq \text{shared} \\ s' = s[x \mapsto v] \end{array}$$

$$(FOR-DEC-SHARED_{BS}) \frac{(s, g) \vdash e_1 \rightarrow (v, g') \quad \langle \text{while } e_c \text{ do } (\sigma_b; \sigma_r), s, g'' \rangle \rightarrow (s', g''')}{\langle \text{for}(m \text{ } t \text{ } x := e_1; e_c; \sigma_r) \sigma_b, s, g \rangle \rightarrow (s', g''')} \quad \begin{array}{l} m = \text{shared} \\ g'' = g'[x \mapsto v] \end{array}$$

$$(FOR-ASS_{BS}) \frac{\langle x = e_1, s, g \rangle \rightarrow (s', g') \quad \langle \text{while } e_c \text{ do } (\sigma_b; \sigma_r), s', g' \rangle \rightarrow (s'', g'')}{\langle \text{for}(x = e_1; e_c; \sigma_r) \sigma_b, s, g \rangle \rightarrow (s'', g'')}$$

$$(FOR-ID_{BS}) \frac{\langle \text{while } e \text{ do } (\sigma_b; \sigma_r), s, g \rangle \rightarrow (s', g')}{\langle \text{for}(x; e; \sigma_r) \sigma_b, s, g \rangle \rightarrow (s', g')}$$

NEW CHANGE HERE! 

$$(\text{ARR-DEC-EMPTY}_{BS}) \frac{\langle \sigma, s[x \mapsto l], g[l \mapsto \varepsilon] \rangle \rightarrow (s', g')}{\langle m \ t[] \dots [] \ x; \sigma, s, g \rangle \rightarrow (s', g')} \quad m \neq \text{shared}$$

$$(\text{ARR-DEC-SHARED-EMPTY}_{BS}) \frac{\langle \sigma, s, g[x \mapsto \varepsilon] \rangle \rightarrow (s', g')}{\langle m \ t[] \dots [] \ x; \sigma, s, g \rangle \rightarrow (s', g')} \quad m = \text{shared}$$

NEW CHANGE HERE! 

$$(\text{ARR-DEC}_{BS}) \frac{(s, g) \vdash (e_1, \dots, e_n) \rightarrow ((v_1, \dots, v_n), g') \quad \langle \sigma, s[x \mapsto l], g'[l \mapsto \varepsilon] \rangle \rightarrow (s', g'')}{\langle m \ t[e_1] \dots [e_n] \ x; \sigma, s, g \rangle \rightarrow (s', g'')} \quad \begin{array}{l} m \neq \text{shared} \\ \forall i \in \{1, \dots, n\} \quad v_i > 0 \end{array}$$

$$(\text{ARR-DEC-SHARED}_{BS}) \frac{(s, g) \vdash (e_1, \dots, e_n) \rightarrow ((v_1, \dots, v_n), g') \quad \langle \sigma, s, g'[x \mapsto \varepsilon] \rangle \rightarrow (s', g'')}{\langle m \ t[e_1] \dots [e_n] \ x; \sigma, s, g \rangle \rightarrow (s', g'')} \quad \begin{array}{l} m = \text{shared} \\ \forall i \in \{1, \dots, n\} \quad v_i > 0 \end{array}$$

NEW CHANGE HERE! 

$$(\text{ARR-DEC-ASS}_{BS}) \frac{(s, g) \vdash e \rightarrow (\bar{a}, g') \quad \langle \sigma, s[x \mapsto l], g'[l \mapsto \bar{a}] \rangle \rightarrow (s', g'')}{\langle m \ t[] \dots [] \ x := e; \sigma, s, g \rangle \rightarrow (s', g'')} \quad m \neq \text{shared}$$

$$(\text{ARR-DEC-ASS-SHARED}_{BS}) \frac{(s, g) \vdash e \rightarrow (\bar{a}, g') \quad \langle \sigma, s, g'[x \mapsto \bar{a}] \rangle \rightarrow (s', g'')}{\langle m \ t[] \dots [] \ x := e; \sigma, s, g \rangle \rightarrow (s', g'')} \quad m = \text{shared}$$

NEW CHANGE HERE! 

$$\begin{array}{c}
(s, g) \vdash (e_1, \dots, e_n) \rightarrow ((v_1, \dots, v_n), g') \\
(s, g') \vdash e_v \rightarrow (\bar{a}, g'') \\
(\text{ARR-DEC-SIZE-ASS}_{BS}) \frac{\langle \sigma, s[x \mapsto l], g''[l \mapsto \bar{a}] \rangle \rightarrow (s', g''')}{\langle m \ t[e_1] \dots [e_n] \ x := e_v; \sigma, s, g \rangle \rightarrow (s', g''')} \quad \begin{array}{l} m \neq \text{shared} \\ \forall i \in \{1, \dots, n\} \quad v_i > 0 \end{array} \\
\\
(s, g) \vdash (e_1, \dots, e_n) \rightarrow ((v_1, \dots, v_n), g') \\
(s, g') \vdash e_v \rightarrow (\bar{a}, g'') \\
(\text{ARR-DEC-SIZE-ASS-SHARED}_{BS}) \frac{\langle \sigma, s, g''[x \mapsto \bar{a}] \rangle \rightarrow (s', g''')}{\langle m \ t[e_1] \dots [e_n] \ x := e_v; \sigma, s, g \rangle \rightarrow (s', g''')} \quad \begin{array}{l} m = \text{shared} \\ \forall i \in \{1, \dots, n\} \quad v_i > 0 \end{array}
\end{array}$$

NEW CHANGE HERE! 

$$\begin{array}{c}
(\text{ARR-INIT}_{BS}) \frac{(s, g) \vdash (e_1, \dots, e_n) \rightarrow ((v_1, \dots, v_n), g')}{\langle x = [e_1] \dots [e_n], s, g \rangle \rightarrow (s, g'')} \quad \begin{array}{l} \forall i \in \{1, \dots, n\} \quad v_i > 0 \\ s(x) = l \\ g(l) = \varepsilon \\ g'' = g'[l \mapsto [v_1] \dots [v_n]] \end{array} \\
\\
(\text{ARR-INIT-SHARED}_{BS}) \frac{(s, g) \vdash (e_1, \dots, e_n) \rightarrow ((v_1, \dots, v_n), g')}{\langle x = [e_1] \dots [e_n], s, g \rangle \rightarrow (s, g'')} \quad \begin{array}{l} \forall i \in \{1, \dots, n\} \quad v_i > 0 \\ x \notin s \\ g(x) = \varepsilon \\ g'' = g'[x \mapsto [v_1] \dots [v_n]] \end{array}
\end{array}$$

NEW CHANGE HERE! 

$$\begin{array}{c}
(\text{ASS-ARR}_{BS}) \frac{(s, g) \vdash (e_1, \dots, e_n) \rightarrow ((v_1, \dots, v_n), g') \quad (s, g') \vdash e_v \rightarrow (v, g'')}{\langle x[e_1] \dots [e_n] = e_v, s, g \rangle \rightarrow (s, g''')} \quad \begin{array}{l} \forall i \in \{1, \dots, n\} \quad v_i > 0 \\ s(x) = l \\ g''' = g''[l[v_1] \dots [v_n] \mapsto v] \end{array} \\
\\
(\text{ASS-ARR-SHARED}_{BS}) \frac{(s, g) \vdash (e_1, \dots, e_n) \rightarrow ((v_1, \dots, v_n), g') \quad (s, g') \vdash e_v \rightarrow (v, g'')}{\langle x[e_1] \dots [e_n] = e_v, s, g \rangle \rightarrow (s, g''')} \quad \begin{array}{l} \forall i \in \{1, \dots, n\} \quad v_i > 0 \\ x \notin s \\ g''' = g''[x[v_1] \dots [v_n] \mapsto v] \end{array}
\end{array}$$

NEW CHANGE HERE! 

$$(\text{AWAIT-ALL-DONE}_{BS}) \frac{\forall i \in \{1, \dots, n\} \quad g(\tau_i) = \text{thread}(tt, \sigma_t, s_t)}{\langle \text{awaitAll}(\tau_1, \dots, \tau_n), s, g \rangle \rightarrow (s, g)}$$

This rule fires only when all threads have finished; until then, this thread blocks waiting for the global state to be updated.

NEW CHANGE HERE!

$$(\text{AWAIT-ANY-DONE}_{BS}) \frac{\exists i \in \{1, \dots, n\} \quad g(\tau_i) = \text{thread}(tt, \sigma_t, s_t)}{\langle \text{awaitAny}(\tau_1, \dots, \tau_n), s, g \rangle \rightarrow (s, g)}$$

This rule fires only when at least one thread has finished; until then, this thread blocks waiting for the global state to be updated.

NEW CHANGE HERE!

$$(\text{CRIT-SEC-NO-REORDER}_{BS}) \frac{\begin{array}{l} \forall x_i \in N, \forall x_j \in s(\mu^O) \quad \text{ord}(x_i) > \text{ord}(x_j) \\ \forall x_i \in N \quad g(\mu_{x_i}) = \text{free} \\ \langle \sigma, s, g' \rangle \rightarrow (s', g'') \end{array}}{\langle \text{critical}(x_1, \dots, x_n) \sigma, s, g \rangle \rightarrow (s', g''')} \quad \begin{array}{l} \tau = \text{current thread} \\ N = \{x_1, \dots, x_n\} \setminus s(\mu^O) \\ g' = g[\mu_{x_i} \mapsto \tau \mid x_i \in N] \\ g''' = g''[\mu_{x_i} \mapsto \text{free} \mid x_i \in N] \end{array}$$

Locks in N are acquired one at a time in ord-order; if the next lock is held by another thread, this thread blocks until it is released before proceeding.

$$(\text{CRIT-SEC-REORDER}_{BS}) \frac{\begin{array}{l} \exists x_i \in N, x_j \in s(\mu^O) \quad \text{ord}(x_i) < \text{ord}(x_j) \\ \forall x_i \in \bar{x} \quad g'(\mu_{x_i}) = \text{free} \\ \langle \sigma, s, g'' \rangle \rightarrow (s', g''') \end{array}}{\langle \text{critical}(x_1, \dots, x_n) \sigma, s, g \rangle \rightarrow (s', g''')} \quad \begin{array}{l} \tau = \text{current thread} \\ N = \{x_1, \dots, x_n\} \setminus s(\mu^O) \\ \bar{x} = \text{sort}(N \cup s(\mu^O)) \\ g' = g[\mu_{x_j} \mapsto \text{free} \mid x_j \in s(\mu^O)] \\ g'' = g'[\mu_{x_i} \mapsto \tau \mid x_i \in \bar{x}] \\ g''' = g''[\mu_{x_i} \mapsto \text{free} \mid x_i \in \bar{x}] \end{array}$$

Outer-held locks are released first, then all locks in $\bar{x}$ are acquired one at a time in ord-order; if the next lock is held by another thread, this thread blocks until it is released before proceeding.

$$\begin{array}{c}
 \forall x_i \in \bar{x} \quad g(\mu_{x_i}) = \tau \\
 \langle \sigma, s, g \rangle \rightarrow (s', g') \\
 \text{(CRIT-SEC-RUN}_{BS}) \frac{}{\langle \text{critical}(x_1, \dots, x_n) \sigma, s, g \rangle \rightarrow (s', g'')}
 \end{array}
 \quad
 \begin{array}{l}
 \tau = \text{current thread} \\
 \bar{x} = \text{sort}(\{x_1, \dots, x_n\} \cup s(\mu^O)) \\
 g'' = g'[\mu_{x_i} \mapsto \text{free} \mid x_i \in \bar{x} \setminus s(\mu^O)]
 \end{array}$$

$$\begin{array}{c}
 \text{(CONTINUE}_{BS}) \frac{}{\langle \text{continue}, s, g \rangle \rightarrow (\text{cnt}, g)} \quad \text{(BREAK}_{BS}) \frac{}{\langle \text{break}, s, g \rangle \rightarrow (\text{brk}, g)} \\
 \text{(RETURN-VOID}_{BS}) \frac{}{\langle \text{return}, s, g \rangle \rightarrow (\text{ret}, g)} \quad \text{(RETURN}_{BS}) \frac{(s, g) \vdash e \rightarrow (v, g')}{\langle \text{return } e, s, g \rangle \rightarrow (\text{ret}(v), g')}
 \end{array}$$

9.3 Acceptance Testing Test Cases

To keep the acceptance tests easy to read, only the parts of the generated Java code that are relevant to each requirement are shown. Repeated elements such as imports and the standard class structure have been left out.

9.3.1 Test Case TC-01

Requirement	Test Case	Description
Requirement 1	TC-01	Verifies that creating a thread in ParaLang produces the correct execution code.
Input		Output
<pre>thread t1 => {}; thread t2 => {};</pre>		<pre>CompletableFuture<Void> t1 = CompletableFuture.runAsync(() ->); CompletableFuture<Void> t2 = CompletableFuture.runAsync(() ->);</pre>

9.3.2 Test Case TC-02

The ParaLang code in sample-input.txt is transpiled by the compiler pipeline, and the resulting Java code is compared against javaOutput.java to verify correct transpilation.

Requirement	Test Case	Description
Requirement 2	TC-02	Verifies that an input text file in ParaLang can be transpiled to a Java file.
Input		Output
<pre>void main() { for (int i = 1; i < 5; i = i + 1) { print(i, "\n"); } print("Out of the loop"); }</pre>		<pre>import java.util.Scanner; import java.util.concurrent.*; import java.util.concurrent.locks.*; import java.util.concurrent.CompletableFuture; import java.lang.Thread; import java.util.Arrays; import java.lang.Math; public class Main { public static void main(String[] args) { Scanner scanner = new Scanner(System.in); for (int i = 1; i < 5; i = i + 1) { System.out.print(i + "\n"); } System.out.print("Out of the loop"); scanner.close(); } }</pre>

9.3.3 Test Case TC-03

Requirement	Test Case	Description
Requirement 5	TC-03	Verifies basic variable declarations using int, float, bool, char, string.
Input		Output
<pre>char c = 'a'; int i = 42; float f = 3.14; bool b = true; string s = "Hello";</pre>		<pre>char c = 'a'; int i = 42; float f = 3.14f; Boolean b = true; String s = "Hello";</pre>

9.3.4 Test Case TC-04

Requirement	Test Case	Description
Requirement 6	TC-04	Verifies the creation of arrays of data types: int, float, bool, char, string.
Input		Output
<pre>int[] numbers = 1, 2, 3; float[] decimals = 1.0, 2.5; bool[] flags = true, false; char[] letters = 'a', 'b', 'c'; string[] words = "hello", "world";</pre>		<pre>int[] numbers = new int[]1, 2, 3; float[] decimals = new float[]1.0f, 2.5f; Boolean[] flags = new Boolean[]true, false; char[] letters = new char[]'a', 'b', 'c'; String[] words = new String[]"hello", "world";</pre>

9.3.5 Test Case TC-05

Requirement	Test Case	Description
Requirement 7	TC-05	Verifies ParaLang support for arithmetic, comparison, logical operators and math functions.
Input		Output
<pre>int add = 5 + 3; int minus = 5 - 3; int mult = 10 * 2; int div = 10 / 2; int mod = 10 % 3; float pow = (3.5 2.5); if (8 == 8) {} if (7 != 8) {} if (true false) {} if (5 < 10) {} if (5 > 10) {} if (true && false) {} if (5 >= 4) {} if (5 <= 4) {}</pre>		<pre>int add = 5 + 3; int minus = 5 - 3; int mult = 10 * 2; int div = 10 / 2; int mod = 10 % 3; float pow = ((float)Math.pow(3.5f, 2.5f)); if (8 == 8) {} if (7 != 8) {} if (true false) {} if (5 < 10) {} if (5 > 10) {} if (true && false) {} if (5 >= 4) {} if (5 <= 4) {}</pre>

9.3.6 Test Case TC-06

Requirement	Test Case	Description
Requirement 8	TC-06	Verifies ParaLang's support for conditional statements, loop statements, variable & function declarations, variable assignments, variable reassignments, console input & output, function call.
Input		Output
<pre> int myFunction(int i) { return i + 1; } void main() { if (5 < 10) {} if (5 < 10) {} else {} if (5 < 10) {} else if (3 > 1) {} else {} for (int i = 0; i < 10; i = i + 1) {} while (5 < 10) {break;} int x; int y = 5; y = 10; print("Hello, World!"); x = read(int); } </pre>		<pre> public class Main { public static int myFunction(int i) { return i + 1; } public static void main(String[] args) { Scanner scanner = new Scanner(System.in); if (5 < 10) {} if (5 < 10) { } else {} if (5 < 10) { } else if (3 > 1) { } else {} for (int i = 0; i < 10; i = i + 1) {} while (5 < 10) { break; } int x = 0; int y = 5; y = 10; int result = myFunction(5); System.out.print("Hello, World!"); x = scanner.nextInt(); scanner.close(); }} </pre>

9.3.7 Test Case TC-07

Requirement	Test Case	Description
Requirement 9	TC-07	Verifies ParaLang's critical section and shared resource synchronization using locks.
Input		Output
<pre>shared int counter = 0; critical(counter) counter = counter + 1;</pre>		<pre>Lock m0 = new ReentrantLock(); int counter = 0; for (double indexer = 100; !m0.tryLock(); indexer = indexer*1.2-((indexer*1.2)%1)) Thread.sleep((long) indexer); try { counter = counter + 1; finally { m0.unlock();</pre>

9.3.8 Test Case TC-08

The lock order is not visible in the generated Java code output; the order was checked manually. Some code formatting has been condensed to improve the fit of the output.

Requirement	Test Case	Description
Requirement 10	TC-08	

Input	Output
<pre> shared int x = 0; shared int y = 0; critical(x, y) { x = x + 1; } critical(y, x) { y = y + 1; } </pre>	<pre> Lock m0 = new ReentrantLock(); // Shared variable: x Lock m1 = new ReentrantLock(); // Shared variable: y int x = 0; int y = 0; { for (double indexer = 100; !m0.tryLock(); indexer = indexer*1.2-((indexer*1.2)%1)) { try { Thread.sleep((long) indexer); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }} for (double indexer = 100; !m1.tryLock(); indexer = indexer*1.2-((indexer*1.2)%1)) { try { Thread.sleep((long) indexer); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }} try { x = x + 1; } finally { m0.unlock(); m1.unlock();}} { for (double indexer = 100; !m0.tryLock(); indexer = indexer*1.2-((indexer*1.2)%1)) { try { Thread.sleep((long) indexer); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }} for (double indexer = 100; !m1.tryLock(); indexer = indexer*1.2-((indexer*1.2)%1)) { try { Thread.sleep((long) indexer); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }} try { y = y + 1; } finally { m0.unlock(); m1.unlock(); }} </pre>

9.3.9 Test Case TC-09

Requirement	Test Case	Description
Requirement 11	TC-09	Verifies basic input and output handling with string concatenation.
Input		Output
<pre>print("Enter your name:"); string name = read(string); print("Hello", name + "!"); print("Age?: "); int age = read(int);</pre>		<pre>System.out.print("Enter your name:"); String name = scanner.nextLine(); System.out.print("Hello" + name + "!"); System.out.print("Age?: "); int age = scanner.nextInt();</pre>

9.3.10 Test Case TC-10

Requirement	Test Case	Description
Requirement 12	TC-10	Verifies the awaitAll construct for synchronizing multiple threads.
Input		Output
<pre>thread t1 => print("in t1") thread t2 => print("in t2") awaitAll(t1, t2);</pre>		<pre>CompletableFuture<Void> t1 = CompletableFuture.runAsync(() -> System.out.print("in t1"));); CompletableFuture<Void> t2 = CompletableFuture.runAsync(() -> System.out.print("in t2"));); CompletableFuture.allOf(t1, t2).get();</pre>

9.3.11 Test Case TC-11

Requirement	Test Case	Description
Requirement 13	TC-11	
Input		Output
<pre>void main() { thread t1 => { print("Thread as data type"); } }</pre>		<pre>CompletableFuture<Void> t1 = CompletableFuture.runAsync(() -> { System.out.print("Thread as data type"); });</pre>

9.3.12 Test Case TC-12

Requirement	Test Case	Description
Requirement 14	TC-12	Verifies explicit type casting between primitive and reference types.
Input		Output
<pre> int srcInt = 32; float srcFloat = 1.0; bool srcBool = true; char srcChar = 'a'; // Int to other types float intToFloat = cast(float) srcInt; bool intToBool = cast(bool) srcInt; string intToString = cast(string) srcInt; // Float to other types int floatToInt = cast(int) srcFloat; bool floatToBool = cast(bool) srcFloat; string floatToString = cast(string) srcFloat; // Bool to other types int boolToInt = cast(int) srcBool; float boolToFloat = cast(float) srcBool; string boolToString = cast(string) srcBool; // Char to string string charToString = cast(string) srcChar; </pre>		<pre> float intToFloat = (float) srcInt; Boolean intToBool = (srcInt == 0) ? false : true; String intToString = String.valueOf(srcInt); int floatToInt = (int) srcFloat; Boolean floatToBool = (srcFloat == 0.0f) ? false : true; String floatToString = String.valueOf(srcFloat); int boolToInt = srcBool ? 1 : 0; float boolToFloat = srcBool ? 1.0f : 0.0f; String boolToString = String.valueOf(srcBool); String charToString = String.valueOf(srcChar); </pre>

9.3.13 Test Case TC-13

Requirement	Test Case	Description
Requirement 15	TC-13	
Input	Output	
<pre> int func1(int a, float b) { return a + cast(int) b; } float func2(float q) { print("q in func2: ", q); return q + 1.5; } shared int q = 10; void main() { int x = func1(5, 3.2); float y = func2(q); print("x after func1: ", x); print("y after func2: ", y); thread t1 => { critical(q) { q = q + 1; } }; thread t2 => { critical(q) { q = q - 1; } }; awaitAll(t1, t2); print("q after threads: ", q); } </pre>	<pre> public static int func1(int a, float b); public static float func2(float q); static int q = 10; Lock m0 = new ReentrantLock(); int x = func1(5, 3.2f); float y = func2(q); CompletableFuture<Void> t1; CompletableFuture<Void> t2; for (double indexer = 100; !m0.tryLock(); Thread.sleep((long) indexer); try { finally { m0.unlock(); CompletableFuture.allOf(t1, t2).get(); </pre>	

9.3.14 Test Case TC-14

Requirement	Test Case	Description
Requirement 16	TC-14	Verifies declaration and handling of shared variables with proper locking mechanism.
Input		Output
shared int counter = 2;		Lock m0 = new ReentrantLock(); int counter = 2;

9.3.15 Test Case TC-15

Requirement	Test Case	Description
Requirement 17	TC-15	Verifies that incorrect ParaLang code produces an error instead of crashing.
Input		Output
void main() int i = 5; if (j < 10)		java.lang.RuntimeException: Variable 'j' not declared.

9.3.16 Test Case TC-16

Requirement	Test Case	Description
Requirement 19	TC-16	Verifies support for static variables, shared variables, constants and main function generation.
Input		Output
static int s = 2; static shared int ss = 3; void main() { shared int counter = 0; const int c = 1; }		static int s = 2; static int ss = 3; public static void main(String[] args) { Lock m0 = new ReentrantLock(); Lock m1 = new ReentrantLock(); int counter = 0; int c = 1; }

9.3.17 Test Case TC-17

Requirement	Test Case	Description
Requirement 20	TC-17	Verifies that single-line and multi-line comments are properly handled (preserved or stripped).
Input		Output
<pre>// This is a single-line comment /* This is a multi-line comment */</pre>		

9.3.18 Test Case TC-18

The lock order is not visible in the generated Java code output; the order was checked manually. Some code formatting has been condensed to improve the fit of the output.

Requirement	Test Case	Description
Requirement 24	TC-18	

Input	Output
<pre> shared int x = 0; shared int y = 0; critical(x, y) { x = x + 1; } critical(y, x) { y = y + 1; } </pre>	<pre> Lock m0 = new ReentrantLock(); // Shared variable: x Lock m1 = new ReentrantLock(); // Shared variable: y int x = 0; int y = 0; { for (double indexer = 100; !m0.tryLock(); indexer = indexer*1.2-((indexer*1.2)%1)) { try { Thread.sleep((long) indexer); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }} for (double indexer = 100; !m1.tryLock(); indexer = indexer*1.2-((indexer*1.2)%1)) { try { Thread.sleep((long) indexer); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }} try { x = x + 1; } finally { m0.unlock(); m1.unlock();}} { for (double indexer = 100; !m0.tryLock(); indexer = indexer*1.2-((indexer*1.2)%1)) { try { Thread.sleep((long) indexer); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }} for (double indexer = 100; !m1.tryLock(); indexer = indexer*1.2-((indexer*1.2)%1)) { try { Thread.sleep((long) indexer); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }} try { y = y + 1; } finally { m0.unlock(); m1.unlock(); }} </pre>

9.3.19 Test Case TC-19

Requirement	Test Case	Description
Requirement 28	TC-19	
Input		Output
<pre>void main() { int x = 5; }</pre>		<pre>public static void main(String[] args) { int x = 5; }</pre>

9.4 ParaLang Factorial code example

```

1  shared int counter = 1;
2  const int LIMIT = 16;
3  int[4] results;
4  int input;
5  int[] arr;
6
7  bool ready = false;
8
9  int worker(int start, int end) {
10     int i = start;
11     int result = counter;
12
13     while (i < end) {
14         result = result * arr[i];
15         i = i + 1;
16     }
17
18     return result;
19 }
20
21 void main() {
22     print("Enter a number:");
23     input = read(int);
24     arr = [input];
25     int index = 0;
26
27     if (input > LIMIT) {
28         print("Input too large, clamping to the LIMIT:", LIMIT, "\n");
29         input = LIMIT;

```

```

30     ready = true;
31 } else {
32     ready = true;
33     print("The input was accepted as is.\n")
34 }
35 if (ready) {
36     for(int i = 0; i < input; i = i + 1) {
37         arr[i] = i + 1;
38     }
39     while(true) {
40         index = index + 1;
41         if (index == 67) {
42             print("SIX SEVEN!!! SIX SEVEN!!! SIX SEVEN!!! SIX SEVEN!!!\n");
43             continue;
44         }
45         thread t1 => {
46             critical(counter) {
47                 counter = worker(0, input / 4);
48             }
49         }
50         thread t2 => {
51             critical(counter) {
52                 counter = worker(input / 4, input / 2);
53             }
54         }
55         thread t3 => {
56             critical(counter) {
57                 counter = worker(input / 2, (3 * input) / 4);
58             }
59         }
60         thread t4 => {
61             critical(counter) {
62                 counter = worker((3 * input) / 4, input);
63             }
64         }
65
66         awaitAll(t1, t2, t3, t4);
67         print("The result of ", index, " is: ", counter, "\n");
68         counter = 1;
69         if (index >= 100) break;
70     }
71 }
72 }

```


9.5 Java Factor Output Code

```

1  import java.util.Scanner;
2  import java.util.concurrent.*;
3  import java.util.concurrent.locks.*;
4  import java.util.concurrent.CompletableFuture;
5  import java.lang.Thread;
6  import java.util.Arrays;
7  import java.lang.Math;
8
9  public class Main {
10     static int counter = 1;
11     static int LIMIT = 16;
12     static int[] results = new int[4];
13     static int input = 0;
14     static int[] arr = new int[input];
15     static Boolean ready = false;
16     public static int worker(int start, int end) {
17         int i = start;
18         int result = counter;
19         while (i < end) {
20             result = result * arr[i];
21             i = i + 1;
22         }
23         return result;
24     }
25     public static void main(String[] args) {
26         Scanner scanner = new Scanner(System.in);
27         Lock m0 = new ReentrantLock(); // Shared variable: counter
28         System.out.print("Enter a number:");
29         input = scanner.nextInt();
30         arr = new int[input];
31         int index = 0;
32         if (input > LIMIT) {
33             System.out.print("Input too large, clamping to the LIMIT:" + LIMIT +
34                 ↵ "\n");
35             input = LIMIT;
36             ready = true;
37         }
38         else {
39             ready = true;
40             System.out.print("The input was accepted as is.\n");
41         }
42         if (ready) {
43             for (int i = 0; i < input; i = i + 1) {

```

```

43     arr[i] = i + 1;
44 }
45 while (true) {
46     index = index + 1;
47     if (index == 67) {
48         System.out.print("SIX SEVEN!!! SIX SEVEN!!! SIX SEVEN!!! SIX
49             ↪ SEVEN!!!\n");
50         continue;
51     }
52     CompletableFuture<Void> t1 = CompletableFuture.runAsync(() -> {
53         {
54             for (double indexer = 100; !m0.tryLock(); indexer =
55                 ↪ indexer*1.2-((indexer*1.2)%1)) {
56                 try {
57                     Thread.sleep((long) indexer);
58                 } catch (InterruptedException e) {
59                     Thread.currentThread().interrupt();
60                 }
61             }
62             try {
63                 counter = worker(0, input / 4);
64             } finally {
65                 m0.unlock();
66             }
67         }
68     });
69     CompletableFuture<Void> t2 = CompletableFuture.runAsync(() -> {
70         {
71             for (double indexer = 100; !m0.tryLock(); indexer =
72                 ↪ indexer*1.2-((indexer*1.2)%1)) {
73                 try {
74                     Thread.sleep((long) indexer);
75                 } catch (InterruptedException e) {
76                     Thread.currentThread().interrupt();
77                 }
78             }
79             try {
80                 counter = worker(input / 4, input / 2);
81             } finally {
82                 m0.unlock();
83             }
84         }
85     });
86     CompletableFuture<Void> t3 = CompletableFuture.runAsync(() -> {
87         {
88             for (double indexer = 100; !m0.tryLock(); indexer =
89                 ↪ indexer*1.2-((indexer*1.2)%1)) {

```

```

86         try {
87             Thread.sleep((long) indexer);
88         } catch (InterruptedException e) {
89             Thread.currentThread().interrupt();
90         }
91     }
92     try {
93         counter = worker(input / 2, (3 * input) / 4);
94     } finally {
95         m0.unlock();
96     }
97 }
98 });
99 CompletableFuture<Void> t4 = CompletableFuture.runAsync(() -> {
100     {
101         for (double indexer = 100; !m0.tryLock(); indexer =
102             ⇨ indexer*1.2-((indexer*1.2)%1)) {
103             try {
104                 Thread.sleep((long) indexer);
105             } catch (InterruptedException e) {
106                 Thread.currentThread().interrupt();
107             }
108             try {
109                 counter = worker((3 * input) / 4, input);
110             } finally {
111                 m0.unlock();
112             }
113         }
114     });
115     try {
116         CompletableFuture.allOf(t1, t2, t3, t4).get();
117     } catch (InterruptedException | ExecutionException e) {
118         e.printStackTrace();
119     }
120     System.out.print("The result of " + index + " is: " + counter +
121         ⇨ "\n");
122     counter = 1;
123     if (index >= 100) break;
124 }
125 }
126 scanner.close();
127 }
128 }
129

```